
linux bytes

The Complete Guide to Linux System
Administration,
DevOps, SRE & FAANG Interview Preparation

By Kallol Chakraborty

First Edition — July 2026

30 Topics Across 4 Phases · 200+ Code Examples · 5 Animated Diagrams

linux bytes

Kallol Chakraborty

Linux System Administration · DevOps · SRE · Cloud Infrastructure

Staff+ Architect's Perspective · FAANG Interview Preparation

Linux Bytes — The Complete Guide

First Edition: July 2026

© 2026 Kallol Chakraborty. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author.

Linux® is the registered trademark of Linus Torvalds in the U.S. and other countries. All other trademarks are the property of their respective owners.

The information in this book is distributed on an "as is" basis, without warranty. While every precaution has been taken in the preparation of this work, the author assumes no responsibility for errors or omissions.

Contents

Part 1 Part I: Foundations

1. History of Linux & Unix
2. Filesystem Hierarchy Standard
3. File Permissions & ACLs
4. CLI Fundamentals
5. Shell Scripting Basics
6. Text Processing: grep, sed, awk
7. Git Version Control

Part 2 Part II: Systems & Operations

8. Process Management & Signals
9. Package Management
10. systemd: The Modern Init System
11. Users, Groups & Authentication
12. SSH: Secure Shell
13. Linux Networking
14. Storage, Filesystems & Disk Management

Part 3 Part III: Advanced Topics

15. Kernel Architecture & Syscalls
16. Memory Management & OOM Handling
17. I/O Stack & Filesystem Performance

- 18. Performance Tuning & Profiling
- 19. BPF & eBPF Deep Dive
- 20. Monitoring & Observability
- 21. GPU Computing on Linux
- 22. Linux Build & Toolchain

Part 4 Part IV: Containers & SRE

- 23. cgroups & Namespaces
- 24. Docker & Container Lifecycle
- 25. Kubernetes for Linux Engineers
- 26. Production Troubleshooting
- 27. System Rescue & Recovery
- 28. Security Hardening
- 29. Linux at FAANG Scale
- 30. FAANG Linux Interview Q&A

PART 1

Foundations

The essential Linux foundations every engineer must know: the filesystem hierarchy, permissions, CLI basics, shell scripting, text processing, and version control with Git.

CHAPTER 1

History of Linux & Unix

Foundations · 4 topics

Linux is the dominant operating system powering 96% of the top 1M web servers, all Android devices, 100% of the TOP500 supercomputers, and the majority of cloud infrastructure. Understanding its origins is essential context for every FAANG interview.

[FAANG](#)[Linux](#)[history](#)[interview](#)

FAANG Architect's Note: When I interview candidates, I ask: 'What makes Linux different from other kernels?' The answer reveals depth. Monolithic kernel (all subsystems in kernel space) vs microkernel — Linux chose performance over isolation. But Linux also has loadable kernel modules, so it's not purely monolithic. The real insight: Linux won because of its **license (GPLv2)**, **community governance (Linus's benevolent dictatorship + maintainer hierarchy)**, and **pragmatic engineering** — prioritize working code over architectural purity.

Interview Angle: 'Why did Linux beat Unix?' — Unix was proprietary, fragmented (SYSV vs BSD), and expensive. Linux was free, ran on commodity x86 hardware, and the community could contribute. The GPL ensured no single company could own it. For FAANG system design, this lesson applies: **community-driven standards (Kubernetes, Docker, Prometheus) often beat proprietary solutions.**

Key Takeaway: Linux is not just an OS — it's an economic and community phenomenon. Understanding this history helps you appreciate why every FAANG company runs Linux internally and expects engineers to be fluent in it.

CHAPTER 2

Filesystem Hierarchy Standard

Foundations · 4 topics

The Filesystem Hierarchy Standard (FHS) defines the directory structure on Linux systems. Knowing where things live — and why — is table-stakes for every FAANG infrastructure interview. When an interviewer says 'your service is logging to `/var/log` and filling up the disk,' you need to know where everything is.

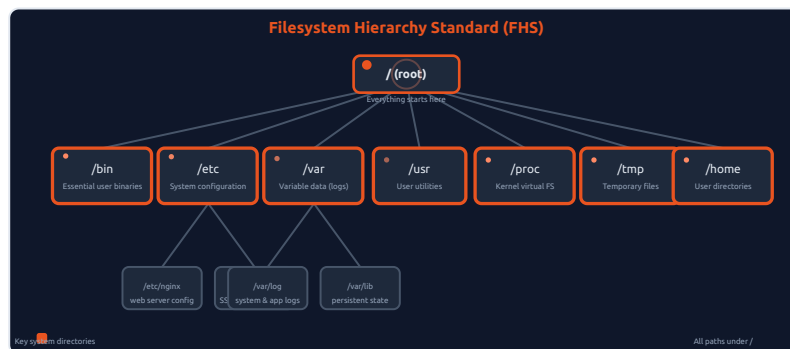
FAANG

Linux

filesystem

interview

The Root Filesystem (/)



Everything on Linux starts at `/`. The root filesystem is the top of the single unified directory tree — there are no drive letters (C:, D:). Every storage device is **mounted** at a directory under `/`. This unified namespace is one of Unix's most important innovations.

```
# Essential root directories at a glance
/bin      # Essential user command binaries (available in single-
user mode)
/sbin     # System administration binaries (e.g., fdisk, mkfs,
iptables)
/etc      # Host-specific system configuration (the nerve center)
/var      # Variable data – logs, databases, spool files
/tmp      # Temporary files (cleared on boot, world-writable)
/usr      # Secondary hierarchy – user utilities and applications
/opt      # Add-on application software packages
/proc     # Virtual filesystem for process and kernel information
/sys      # Virtual filesystem for kernel objects and device info
/dev      # Device files (every hardware device is a file)
/run      # Runtime variable data (PID files, sockets – tmpfs since
boot)
/home     # User home directories
/root     # Root user's home directory

# A FAANG Architect's quick map:
# /etc    → Configuration
# /var    → Data (logs, DBs)
# /proc   → Kernel/process introspection
# /dev    → Hardware as files
# /tmp    → Scratch space (watch for security!)
```

/etc — The Configuration Nerve Center

/etc holds all host-specific configuration. If something isn't working, you start here. In FAANG production environments, **/etc** is typically managed by configuration management (Ansible, Puppet, Chef) and changes are reviewed through code review.

```
ls /etc
# Key files every engineer should know:
/etc/passwd      # User account info (login shells, UIDs)
/etc/shadow      # Encrypted password hashes (root-readable only)
/etc/group       # Group definitions
```

```

/etc/sudoers      # Sudo privileges (edit with visudo!)
/etc/hosts        # Static hostname-to-IP mapping
/etc/resolv.conf  # DNS resolver configuration
/etc/hostname     # System hostname
/etc/fstab        # Filesystem mount table (auto-mount on boot)
/etc/crontab      # System-wide cron schedule
/etc/ssh/sshd_config # SSH server configuration
/etc/nginx/nginx.conf # Nginx configuration (if installed)
/etc/systemd/     # Systemd unit files and system configuration

# Tip: /etc is typically on the root filesystem
# Don't let logs fill up / - it can make the system unbootable

```

/proc & /sys — Virtual Filesystems for Kernel Introspection

`/proc` and `/sys` are virtual filesystems — they don't exist on disk. Reading files in `/proc` triggers kernel code that returns real-time data. Writing to some files changes kernel parameters. These are your window into the running kernel. Every FAANG SRE knows `/proc` intimately.

```

# === /proc - Process and System Information ===
cat /proc/cpuinfo      # CPU details (model, cores, flags, cache)
cat /proc/meminfo      # Memory: total, free, buffers, cache, swap
cat /proc/uptime       # System uptime in seconds
cat /proc/version      # Linux kernel version string
cat /proc/loadavg       # System load averages (1, 5, 15 min)
cat /proc/1/cmdline     # Command line of PID 1 (init/systemd)
cat /proc/self/status  # Current process's status (Name, Pid, VmRSS, ...)
ls /proc/<PID>/fd/       # Open file descriptors for a process
cat /proc/<PID>/maps     # Memory-mapped regions of a process

# === /proc/sys - Kernel Tunables (sysctl interface) ===
cat /proc/sys/net/ipv4/ip_forward # Is IP forwarding

```

```

enabled?
cat /proc/sys/net/ipv4/tcp_tw_reuse      # TCP TIME_WAIT reuse
cat /proc/sys/vm/swappiness              # Swap tendency (0-100)
cat /proc/sys/vm/dirty_ratio             # Max dirty pages %%
before writeback

# === /sys - Device and Driver Model ===
ls /sys/class/block/      # All block devices (sda, nvme0n1, etc.)
ls /sys/class/net/        # All network interfaces (eth0, lo, wlan0)
ls /sys/devices/system/cpu/ # Per-CPU information

# FAANG insight: When debugging production issues,
# /proc/<PID>/ is your first stop for process-level diagnostics

```

/var — Variable Data (Logs, Databases, Spool)

/var holds data that changes in size and content during normal operation. This is where logs go, and disk space monitoring of /var is a standard part of any SRE runbook. **Common FAANG interview scenario:** 'Disk is full, find and clean up /var/log without restarting services.'

```

ls /var
/var/log      # System and application logs (journalctl,
syslog, nginx)
/var/lib      # Persistent state data (databases: mysql,
postgres, docker)
/var/cache    # Application cache data (apt, yum, pip)
/var/spool    # Print and mail spools
/var/tmp      # Temporary files preserved across reboots
/var/opt      # Variable data for /opt packages

# === FAANG Scenario: Log Rotation ===
# Logrotate manages log rotation. Config at /etc/logrotate.conf
# Watch for: services that log too verbosely, filling /var

# Check disk usage by directory:

```

```
du -sh /var/log/* 2>/dev/null | sort -rh | head -10

# Safely truncate a log without restarting:
# (The file descriptor stays valid even after truncation)
> /var/log/myapp/access.log          # Truncate in place
# OR:
truncate -s 0 /var/log/myapp/access.log
```

FAANG Interview Q&A

Q: 'Your production server is running out of disk space on /. Walk through your debugging process.'

A: 'First, `df -h` to see which filesystem is full. Then `du -sh /* | sort -rh` to find the largest top-level directory. Common culprits: /var/log (unrotated logs), /tmp (temp files from batch jobs), Docker overlay storage in /var/lib/docker. For /var/log, check logrotate config. For Docker, prune unused images and containers. In a FAANG environment, we'd have monitoring alerting on disk usage at 80%, 90%, 95% — and automated cleanup via cron or systemd timers.'

Q: 'What's the difference between /proc and /sys?'

A: '/proc was originally process information — the original design was one directory per process (/proc/PID). It evolved to include system info (/proc/cpuinfo, /proc/meminfo, /proc/sys). /sys (sysfs) is newer — it exports the kernel's device and driver model in a structured way. /sys/class organizes by type (block, net, input), /sys/devices by topology. Both are virtual filesystems backed by kernel callbacks. Use /proc for process data and kernel tunables;

use /sys for hardware and driver information.'

Q: 'Why does Linux use forward slashes (/) instead of backslashes (\\)?'

A: 'Historical compatibility with the MULTICS/Unix design choice. Forward slashes were chosen because they didn't conflict with the shell's escape character (backslash). Every major OS except Windows uses forward slashes.'

References & Further Reading

- **Filesystem Hierarchy Standard 3.0** — The official FHS specification
- **man hier** — Description of the filesystem hierarchy on your system
- **Linux Filesystem Hierarchy** — TLDP comprehensive guide

FAANG Architect's Note: The Unix filesystem hierarchy isn't arbitrary — it reflects the system's engineering philosophy: **everything is a file**. Devices (`/dev`), processes (`/proc`), kernel parameters (`/proc/sys`) — all accessible with standard file APIs (`open`, `read`, `write`, `close`). This uniform interface is why you can use the same tools (`grep`, `awk`, `cat`) on hardware status and text files alike.

Production Tip: In containerized environments (Docker/Kubernetes), some of these directories behave differently. `/proc` and `/sys` are typically mounted from the host kernel (read-only for security). Never assume a container has access to all `/proc` entries — `security_context` settings may restrict them.

Interview Tip: When asked 'how do you debug X on Linux,' always start with the filesystem location of the relevant data. Interviewers test your muscle memory — can you navigate to the right `/proc` file or `/var/log` without googling?

CHAPTER 3

File Permissions & ACLs

Foundations · 5 topics

Linux file permissions are the foundation of system security. Every FAANG SRE and infrastructure engineer must understand the permission model cold — including SUID/SGID, sticky bits, ACLs, and how they interact. Permission misconfiguration is a leading cause of security vulnerabilities in production.

FAANG

Linux

permissions

security

interview

The Permission Triplet: Owner-Group-Other

Every file and directory has three permission sets: **owner (u)**, **group (g)**, and **others (o)**. Each set has three bits: **read (r=4)**, **write (w=2)**, **execute (x=1)**. This 9-bit mode (plus 3 special bits) is the standard Unix permission model.

```
# Permission modes: rwxr-xr--
#
#          uuu ggg ooo
#          user group others

ls -l /etc/passwd
# -rw-r--r-- 1 root root 2.8K Mar 15 12:34 /etc/passwd
# ^ | | | ^ ^ ^
# | u g o | | size
# type      owner group

# Numeric (octal) representation:
# rwx = 4+2+1 = 7
```

```
# rw- = 4+2+0 = 6
# r-x = 4+0+1 = 5
# r-- = 4+0+0 = 4
# --- = 0+0+0 = 0

chmod 755 script.sh    # rwxr-xr-x - owner: all, group/other:
                        read+execute
chmod 644 config.conf  # rw-r--r-- - owner: read+write,
                        group/other: read
chmod 600 id_rsa       # rw----- - SSH private key: owner only!
chmod 700 ~/.ssh       # rwx----- - directory: owner only (lock
                        down SSH)

chmod u+x script.sh    # Add execute for user
chmod g-w file.txt     # Remove write for group
chmod o=r file.txt     # Set others to read-only
chmod a+rx /app/bin    # Add read+execute for all
                        (user,group,other)
```

Directories: Execute Bit = Traversal

On directories, the permission bits have different meanings: **r** = list files, **w** = create/delete files, **x** = traverse/search (enter) the directory. Without the execute bit, you can't access any files in the directory even if you know their names — a common source of confusion.

```
# Directory permissions explained:
# r (read):    can list directory contents (ls)
# w (write):   can create, rename, delete files in the directory
# x (execute): can traverse the directory (cd into it, access files
                by name)

# Common directory permissions:
chmod 755 /home/user    # rwxr-xr-x - standard: owner writes,
                        others traverse
chmod 700 /home/user/.ssh # rwx----- - SSH directory: only owner!
```

```

chmod 1777 /tmp          # rwxrwxrwt – world-writable with sticky
bit

# Gotcha: read WITHOUT execute on a directory
chmod 644 /restricted-dir # rw-r--r--
ls /restricted-dir        # Can LIST files (has r)
cat /restricted-dir/file  # Permission denied! Can't traverse (no
x)

# Minimum permissions for web server to serve files:
# Directory: r-x (5) – traverse and list
# File:      r-- (4) – read only
# No need for w (2) on files or directories

```

Special Permissions: SUID, SGID, Sticky Bit

Three special permission bits add additional behavior. **SUID (4000)** — run a file with the owner's privileges (e.g., `passwd` runs as root). **SGID (2000)** — on files, run with group's privileges; on directories, new files inherit the directory's group. **Sticky Bit (1000)** — only file owners can delete their files (used on `/tmp`). These are frequent interview topics.

```

# SUID (Set User ID) – runs as file owner, not the user executing
it
chmod u+s /usr/bin/passwd # Set SUID (s in user execute position)
ls -l /usr/bin/passwd
# -rwsr-xr-x ... passwd – the 's' means SUID + executable

# Check all SUID binaries on your system:
find / -perm -4000 -type f 2>/dev/null

# SGID (Set Group ID) – on directories: new files inherit group
chmod g+s /shared/project # Set SGID (s in group execute
position)
# Now any file created in /shared/project belongs to the
directory's group

```

```
# Sticky Bit – only owners can delete their own files
chmod +t /tmp          # Set sticky bit (t in others execute
                        # position)
ls -ld /tmp
# drwxrwxrwt ... /tmp – the 't' means sticky bit is set

# === FAANG Interview: SUID Security ===
# Why is SUID dangerous? If a SUID root binary has a vulnerability,
# any user can escalate to root.
# Mitigation: minimize SUID binaries (use sudo instead),
# mount filesystems with 'nosuid' option where possible.

# Numeric shortcuts:
chmod 4755 script      # SUID + rwxr-xr-x
chmod 2755 dir         # SGID + rwxr-xr-x
chmod 1777 /tmp        # Sticky bit + rwxrwxrwx
```

Access Control Lists (ACLs)

POSIX ACLs extend the traditional permission model to support multiple users and groups. When the basic u/g/o model is insufficient, ACLs provide fine-grained control. **FAANG production use case:** shared team directories where multiple service accounts and humans need different access levels.

```
# Check if ACLs are enabled on a filesystem
mount | grep acl
# (Most modern filesystems have ACLs enabled by default)

# View ACLs on a file/directory
getfacl /shared/project
# file: shared/project
# owner: alice
# group: dev-team
# user::rwx
# user:bob:r-x      # Bob has read+execute only
```

```
# group::r-x
# mask::r-x
# other::---

# Set ACLs
setfacl -m u:bob:rx /shared/project    # Add Bob with read+execute
setfacl -m g:devops:rwx /shared/tools   # Add devops group with full
access
setfacl -m o::--- /shared/project      # Remove all access for
others

# Recursive ACL (be careful – performance impact)
setfacl -R -m u:ci-user:rx /app/deploy

# Remove specific ACL entry
setfacl -x u:bob /shared/project

# Default ACL – new files/dirs inherit these ACLs
setfacl -d -m g:dev-team:rwx /shared/project

# === FAANG Insight ===
# ACLs add complexity. Before extending ACLs, ask:
# Can I solve this with Unix groups in /etc/group?
# ACLs are harder to audit and debug than traditional permissions.
# In FAANG environments, we minimize ACL usage and rely on:
# - Clear group membership
# - Service accounts with minimal permissions
# - Config management (Ansible/Terraform) to enforce permissions
```

umask & Default Permissions

The `umask` determines default permissions for newly created files and directories. It's a mask (inverted) — the permissions are `base - umask`. Files default to `666 (rw-rw-rw-)`, directories to `777 (rwxrwxrwx)`. The umask subtracts from these.

```
# Current umask value
umask                # 0022 (most common)
umask -S             # Symbolic: u=rwx,g=rx,o=rx

# How umask works:
# File defaults:    666 (rw-rw-rw-)
# Minus umask:      022 (---w--w-)
# Result:           644 (rw-r--r--)

# Directory defaults: 777 (rwxrwxrwx)
# Minus umask:        022 (---w--w-)
# Result:             755 (rwxr-xr-x)

# Common umask values:
# 0022 - rw-r--r-- files, rwxr-xr-x dirs (standard)
# 0027 - rw-r-----, rwxr-x--- (shared team with restricted group)
# 0077 - rw-----, rwx----- (private - SSH keys!)

umask 0077           # Set for this shell session
# Add to ~/.bashrc for persistence

# === FAANG Interview ===
# Q: 'A user reports that files they create in /shared/project
#    are not accessible by their teammates. What's wrong?'
# A: Check umask. If it's 0077, files are created with no group
#    access.
#    Fix: set umask 0027 in the team's .bashrc or the shared
#    script.
```

FAANG Interview Q&A

Q: 'What does chmod 4755 do?'

A: '4 indicates the SUID bit is set. 755 means rwxr-xr-x. So the file runs with the owner's privileges regardless of who executes it. Common for utilities like passwd and ping that need elevated

access.'

Q: 'How do you audit permissions across a fleet of 1000 Linux servers?'

A: 'At FAANG scale, we never SSH into individual servers to check permissions. Instead, configuration management (Ansible, Chef, Puppet) enforces desired permissions via declarative config. We also use auditing tools like `auditd` with watches on sensitive paths, and drift detection that compares current permissions against a golden baseline. For security-critical checks (SUID, world-writable files), we run automated scans and alert on changes.'

Q: 'Scenario: You need to give a CI/CD service account write access to `/app/deploy` but read-only access to `/app/config`. The owner is `deploy-user`, group is `dev-team`. How?'

A: 'Traditional permissions: if `ci-user` needs different access than `deploy-user`, the basic `u/g/o` model won't work. I'd use ACLs: `setfacl -m u:ci-user:rwX /app/deploy` and `setfacl -m u:ci-user:rx /app/config`. Or create a dedicated group (`ci-agents`), add `ci-user` to it, and set group permissions accordingly. The group approach is simpler to audit.'

References & Further Reading

- **man chmod** — Change file modes bits
- **man getfacl / setfacl** — POSIX ACL management

- **visudo** — Edit sudoers safely

FAANG Architect's Note: Permission management at Internet scale is fundamentally different from single-server Linux. We use **Infrastructure as Code** to declare file and directory permissions (e.g., Terraform's `file_permission` attribute, Ansible's `file` module). We never rely on manual `chmod` or ACL configuration in production. The principle of least privilege applies: every service account should have exactly the permissions it needs — no more.

Interview Tip: When asked about permissions, immediately think about the security implications. A candidate who says 'chmod 777 to fix it' fails immediately. The right answer always involves 'what's the minimum permission needed?'

CHAPTER 4

CLI Fundamentals

Foundations · 5 topics

The Linux command line is the primary interface for every engineer at FAANG. GUI tools are rarely used in production. This guide covers the essential commands, the pipe model, I/O redirection, environment variables, and the shell startup sequence — all from a FAANG Architect's perspective.

FAANG

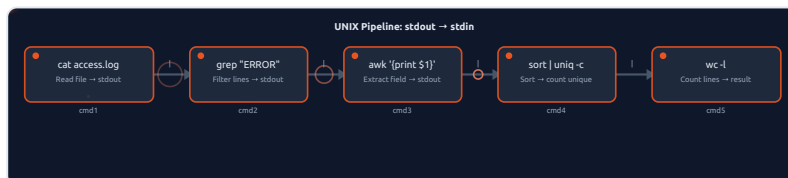
Linux

CLI

basics

interview

The Unix Philosophy: Small Composable Tools



Every Linux command does one thing well. Commands are connected via **pipes (|)** to build powerful pipelines. This composability is the key insight that makes the CLI so effective. In FAANG interviews, candidates who think in pipelines demonstrate system-level thinking.

```
# The pipe model: command1 | command2 | command3
# stdout of first -> stdin of second -> stdin of third

# Classic examples:
```

```
# Find top 5 largest files in /var/log:
du -sh /var/log/* 2>/dev/null | sort -rh | head -5

# Count unique IPs in an access log:
awk '{print $1}' /var/log/nginx/access.log | sort -u | wc -l

# Find processes using most memory:
ps aux --sort=-%mem | head -10

# The building blocks:
# stdin (0) - input to command
# stdout (1) - normal output
# stderr (2) - error output

ls /nonexistent > output.txt 2>&1    # Redirect both stdout and
stderr                               # stderr
ls /nonexistent &>> output.txt        # Bash 4+: append both

# /dev/null - the bit bucket
grep -r 'secret' . 2>/dev/null       # Suppress errors
```

Essential Commands Every FAANG Engineer Knows Cold

These commands should be second nature. Your interviewer won't wait while you figure out basic syntax. **Knowing these without thinking signals senior-level muscle memory.**

```
# === NAVIGATION ===
pwd          # Print working directory
cd -         # Go to previous directory
ls -la       # List all files with details
ls -lhSr     # List sorted by size (smallest first)
tree -L 2    # Show directory tree 2 levels deep

# === FILE OPERATIONS ===
cp -r src/ dst/          # Recursive copy
```

```

mv file.txt /new/location/      # Move (also rename)
rm -rf dir/                     # Remove directory and contents
                                # (DANGER!)
mkdir -p a/b/c/d                # Create nested directories
touch file.txt                  # Create empty file or update
                                # timestamp
ln -s /actual/path link_name    # Create symbolic link

# === VIEWING FILES ===
cat file.txt                    # Full file to stdout (small files only!)
less file.txt                   # Scrollable viewer (q to quit, / to search)
head -20                        # First 20 lines
tail -f log                     # Follow log in real time (journalctl -f for
                                # modern)
tail -n100                      # Last 100 lines

# === SEARCHING ===
grep -r 'error' /var/log/       # Recursive search
find / -name '*.log' -type f    # Find files by name
locate nginx.conf               # Search indexed database (updatedb)

# === PROCESS ===
ps aux                          # All processes
ps auxf                         # Forest view (show parent-child)
top                              # Interactive process viewer
htop                            # Better top (install if needed)
kill -9 <PID>                    # Force kill (SIGKILL) – use as last resort!
kill -15 <PID>                  # Graceful termination (SIGTERM) – preferred

# === SYSTEM ===
df -h                           # Disk free (human-readable)
du -sh /dir/                    # Disk usage (summary)
free -h                         # Memory usage
uptime                          # How long since boot + load averages
uname -a                        # Kernel version and system info
whoami                          # Current user

```

I/O Redirection & Here Documents

Redirection is how you control where command output goes. Beyond basic `>` and `>>`, understanding file descriptors and here-docs is essential for scripting and debugging.

```
# File descriptors
# 0 = stdin, 1 = stdout, 2 = stderr

command > file      # Redirect stdout to file (overwrite)
command >> file     # Redirect stdout to file (append)
command 2> file     # Redirect stderr to file
command &> file     # Redirect both stdout and stderr (Bash 4+)
command > file 2>&1 # Classic: stdout to file, then stderr to
same place

# Suppress all output
silent_cmd > /dev/null 2>&1

# Here Document – feed multi-line input into a command
cat << 'EOF' > config.txt
server {
    listen 80;
    server_name example.com;
}
EOF

# Here String – feed a string into stdin
grep 'error' <<< "this line has an error here"

# Process substitution – treat command output as a file
diff <(ls dir1/) <(ls dir2/)

# tee – output to file AND stdout
echo "deploying..." | tee deploy.log
```

Environment Variables & Shell Configuration

Environment variables control almost everything on Linux: the PATH for finding commands, the editor for git, locale settings, and proxy configuration. Understanding the shell startup sequence (~/.bashrc, ~/.bash_profile, /etc/profile) is a common interview topic.

```
# Essential environment variables
echo $PATH          # Colon-separated list of command directories
echo $HOME          # Current user's home directory
echo $USER          # Current username
echo $SHELL         # Current shell (/bin/bash, /bin/zsh)
echo $LANG          # Locale settings
echo $EDITOR        # Default editor (vim, nano, code)

# Shell startup order (Bash):
# 1. /etc/profile    - System-wide (login shell)
# 2. ~/.bash_profile - User-specific (login shell, overrides #1)
# 3. ~/.bashrc       - User-specific (interactive non-login)
# 4. /etc/bashrc     - System-wide (interactive)

# Zsh startup order:
# .zshenv → .zprofile → .zshrc → .zlogin

# Common config to add to ~/.bashrc:
export PATH="$HOME/bin:$PATH" # Add personal bin to PATH
export EDITOR=vim             # Set default editor
unalias grep 2>/dev/null; alias grep='grep --color=auto'
export HISTSIZE=10000         # Bigger history
export HISTCONTROL=ignoredups # No duplicate history lines

# Source a file after editing:
source ~/.bashrc # or just: . ~/.bashrc

# Set environment variable for a single command:
EDITOR=nano git commit # Uses nano just for this commit
```

Job Control & Process Management

The shell provides built-in job control: running commands in the background, suspending them, and bringing them to the foreground. Essential for managing multiple tasks in a terminal session.

```
# Background a command
long_running_task &
sleep 10 &

# List background jobs
jobs
# [1]+  Running                  sleep 10 &

# Bring job to foreground
fg %1          # Bring job 1 to foreground

# Suspend current foreground job
# Press: Ctrl+Z  # Suspends the running command
# Now it appears in jobs list as 'Stopped'

# Run suspended job in background
bg %1

# No-hup – keep running after logout
nohup long_task &

# disown – remove job from shell's job table
long_task &
disown          # Now closing terminal won't kill it

# screen/tmux – terminal multiplexers (essential for remote work)
screen -S mysession # Create named session
screen -ls          # List sessions
screen -r mysession # Reattach to session

# Ctrl+A, D – detach from screen (stays running)
```

FAANG Interview Q&A

Q: 'You SSH into a production box. There's no htop, no standard monitoring tools. List the commands you'd run in the first 30 seconds to understand what's wrong.'

A: ' `uptime` for load averages, `free -h` for memory, `df -h` for disk, `top -bn1 | head -5` for top CPU consumers, `dmesg | tail -20` for kernel messages (OOM, disk errors), `journalctl -xe -n 50 --no-pager` for recent system logs.'

Q: 'What's the difference between `.bashrc` and `.bash_profile`?'

A: 'When you SSH into a machine, you get a login shell — it reads `.bash_profile`. When you open a new terminal window in a GUI, it's typically an interactive non-login shell — it reads `.bashrc`. Best practice: put everything in `.bashrc`, and source `.bashrc` from `.bash_profile`.'

Q: 'You accidentally ran `rm -rf` on important files. What do you do?'

A: 'Immediately stop all writes to the filesystem. Unmount the partition if possible. Use `extundelete` or `testdisk` to recover. If this is production, fail over to a replica. In FAANG, we have automated backups — no critical data lives on a single server without replication. Also, train your team to alias `rm` to '`rm -i`' or use `trash-cli`.'

References & Further Reading

- **Bash Reference Manual** — Official GNU Bash documentation
- **The Linux Command Line** — William Shotts, free online book

FAANG Architect's Note: The most surprising thing for new engineers at FAANG is that the CLI isn't just for 'Linux people' — everyone uses it. Product managers query dashboards with jq. Data scientists pipe CSV data through awk. The CLI is the universal interface. Being fluent in pipes and redirection is like being fluent in English at an American company — it's the baseline.

Production Tip: Never run an untested command on a production server. Always use '--dry-run' flags, pipe to 'less', or wrap destructive commands in a script with safety checks. At FAANG, we use 'canary' instances and staged rollouts — but even on a single server, think before you execute.

CHAPTER 5

Shell Scripting Basics

Foundations · 5 topics

Shell scripting is how Linux systems are automated. From CI/CD pipelines to deployment scripts to cron jobs — if you work at FAANG, you will read and write shell scripts. This guide covers the essentials with a focus on patterns that matter in production environments.

FAANG

Linux

shell-scripting

bash

interview

Script Anatomy & Shebang

Every shell script starts with a shebang (`#!/bin/bash` or `#!/bin/sh`) and should set strict error handling. The shebang tells the kernel which interpreter to use. In FAANG production scripts, you'll almost always see `#!/bin/bash` with `set -euo pipefail` .

```
#!/bin/bash
# The standard FAANG production header:
set -euo pipefail
# -e: Exit on error (any command fails)
# -u: Treat unset variables as error
# -o pipefail: Pipeline fails if any command fails (not just the
last)
# -x: Print each command before executing (for debugging)

# Making your script executable:
chmod +x script.sh
./script.sh      # Run it
bash script.sh   # Also works
```

```
# FAANG note: Never use #!/bin/sh in production
# On Ubuntu/Debian, /bin/sh is dash (minimal POSIX)
# On RHEL/CentOS, /bin/sh is bash
# Use bash explicitly to avoid compatibility surprises
```

Variables, Parameters & Quoting

Variables, command-line arguments, and proper quoting are the foundation. **Always quote your variables** — unquoted variables undergo word splitting and glob expansion, causing bugs that are hard to debug. This is a FAANG interview red flag when missed.

```
#!/bin/bash
set -euo pipefail

# Variables – no spaces around =
NAME="World"
echo "Hello, $NAME"           # Variable expansion
echo "Hello, ${NAME}!"        # Braces for clarity/concatenation

# Command-line arguments
echo "Script: $0"             # Script name
echo "First arg: $1"          # $1-$9 are positional args
echo "All args: $@"           # All args as separate words
echo "Arg count: $#"          # Number of args
shift                         # Shift $2→$1, $3→$2, etc.

# Special variables
echo "Exit code: $?"          # Exit code of last command
echo "PID: $$"                # Current script PID
echo "Last BG PID: $!"        # PID of last background process

# QUOTING – this will bite you in production:
FILE="My Document.txt"       # Filename with space
cat $FILE                     # ERROR: tries to cat 'My' and
'Document.txt'
```

```

cat "$FILE"                # CORRECT: passes the whole filename

# Using defaults
echo "${NAME:-default}"    # Use default if unset
echo "${NAME:=default}"    # Set default AND use it

# Command substitution (two styles)
CURRENT_DIR=$(pwd)         # Modern style (preferred)
CURRENT_DIR=`pwd`          # Old style (avoid)

# Arithmetic
COUNT=$((COUNT + 1))    # $(( ... )) for integer math

```

Conditionals & Control Flow

The `if` / `test` / `[ ]` / `[[ ]]` system is unlike any other language. The `test` command (or `[ ]`) checks conditions. The double-bracket `[[ ]]` is a Bash extension with more features (pattern matching, regex).

```

#!/bin/bash
set -euo pipefail

# File tests (most common in scripts)
if [ -f "$FILE" ]; then echo "$FILE exists"; fi
if [ -d "$DIR" ]; then echo "$DIR is a directory"; fi
if [ -x "$BINARY" ]; then echo "$BINARY is executable"; fi
if [ -z "$STRING" ]; then echo "String is empty"; fi
if [ -n "$STRING" ]; then echo "String is not empty"; fi

# String comparison
if [ "$NAME" = "root" ]; then echo "Is root"; fi    # POSIX:
single =
if [ "$NAME" != "admin" ]; then echo "Not admin"; fi
if [[ "$NAME" == r* ]]; then echo "Starts with r"; fi # Pattern
match (Bash)
if [[ "$HOST" =~ ^web[0-9]+ ]]; then echo "Web server"; fi # Regex

```

```
match

# Numeric comparison
if [ "$COUNT" -gt 10 ]; then echo ">10"; fi      # -gt, -lt, -
ge, -le, -eq
if (( COUNT > 10 )); then echo ">10"; fi          # Modern math
syntax

# File descriptors in conditionals
if [ -t 0 ]; then echo "stdin is a terminal"; fi  # Interactive
session?

# elif and else
if [ "$1" = "start" ]; then
    /usr/bin/myservice start
elif [ "$1" = "stop" ]; then
    /usr/bin/myservice stop
else
    echo "Usage: $0 {start|stop}"
    exit 1
fi

# case statement – cleaner than long if-elif chains
case "$1" in
    start|restart)  action="start" ;;
    stop)           action="stop" ;;
    *)              echo "Unknown: $1"; exit 1 ;;
esac
```

Loops & Arrays

Loops iterate over files, commands, or sequences. Arrays are underused in shell scripts — they make scripts cleaner and more maintainable.

```
#!/bin/bash
set -euo pipefail
```

```
# for loop over explicit list
for fruit in apple banana cherry; do
    echo "$fruit"
done

# for loop over command output
for file in $(ls /etc/*.conf); do
    echo "Found: $file"
done

# for loop with seq (C-style)
for ((i=0; i<10; i++)); do
    echo "Number $i"
done

# while loop – read file line by line
while IFS= read -r line; do
    echo "Line: $line"
done < input.txt
# IFS= preserves leading whitespace
# -r prevents backslash interpretation

# Arrays
FRUITS=("apple" "banana" "cherry")
echo "${FRUITS[0]}"           # First element
echo "${FRUITS[@]}"           # All elements
echo "${#FRUITS[@]}"          # Array length
FRUITS+=("date")              # Append

# Iterate array
for fruit in "${FRUITS[@]"; do
    echo "$fruit"
done

# Associative arrays (Bash 4+)
declare -A COLORS
COLORS[apple]="red"
COLORS[banana]="yellow"
echo "${COLORS[apple]}"       # red
```

Functions & Error Handling

Functions organize reusable logic. Error handling patterns with `trap` ensure cleanup on failure — this is non-negotiable in FAANG production scripts.

```
#!/bin/bash
set -euo pipefail

# Define a function
log() {
    local level="$1"          # local makes variable function-scoped
    local message="$2"
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] [$level] $message"
}

log "INFO" "Starting deployment"
log "ERROR" "Connection failed"

# Return values
is_root() {
    if [ "$EUID" -eq 0 ]; then
        return 0             # 0 = success (true)
    else
        return 1             # non-zero = failure (false)
    fi
}

if is_root; then
    echo "Running as root"
fi

# === The FAANG production pattern: trap and cleanup ===
cleanup() {
    log "INFO" "Cleaning up..."
    rm -f /tmp/deploy.$$
    log "INFO" "Cleanup complete"
}

trap cleanup EXIT           # cleanup runs on any exit (success,
```

```
error, signal)
trap 'exit 1' INT TERM    # Handle Ctrl+C gracefully

# FAANG standard deployment wrapper pattern:
deploy() {
    local version="$1"
    log "INFO" "Deploying version $version"

    if ! ./preflight.sh; then
        log "ERROR" "Preflight failed"
        return 1
    fi

    ./deploy.sh "$version"
    log "INFO" "Deploy complete"
}

# Run it
deploy "v2.3.1" || exit 1
```

FAANG Interview Q&A

Q: 'Write a script that monitors a log file for ERROR patterns and sends an alert if more than 10 errors appear in 60 seconds.'

A: `'tail -f /var/log/app.log | grep --line-buffered ERROR | head -10'`

— but for a real solution, use a while loop with timestamp checking and a counter. In production, you'd use a proper monitoring system (Prometheus, Datadog) — this script is for bootstrap scenarios.'

Q: 'What does `set -euo pipefail` do, and why do you always use it in production scripts?'

A: '-e: exit on first error. -u: error on unset variables (catches typos). -o pipefail: propagate failures through pipes. Without it, 'false | true' would succeed. These three flags turn shell scripts from fragile collections of commands into reliable programs.'

Q: 'Write a script that backs up /var/lib/mysql to /backup, rotating the last 7 days.'

A: See the code block for a complete backup rotation script.

```
#!/bin/bash
set -euo pipefail

BACKUP_DIR="/backup/mysql"
SOURCE="/var/lib/mysql"
RETENTION=7

mkdir -p "$BACKUP_DIR"

# Create timestamped backup
TIMESTAMP=$(date '+%Y%m%d_%H%M%S')
BACKUP_FILE="$BACKUP_DIR/mysql_backup_${TIMESTAMP}.tar.gz"

tar -czf "$BACKUP_FILE" "$SOURCE"
echo "Created: $BACKUP_FILE"

# Remove backups older than RETENTION days
find "$BACKUP_DIR" -name 'mysql_backup_*.tar.gz' -mtime +$RETENTION -delete
echo "Cleaned up backups older than $RETENTION days"

# Verify backup integrity
tar -tzf "$BACKUP_FILE" > /dev/null && echo "Backup verified successfully"
```

```
# Report to monitoring
curl -s "https://monitor.internal/report?
metric=mysql.backup&value=1" > /dev/null || true
```

References & Further Reading

- **Bash Reference Manual** — Official documentation
- **Google Shell Style Guide** — FAANG standard for shell script style
- **ShellCheck** — Shell script static analysis (use this!)

FAANG Architect's Note: Shell scripts are the most common type of code I see deployed in production that has **no tests**. At FAANG, we enforce: (1) ShellCheck runs as part of CI, (2) scripts must have error handling (set -euo pipefail), (3) no script longer than 200 lines without justification. Longer scripts should be rewritten in Python or Go with proper testing.

Interview Tip: When asked to write a shell script in an interview, always start with 'set -euo pipefail' and '#!/bin/bash'. These lines alone signal production experience. Also, remember that shell scripts are glue code — they call other programs, handle arguments, and manage flow. Keep them simple.

CHAPTER 6

Text Processing: grep, sed, awk

Foundations · 7 topics

grep, sed, and awk are the three pillars of Linux text processing. At FAANG, every engineer — from SRE to ML researcher — uses these tools daily. Mastering them signals that you think in Unix pipelines: finding patterns, transforming output, and extracting insights without needing to write a script.

FAANG Linux text-processing grep sed awk interview

grep — Global Regular Expression Print

grep searches files for lines matching a pattern. It's the first tool you reach for when investigating anything on a Linux system. FAANG engineers use grep constantly — finding error messages, filtering log output, checking configurations across directories.

```
# Basic usage
grep 'error' /var/log/syslog          # Find all lines
containing 'error'
grep -i 'warning' app.log            # Case-insensitive
grep -v 'debug' app.log              # Invert match (exclude
debug)
grep -c 'timeout' access.log         # Count matches
grep -n 'exception' stacktrace.log   # Show line numbers

# Useful flags
grep -r 'TODO' /src/                 # Recursive search in
directory
```

```

grep -l 'config' /etc/**/*.conf      # List only filenames
with matches
grep -w 'class' schema.sql            # Whole word match (not
'subclass')
grep -A3 -B2 'FATAL' app.log          # Show 2 lines before, 3
after context
grep -E 'error|fail|timeout' log      # Extended regex
(multiple patterns)

# Piped grep – filter command output
dmesg | grep -i 'error'               # Kernel errors
ps aux | grep 'python'               # Find python processes
(but grep itself
matches!)
ps aux | grep '[p]ython'              # Trick: brackets prevent
grep from matching
itself

# FAANG scenario: count unique error types in a 10GB log
grep -oP 'ERROR \K\w+' app.log | sort | uniq -c | sort -rn
# -o: only matching text, -P: perl regex, \K: discard before
# (This extracts the word after 'ERROR ')

```

sed — Stream Editor

sed transforms text streams. It's grep's more powerful cousin — find patterns AND modify them. sed is essential for batch edits, text replacement, and pipeline formatting.

```

# Basic substitution (the most common operation)
sed 's/old/new/' file.txt             # Replace first
occurrence per line
sed 's/old/new/g' file.txt            # Replace ALL occurrences
per line (global)
sed 's/old/new/gi' file.txt           # Global + case-
insensitive
sed 's/old/new/2' file.txt            # Replace second
occurrence only

```

```

# In-place editing (careful!)
sed -i 's/foo/bar/g' config.conf      # Edit file in place
(Linux version)

sed -i '' 's/foo/bar/g' config.conf   # macOS/BSD: empty
extension = no backup

sed -i.bak 's/foo/bar/g' config.conf  # Create backup with .bak
extension

# Line addressing
sed '10s/old/new/' file.txt           # Replace on line 10 only
sed '5,10s/old/new/g' file.txt        # Replace on lines 5-10
sed '5,$s/old/new/g' file.txt         # Replace from line 5 to
end

# Delete lines
sed '/debug/d' app.log                # Delete lines matching
'debug'

sed '1,5d' file.txt                   # Delete first 5 lines
sed '/^$/d' file.txt                  # Delete empty lines

# Print specific lines
sed -n '10,20p' file.txt              # Print lines 10-20 (like
head/tail combo)

sed -n '/ERROR/p' /var/log/syslog      # Print only lines
matching ERROR

# Multiple commands
sed -e 's/foo/bar/' -e 's/abc/xyz/' file.txt

# FAANG scenario: mask IP addresses in a log file
sed -E 's/[0-9]{1,3}(\.[0-9]{1,3}){3}/***.***.***.***/' access.log

```

awk — Pattern Scanning and Processing Language

awk is a full programming language designed for text processing. It splits lines into fields, processes them with C-like syntax, and is unmatched for extracting columns, computing statistics, and reformatting output. If `grep` is a scalpel and `sed` is a Swiss Army knife, `awk` is a workshop.

```
# awk's basic structure:
# awk 'pattern { action }' file

# Field splitting (default: whitespace)
awk '{print $1}' access.log           # Print first field (IP
address)
awk '{print $1, $NF}' access.log      # First and last field
awk -F: '{print $1}' /etc/passwd     # Custom delimiter (:)

# Patterns
awk '/error/ {print $0}' app.log      # Print lines matching
'error'
awk 'NR > 5 {print}' file.txt         # Skip first 5 lines
awk 'length($0) > 80' file.txt       # Lines longer than 80
characters

# BEGIN and END blocks
awk 'BEGIN {print "Report"} {print $0} END {print "---End---"}'
file

# Computations
awk '{sum += $1} END {print sum}' numbers.txt # Sum of first
column
awk '{count++} END {print count}' file.txt    # Line count
awk '{sum += $1} END {print sum/NR}' numbers.txt # Average

# Formatted output
awk '{printf "%-20s %s\n", $1, $NF}' /etc/passwd

# Multi-line awk scripts
awk '{
    if ($1 ~ /ERROR/) {
        errors++
    }
}
```

```

        print $0 > "errors.log"
    }
} END {
    print "Total errors:", errors
}' app.log

# FAANG scenario: parse nginx access log (combined format)
# Fields: $1=IP, $6=method, $7=path, $9=status, $10=bytes
awk '{
    ip[$1]++                # Count requests per IP
    status[$9]++            # Count HTTP status codes
    total_bytes += $10      # Total bytes served
} END {
    print "=== Top IPs ===";
    for (i in ip) print i, ip[i];
    print "=== Status Codes ===";
    for (s in status) print s, status[s];
    print "Total bytes:", total_bytes;
}' /var/log/nginx/access.log

```

xargs — Build and Execute Commands

xargs takes input from stdin and passes it as arguments to another command. It's the glue that connects find/grep output to commands like rm, chmod, or cp. Understanding xargs is what separates intermediate from advanced CLI users.

```

# Basic usage: pass filenames to a command
find /tmp -name '*.tmp' -type f | xargs rm -f    # Remove temp
files

find . -name '*.py' | xargs wc -l               # Count lines in
all Python files

# Handle filenames with spaces (use -0 with null-separated)
find . -name '*.log' -print0 | xargs -0 rm

# Parallel execution

```

```

find . -name '*.gz' | xargs -P 4 gunzip      # Unzip 4 files at
once

# Replace string placeholder
ls *.txt | xargs -I {} cp {} /backup/      # Copy each file
to /backup

# Prompt before executing (safe mode)
find . -name '*.swp' | xargs -p rm        # 'y' to confirm
each

# FAANG scenario: batch chmod with confirmation
find /app/deploy -type f | xargs -p -I {} chmod 644 {}

# Count lines across source files
grep -rl 'TODO' /src | xargs wc -l | tail -1  # Total TODO lines

```

Pipeline-Fu: Combining Everything

The real power comes from combining these tools. These compound pipelines are the kind of thing FAANG engineers type daily — and they make excellent interview problems.

```

# Find top 5 slowest requests from an nginx log
# ($NF is request time in seconds)
awk '{print $NF, $0}' access.log | sort -rn | head -5 | awk '{print
$2}'

# Find all unique IPs accessing /api, count requests
awk '$7 ~ /\^\/api/ {print $1}' access.log | sort | uniq -c | sort -
rn

# Report on HTTP status codes with percentage
awk '{
    total++
    status[$9]++
} END {
    print "Total requests:", total;

```

```

        for (s in status) printf "%s: %d (%.1f%%)\n", s, status[s],
status[s]/total*100
    }' access.log

# Find processes with high memory, format as CSV
ps aux --sort=-%mem | awk 'NR>1 {printf "%s,%s,%s\n", $2, $11, $4}'
| head -10

# Monitor log file in real-time for error patterns
tail -f /var/log/app.log | grep --line-buffered 'ERROR' | awk
'{print $1, $3, $NF}'

# Batch rename files with sed
for f in *.txt; do mv "$f" "$(echo "$f" | sed 's/ /_/g')"; done

# FAANG scenario: find files modified in last 24h, compress them
find /var/log/myapp -mtime -1 -name '*.log' | xargs tar -czf
backup.tar.gz

```

FAANG Interview Q&A

Q: 'Using a one-liner, find the top 10 IP addresses hitting your web server from an nginx access log.'

A: ' `awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -10` '

Q: 'You need to replace all occurrences of 'http://' with 'https://' in a directory of config files, but only in lines that contain 'api.' Do it without an editor.'

A: ' `sed -i '/api/s|http://|https://|g' /etc/nginx/sites-enabled/*` — the /api/ pattern limits sed to lines containing 'api', and I use | as delimiter to avoid escaping slashes.'

Q: 'Parse /etc/passwd and list all users with login shells that aren't /sbin/nologin or /bin/false.'

A: `awk -F: '$7 !~ /nologin|false/ {print $1, $7}' /etc/passwd`

References & Further Reading

- **GNU Grep Manual**
- **GNU Sed Manual**
- **GAWK (Awk) Manual** — The definitive reference
- **sed & awk** — Dale Dougherty & Arnold Robbins (O'Reilly)

FAANG Architect's Note: In my first week at Amazon, a principal engineer solved a production incident by typing a single awk pipeline that analyzed 50GB of logs in 10 seconds. I was floored. That's the power of Unix text processing. At FAANG scale, you can't always load data into Python or SQL — sometimes you need to find the needle in a haystack with nothing but the tools built into every Linux server since 1970.

Interview Tip: When asked a log analysis problem in an interview, always consider whether a one-liner can solve it before reaching for a full program. The ability to write pipelines on the whiteboard impresses interviewers — it shows systems thinking.

CHAPTER 7

Git Version Control

Foundations · 5 topics

Git is the version control system that underpins all modern software development. Created by Linus Torvalds for Linux kernel development, Git is now used by every FAANG company. Understanding Git deeply — not just add/commit/push — is essential for working in large engineering teams.

[FAANG](#)[Linux](#)[git](#)[version-control](#)[interview](#)

Git Objects: Blobs, Trees, Commits

Git is fundamentally a **content-addressable filesystem** with a VCS UI on top. Every file version is stored as a blob, directories as trees, and snapshots as commits — all referenced by their SHA-1 hash. Understanding this model is the key to mastering Git.

```
# Git's object model:
# blob → file content (hash of content)
# tree → directory listing (maps names → blobs/trees)
# commit → snapshot + metadata (tree hash, parent hash, author,
message)

# Explore the object database:
git init myrepo && cd myrepo
echo "hello" > file.txt
git add file.txt
git commit -m "Initial commit"
```

```
# What just happened:
git cat-file -p HEAD
# tree <tree-hash>
# author Kallol ...
# committer Kallol ...
# Initial commit

git cat-file -p HEAD^{tree}
# 100644 blob <blob-hash>    file.txt

git cat-file -p <blob-hash>
# hello

# Key insight: Git stores content, not files.
# If you rename a file without changing content, the blob is
# reused.
# If two files have the same content, they share a blob.
# This is why git is space-efficient for text files.
```

Branching & Merging Strategies

Branches are lightweight pointers to commits. Creating a branch is nearly instant — just a 41-byte file. FAANG teams use various branching strategies; understanding the tradeoffs between merge, rebase, and squash is an interview topic.

```
# Branches are just labels on commits
# main → C3 ← feature → C4
#           ↖           ↗
#           merge commit

git branch          # List local branches
git branch feature  # Create branch from HEAD
git checkout feature # Switch to branch
git checkout -b feature # Create and switch (one command)
git switch feature   # Modern alternative to checkout
```

```
# === Merge ===
git checkout main
git merge feature      # Creates a merge commit
# Pros: preserves history, safe
# Cons: cluttered history with merge commits

# === Rebase ===
git checkout feature
git rebase main        # Replay feature commits on top of main
git checkout main
git merge feature      # Fast-forward merge
# Pros: linear history, clean
# Cons: rewrites history (never rebase shared branches!)

# === Squash ===
git merge --squash feature
git commit -m "Add feature"
# Pros: single commit per feature
# Cons: loses intermediate commits (debugging harder)

# FAANG common workflows:
# Google: squash-merge (single commit per CL)
# Meta: rebase + merge (linear history)
# Amazon: merge commits (preserve full history)
```

Interactive Rebase & History Rewriting

Interactive rebase lets you rewrite history before sharing. **Never rewrite history on shared/public branches.** For personal feature branches, it keeps the commit log clean and reviewable.

```
# Interactive rebase – reorder, squash, edit, drop commits
git rebase -i HEAD~5    # Rebase last 5 commits

# In the editor, you'll see:
# pick abc1234 First commit
# pick def5678 Second commit
# pick hij9010 Third commit
```

```
#
# Commands:
# pick  = use commit as-is
# reword = change commit message
# edit   = stop to amend
# squash = combine with previous commit
# fixup  = squash but discard message
# drop   = delete commit

# Example: squash the last 3 commits into one:
git rebase -i HEAD~3
# Change 'pick' to 'squash' for commits 2 and 3

# Amend the last commit (fix message or add forgotten changes)
git add forgotten-file.txt
git commit --amend --no-edit # Add to last commit without editing
message

# Undo the last commit but keep changes staged
git reset --soft HEAD~1

# Undo the last commit and unstage changes
git reset HEAD~1          # --mixed (default)

# Completely discard last commit and changes
git reset --hard HEAD~1    # DANGER: destructive!
```

Git in Production: The FAANG Workflow

At FAANG, Git workflows enforce code review, CI/CD, and deployment gates. Understanding these patterns helps you navigate large codebases and collaborate effectively.

```
# === Typical FAANG Feature Workflow ===

# 1. Start from latest main
git checkout main
git pull --rebase          # Fetch + rebase local changes
```

```
# 2. Create feature branch
git checkout -b feat/my-feature

# 3. Make changes, commit often (incrementally)
git add -p                # Interactive staging (hunk by hunk)
git commit -m "feat: add user auth"

# 4. Keep branch updated with main
git fetch origin
git rebase origin/main    # Rebase instead of merge

# 5. Final cleanup before push
git rebase -i origin/main # Squash fixup commits
git log --oneline         # Verify clean history

# 6. Push and create pull request
git push -u origin feat/my-feature
# (Open PR in GitHub/GitLab/Bitbucket)

# 7. Address review feedback
# Make changes, then:
git add -p
git commit --amend --no-edit # Keep single commit (if squashing)
# or: just add more commits (if merging)
git push --force-with-lease  # Safe force-push after rebase

# 8. Merge (via PR button – squash or merge commit)

# === Cherry-pick for hotfixes ===
git checkout release/v2.0
git cherry-pick abc1234    # Pick a specific commit from main

# === Bisect: find the commit that introduced a bug ===
git bisect start
git bisect bad             # Current commit is bad
git bisect good v1.0       # v1.0 is known good
# Git checks out a midpoint commit – test it
git bisect good            # Or: git bisect bad
# Repeat until the first bad commit is found
git bisect reset
```

Git Configuration & Productivity Tips

A well-configured Git setup saves hours. These are the configurations every FAANG engineer sets up on day one.

```
# Global configuration (set once)
git config --global user.name "Kallol Chakraborty"
git config --global user.email "kallol@example.com"
git config --global core.editor vim
git config --global init.defaultBranch main

# Useful aliases
git config --global alias.lg "log --oneline --graph --all --decorate"
git config --global alias.last "log -1 HEAD"
git config --global alias.unstage "reset HEAD --"
git config --global alias.amend "commit --amend --no-edit"
git config --global alias.st "status -sb"

# Better diffing
git config --global diff.algorithm patience
git config --global diff.context 5

# .gitignore essentials for Linux projects:
# *.log
# *.swp
# .DS_Store
# build/
# dist/
# node_modules/
# .env

# === FAANG scenario: large monorepo ===
git clone --depth 1 <repo>          # Shallow clone (no history)
git clone --filter=blob:none <repo> # Partial clone (no blobs until needed)
git sparse-checkout set /path       # Checkout only specific directories
```

```
# === Stashing ===
git stash                # Save uncommitted changes
git stash -u             # Include untracked files
git stash pop            # Restore and remove stash
git stash list           # List all stashes
git stash drop stash@{0} # Delete a stash
```

FAANG Interview Q&A

Q: 'Explain the difference between git merge and git rebase. When would you use each?'

A: 'Merge creates a merge commit that ties two branches together — it preserves history exactly as it happened. Rebase rewrites your feature branch commits on top of the target branch — it creates a linear history. Use merge for shared branches (main, release) to preserve the true sequence of events. Use rebase for personal feature branches to keep history clean before review.'

Q: 'You accidentally committed to the wrong branch. How do you move the commit to the correct branch?'

A: ' `git log` to find the commit hash. `git checkout correct-branch` . `git cherry-pick` . `git checkout wrong-branch` . `git reset --hard HEAD~1` . Or use `git rebase --onto` to move a range of commits.'

Q: 'A junior engineer force-pushed to main and rewrote history. How do you recover?'

A: ' `git reflog` on the remote (if you have access) to find the original commit. `git reset --hard .` `git push --force-with-lease` to restore. Enable branch protection on main to prevent this — main should require PRs and disallow force-push.'

References & Further Reading

- **Pro Git** — The free online Git book (Scott Chacon & Ben Straub)
- **Atlassian Git Tutorials** — Visual explanations of Git concepts
- **Oh Shit, Git!** — Practical recoveries for common Git mistakes

FAANG Architect's Note: Git was created to solve a specific problem: 20,000+ developers contributing to the Linux kernel simultaneously. The FAANG monorepo models (Google's Piper, Meta's Mercurial-based repo) are more sophisticated, but Git skills translate directly. If you understand Git's object model deeply, you'll understand any VCS.

Interview Tip: When asked Git questions, show depth by referencing the underlying object model (blobs, trees, commits). Candidates who only know 'add, commit, push' don't demonstrate senior-level understanding. Know how to recover from mistakes — that's what separates experienced engineers.

PART 2

Systems & Operations

Systems operations for production environments: process management, package management, systemd, user administration, SSH, networking, storage, and security hardening.

CHAPTER 8

Process Management & Signals

Systems & Operations · 6 topics

Processes are the fundamental unit of execution on Linux. Every program, daemon, and shell command runs as one or more processes. At FAANG, understanding process lifecycle, signals, and the /proc filesystem is essential for debugging production issues, building resilient services, and optimizing performance.

FAANG

Linux

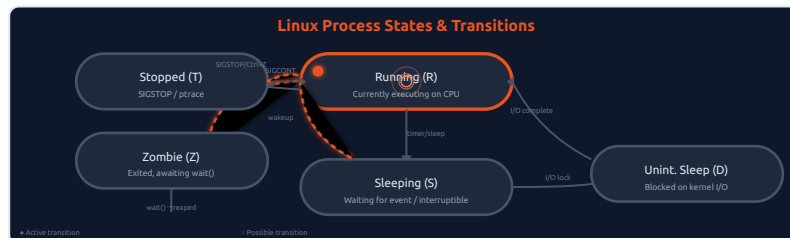
processes

signals

cgroups

production debugging

Process Lifecycle & States



A Linux process transitions through states: **Running (R)**, **Sleeping (S)**, **Uninterruptible Sleep (D)**, **Zombie (Z)**, and **Stopped (T)**. The kernel's scheduler allocates CPU time slices (time slices are

~100ms on desktop kernels, configurable via `sched_latency_ns`). At FAANG, 'D' state processes blocking I/O are a common cause of system stalls — you diagnose this via `ps` and `/proc//status`.

```
# === PS (Process Snapshot) ===
ps aux                # All processes, BSD-style
ps -ef               # All processes, standard format
ps auxf              # Forest view (tree with parent-child)
ps aux --sort=-%mem | head -10 # Top 10 by memory
ps aux --sort=-%cpu | head -10 # Top 10 by CPU
ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%cpu # Custom columns

# === TOP (Real-time) ===
top                  # Interactive
# Inside top:
# 1 - toggle per-CPU view
# M - sort by memory
# P - sort by CPU
# k - kill a process
# q - quit

htop                 # Better top (color, scroll, tree mode)

# === /proc - The Virtual Filesystem ===
cat /proc/cpuinfo    # CPU details (model, cores, flags)
cat /proc/meminfo     # Memory details (total, free, cached)
cat /proc/loadavg     # Load averages + running/total processes
cat /proc/1/cmdline   # Command line of PID 1 (init/systemd)
ls /proc/<PID>/fd/      # Open file descriptors (great for leaking FDs)
cat /proc/<PID>/status # Process state, memory, capabilities
cat /proc/<PID>/maps   # Memory-mapped regions
cat /proc/<PID>/environ # Environment variables (null-separated, use tr)
```

```
tr '\0' '\n' < /proc/<PID>/environ

# === Key fields in ps ===
# PID   - Process ID
# PPID  - Parent Process ID
# %CPU  - CPU usage (% of one core)
# %MEM  - Memory usage (% of total RAM)
# VSZ   - Virtual memory size
# RSS   - Resident Set Size (actual physical memory)
# STAT  - Process state (R, S, D, Z, T, etc.)
# WCHAN - Kernel function the process is sleeping in

# === Finding specific processes ===
pgrep -f "python server.py"      # Find PID by pattern
pgrep -u www-data                # All processes for a user
pidof nginx                      # Find PIDs of named process
pkill -f "stale_script"          # Kill processes matching pattern
```

Signals: The Linux Interrupt System

Signals are asynchronous notifications sent to processes. They can be generated by the kernel (SIGSEGV on segfault), by user actions (Ctrl+C = SIGINT), or by other processes (kill command).

Understanding signals is critical for writing robust daemons and debugging crashes in production. At FAANG, signal handlers must be carefully written to avoid race conditions and deadlocks.

```
# === Common Signals ===
# SIGTERM (15) - Graceful termination (default for kill)
# SIGKILL (9)  - Force kill (cannot be caught/ignored)
# SIGINT (2)   - Interrupt from keyboard (Ctrl+C)
# SIGQUIT (3)  - Quit with core dump (Ctrl+\)
# SIGHUP (1)   - Hangup (reload config for many daemons)
# SIGUSR1/2 (10,12) - User-defined signals
# SIGSTOP (19) - Stop process (cannot be caught/ignored)
# SIGCONT (18) - Continue a stopped process
# SIGSEGV (11) - Segmentation fault (invalid memory access)
```

```

# SIGPIPE (13) – Write to broken pipe (common in pipes)

# === Sending signals ===
kill -15 <PID>          # SIGTERM (graceful)
kill -9 <PID>           # SIGKILL (force, use as last
                        resort)
kill -1 <PID>           # SIGHUP (reload config)
kill -SIGUSR1 <PID>     # Send user-defined signal
killall nginx          # Send signal to all matching
                        processes

# === Trapping signals in scripts ===
#!/bin/bash
cleanup() {
    echo "Cleaning up..."
    rm -f /tmp/lockfile
    exit 0
}
trap cleanup SIGTERM SIGHUP SIGINT

# Run indefinitely until killed
echo "PID: $$"; while true; do sleep 1; done

# === Zombie Processes ===
# A zombie process (STAT=Z) has exited but its parent hasn't called
wait().
# Zombies are harmless but indicate a buggy parent. They consume a
PID + entry.
# You cannot kill a zombie. You must kill its parent (PPID) to let
init reap it.

# === Process Priority & Niceness ===
# Nice values: -20 (highest priority) to +19 (lowest priority)
nice -n 10 ./slow_task      # Run with lower priority
(nice=10)
renice -n -5 -p <PID>       # Change priority of running
process
chrt -r 1 ./realtime_task   # Set real-time scheduling
(SCHED_RR)

# === Limits ===

```

```

ulimit -a                # All limits for current shell
ulimit -n 65536          # Max open file descriptors
cat /proc/<PID>/limits    # Limits for a specific process

```

Daemons & Process Supervision

A daemon is a background process that runs independently of any terminal. Modern Linux uses **systemd** for service supervision, but legacy approaches include **init.d scripts** and **supervisord**. In FAANG environments, every service has a supervision system that auto-restarts on failure and collects metrics.

```

# === Systemd Service ===
systemctl status nginx      # Check status
systemctl start nginx       # Start service
systemctl stop nginx        # Stop service
systemctl restart nginx     # Restart
systemctl enable nginx      # Start on boot
systemctl disable nginx     # Remove from autostart
systemctl daemon-reload    # Reload unit files after editing
journalctl -u nginx         # View logs for service

# === Background & Daemonize ===
# Simple backgrounding
./server.sh &

# Using nohup to survive terminal close
nohup ./long_task > output.log 2>&1 &

disown                      # Remove from shell job table

# daemon(3) / double-fork – traditional daemon creation
# 1. Fork -> parent exits -> child orphaned to init
# 2. Child calls setsid() to create new session
# 3. Fork again -> second child continues
# 4. Close stdin/stdout/stderr, chdir to /
# This is handled automatically by systemd unit files.

```

```
# === FAANG pattern: Health checks ===
# Every daemon should answer a health check endpoint.
# Pattern: GET /healthz returns 200 OK
kill -0 <PID>                # Check if process exists (no
                             # signal sent)
if kill -0 "$PID" 2>/dev/null; then
    echo "Process $PID is running"
fi
```

cgroups & Containers (Modern Process Isolation)

Control groups (cgroups) limit and account for resource usage (CPU, memory, I/O) across process hierarchies. **Docker and Kubernetes are built on cgroups + namespaces.** At FAANG, every production process runs inside a cgroup — resource limits are non-negotiable.

```
# === cgroups v2 (modern) ===
# Hierarchy at /sys/fs/cgroup/
ls /sys/fs/cgroup/

# Create a cgroup and set limits
sudo mkdir /sys/fs/cgroup/myapp
sudo sh -c 'echo "100000 200000" > /sys/fs/cgroup/myapp/cpu.max'
sudo sh -c 'echo "100M" > /sys/fs/cgroup/myapp/memory.max'

# Add a process to the cgroup
echo $PID | sudo tee /sys/fs/cgroup/myapp/cgroup.procs

# === cgroups v1 (legacy) ===
# CPU shares (relative weight, default 1024)
sudo cgcreate -g cpu:mygroup
sudo cgset -r cpu.shares=512 mygroup
sudo cgclassify -g cpu:mygroup <PID>
```

```
# === Inspect container cgroups ===
# Inside a Docker container:
cat /proc/1/cgroup

# From host:
ls /sys/fs/cgroup/system.slice/docker-<CONTAINER_ID>.scope/

# === Resource limits via systemd ===
# In a systemd unit file:
[Service]
MemoryMax=256M
CPUQuota=50%
TasksMax=100

# === /proc/self/status != cgroup limits
# Remember: /proc/meminfo shows host memory, not cgroup memory.
# Use cgroup memory stats for container-aware monitoring.
cat /sys/fs/cgroup/memory.current
```

FAANG Interview Q&A

Q: 'A process on a production server is stuck in 'D' (uninterruptible sleep) state. What does this mean, and how do you diagnose it?'

A: 'D state means the process is waiting on I/O (disk or network) in a kernel operation that cannot be interrupted. Common causes: NFS server timeout, disk controller failure, slow storage. Diagnose by checking `ps aux | grep 'D'` to find the PID, then look at `/proc//stack` to see which kernel function is blocking, and `dmesg | tail` for I/O errors. If the underlying device recovers, processes leave D state — but sometimes they're stuck permanently and need a reboot.'

Q: 'Explain the difference between SIGTERM and SIGKILL. When would you use each?'

A: 'SIGTERM (15) is graceful termination — the process can catch it, clean up resources, flush buffers, and exit cleanly. SIGKILL (9) cannot be caught or ignored — the kernel immediately terminates the process without cleanup. Always try SIGTERM first. Use SIGKILL only when a process is ignoring SIGTERM or is stuck in an unkillable state (like D state, where even SIGKILL is queued and won't take effect until the kernel operation completes). At FAANG, our service managers (like our internal equivalent of systemd) send SIGTERM and wait 30 seconds before escalating to SIGKILL.'

Q: 'What is a zombie process, and why can't you kill it?'

A: 'A zombie process has already exited but still has an entry in the process table because its parent hasn't called wait() / waitpid() to read its exit status. It's dead — 'kill' won't work because there's nothing to signal. The fix is to kill the parent (PPID) so init (PID 1) adopts and reaps the zombie children. In production, a long-running parent with a buggy signal handler can accumulate thousands of zombies, exhausting the PID table.'

References & Further Reading

- [signal\(7\) man page](#) — Complete Linux signal reference
- [cgroups v2 Documentation](#) — Kernel.org official docs

- **The Linux Programming Interface** — Michael Kerrisk, the definitive reference on Linux processes and signals

FAANG Architect's Note: At FAANG scale, you rarely SSH into individual boxes to check processes. Instead, processes are monitored via centralized metrics (CPU, memory, FD count, context switches) and alerting systems. But when you *do* need to debug a single node — say, during a production incident where auto-scaling has failed — your ability to navigate `/proc`, interpret process states, and trace system calls is what separates the engineer who fixes the issue in 5 minutes from the one who's still reading man pages 30 minutes later. The tool changes with scale, but the fundamentals never do.

Production Tip: Set `ulimit -n` (file descriptors) appropriately for your workload. Web servers and databases commonly need 65536+ FDs. Check `cat /proc//limits` to verify running processes have the expected limits. A common FAANG incident root cause: a process inherited a low `ulimit` from its parent and hit 'too many open files' during traffic spike.

CHAPTER 9

Package Management

Systems & Operations · 7 topics

Package management is how Linux systems install, update, and remove software. At FAANG, you'll work across Debian-based (Ubuntu), RHEL-based (Amazon Linux 2/2023), and containerized environments. Understanding the differences between dpkg/APT, rpm/YUM/DNF, and compiling from source is essential for building reproducible infrastructure.

FAANG Linux apt yum dnf package management
containers

Debian/Ubuntu: APT & dpkg

Ubuntu (the most common dev environment at FAANG) uses **APT** (Advanced Package Tool) on top of **dpkg**. Packages are `.deb` files. `/etc/apt/sources.list` and `/etc/apt/sources.list.d/` define repositories. Always run `apt update` before `apt install` — this refreshes the package index.

```
# === Repository management ===
cat /etc/apt/sources.list           # Main repos
ls /etc/apt/sources.list.d/         # Third-party repos
(Docker, Kubernetes,
etc.)

# === APT commands ===
sudo apt update                    # Refresh package
index (must run first!)
```

```

sudo apt upgrade                                # Upgrade all
packages
sudo apt full-upgrade                          # Upgrade with
dependency changes
sudo apt autoremove                            # Remove orphaned
dependencies

sudo apt install nginx                        # Install package
sudo apt install -y nginx                    # Non-interactive
(script-friendly)
sudo apt remove nginx                        # Remove package
(leave config)
sudo apt purge nginx                         # Remove package +
config files

apt list --installed                          # List installed
packages
apt list --upgradable                       # List available
updates
apt search "web server"                     # Search packages
apt show nginx                              # Show package
details

# === dpkg (low-level) ===
dpkg -i package.deb                        # Install .deb file
directly
dpkg -r package                            # Remove package
dpkg -l                                    # List installed
packages
dpkg -L nginx                             # List files owned by
package
dpkg -S /usr/bin/nvim                      # Find which package
owns a file

# === Fix broken dependencies ===
sudo apt --fix-broken install                # Repair broken dpkg
state

# === Pinning package versions ===
# /etc/apt/preferences.d/pin-nginx
# Package: nginx
# Pin: version 1.24.*

```

```
# Pin-Priority: 1001

# === Installing specific version ===
apt list -a nginx                                # All available
versions
sudo apt install nginx=1.24.0-1ubuntu1           # Install specific
version
sudo apt-mark hold nginx                          # Prevent automatic
upgrade
sudo apt-mark unhold nginx                        # Re-enable upgrades
```

RHEL/CentOS/Amazon Linux: RPM & YUM/DNF

Amazon Linux 2 and Amazon Linux 2023 (the standard at AWS/FAANG) are RHEL-based. They use **RPM** (`.rpm` packages) with **YUM** (Amazon Linux 2) or **DNF** (Amazon Linux 2023) as the high-level package manager. The underlying tool is `rpm`.

```
# === YUM (Amazon Linux 2, CentOS 7) ===
sudo yum update -y                                # Update all
packages
sudo yum install -y nginx                         # Install
sudo yum remove nginx                             # Remove
sudo yum list installed                           # List installed
yum list available "nginx*"                       # Search available
yum info nginx                                    # Package details
sudo yum autoremove                               # Clean orphaned
deps
sudo yum history                                  # Transaction
history

# === DNF (Amazon Linux 2023, Fedora, RHEL 9) ===
sudo dnf install -y nginx
sudo dnf remove nginx
sudo dnf upgrade --refresh
sudo dnf list --installed
sudo dnf repolist                                # List configured
```

```

repos
sudo dnf provides /usr/bin/nginx          # Which package
provides a file?
sudo dnf history

# === RPM (low-level) ===
rpm -ivh package.rpm                     # Install RPM
rpm -e package                           # Remove RPM
rpm -qa                                  # Query all
installed packages
rpm -ql nginx                            # List files owned
by package
rpm -qf /usr/sbin/nginx                  # Find package
owning a file
rpm -qi nginx                             # Package info

# === EPEL (Extra Packages for Enterprise Linux) ===
sudo amazon-linux-extras install epel    # Amazon Linux 2
sudo dnf install epel-release             # Amazon Linux 2023
/ RHEL 9

# === Amazon Linux Extras ===
sudo amazon-linux-extras list             # Available topics
sudo amazon-linux-extras install nginx1   # Install from
extras

# === Locking versions ===
echo "exclude=nginx*" | sudo tee -a /etc/yum.conf # Prevent
updates (YUM)
sudo dnf versionlock add nginx*           # Lock version (DNF)

```

Compiling from Source & Alternatives

Sometimes you need software not in package repos, or you need specific compile-time flags. At FAANG, critical services are often compiled with custom optimization flags (PGO, LTO, specific CPU architectures). **Understanding the build chain (configure, make, make install) is essential for infrastructure engineers.**

```

# === Classic build chain ===
wget https://example.com/software-1.0.tar.gz
tar -xzf software-1.0.tar.gz
cd software-1.0/

# Step 1: Configure (check dependencies, set options)
./configure --prefix=/usr/local --enable-optimizations

# Step 2: Build
make -j$(nproc)                                # Parallel build
using all cores

# Step 3: Install
sudo make install                                # Copy to system
directories

# === Common configure flags ===
# --prefix=/usr      - Install to /usr (not /usr/local)
# --sysconfdir=/etc   - Config files location
# --with-ssl          - Enable SSL support
# --enable-debug      - Build with debug symbols

# === Checkinstall (create a .deb/.rpm from source) ===
sudo apt install checkinstall
sudo checkinstall make install                    # Creates and
installs a .deb

# === Stow (manage manually compiled software) ===
# Install each version under /usr/local/stow/<pkg>-<ver>/
# Symlink to /usr/local/ with stow
sudo stow -d /usr/local/stow -S nginx-1.24

# === FAANG pattern: hermetic builds ===
# At FAANG, we don't compile on production servers.
# Builds happen in hermetic build systems (e.g., Bazel, Buck).
# Artifacts are versioned and deployed via package managers or
containers.

# === Alternatives: Homebrew on Linux ===
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

```

```
"  
brew install nginx  
brew services start nginx
```

Docker & Containers: The Modern "Package"

At FAANG, most services are packaged as **container images** rather than native packages. Containers bundle the application + its dependencies + the package manager snapshot.

Understanding the **Dockerfile** and **multi-stage builds** is essential for modern infrastructure.

```
# === Dockerfile with package management ===  
FROM ubuntu:22.04 AS builder  
  
# Always combine apt update + install in ONE RUN layer  
RUN apt-get update && apt-get install -y \  
    build-essential \  
    libssl-dev \  
    && rm -rf /var/lib/apt/lists/*  
  
# Multi-stage build: builder → runtime  
FROM ubuntu:22.04 AS runtime  
  
# Copy only the compiled binary, not the build tools  
COPY --from=builder /app/compiled_binary /usr/local/bin/  
  
# Distroless images (no package manager at all)  
FROM gcr.io/distroless/base-debian12:latest  
  
# === Best practices ===  
# 1. Pin base image digests (not tags)  
#    FROM ubuntu:22.04@sha256:abc123...  
# 2. Combine RUN apt-get update && apt-get install (cache busting)
```

```
# 3. Clean apt cache in same layer: rm -rf /var/lib/apt/lists/*  
# 4. Use --no-install-recommends to reduce image size  
# 5. Never leave a package manager in the final image
```

FAANG Interview Q&A

Q: 'You need to install a specific version of nginx on Ubuntu, but apt installs the wrong version. What do you do?'

A: 'First, run `apt list -a nginx` to see all available versions. If my version is there, install it with `apt install nginx=<version>`. If not, I'd add the official nginx repository to `/etc/apt/sources.list.d/` with the correct distribution codename. As a last resort, I'd download the .deb directly or compile from source. At FAANG, we would never apt install on production — instead, we'd bake the correct version into an AMI or container image.'

Q: 'What's the difference between apt update and apt upgrade?'

A: '`apt update` downloads the latest package lists from repositories — it tells you what's available. `apt upgrade` actually installs newer versions of installed packages based on those updated lists. Running upgrade without update will give you stale information.'

Q: 'How do package managers resolve dependencies?'

A: 'APT, YUM, and DNF maintain a dependency graph. Each package declares its dependencies (Depends:), conflicts (Conflicts:), and provides (Provides:). The solver finds a consistent

set of package versions satisfying all constraints. If there's an unresolvable conflict, the tool reports it. At FAANG, we avoid these issues by using deterministic builds and container images with fixed package snapshots.'

References & Further Reading

- **Debian APT Wiki** — Official APT documentation
- **DNF Documentation** — Official DNF reference
- **Dockerfile Best Practices** — Official Docker guide

FAANG Architect's Note: In 15+ years across multiple FAANG companies, I've seen package management evolve from 'YUM on a fleet of pets' to 'everything runs in containers with immutable base images.' The days of SSH'ing into a box and running apt-get are over at scale. But the *concepts* live on: dependencies, version pinning, repository management, and build toolchains are just as relevant — they've moved upstream into the CI/CD pipeline. If you understand RPM and dpkg internals, you'll understand container layer caching and multi-stage builds intuitively.

Production Tip: Never run unattended-upgrades on production servers. At FAANG, package updates are applied by rebuilding AMIs or rolling out new container versions — never in-place on running instances. This ensures that every instance is identical and rollbacks are as simple as pointing to the previous image.

CHAPTER 10

systemd: The Modern Init System

Systems & Operations · 6 topics

systemd is the init system and service manager for virtually every modern Linux distribution. It replaced SysV init and handles service startup, dependency management, logging, timers, socket activation, and resource control. At FAANG, understanding systemd is essential for writing production service units, debugging boot issues, and building container runtimes.

FAANG

Linux

systemd

journal

init

service management

Core Concepts: Units & Targets

systemd manages **units**. There are several types: **service** (daemons), **socket** (socket activation), **timer** (scheduled tasks, replacing cron), **mount** (filesystem mounts), **target** (group of units, replacing runlevels). Units are defined in `.unit` files located in `/usr/lib/systemd/system/` (distro) and `/etc/systemd/system/`

```
# === Basic systemctl commands ===
systemctl status nginx          # Status + recent logs
systemctl start nginx           # Start immediately
systemctl stop nginx            # Stop
systemctl restart nginx         # Restart (stop + start)
systemctl reload nginx          # Send SIGHUP (reload
                                config)
```

```
systemctl enable nginx          # Start on boot (creates
symlinks)
systemctl disable nginx         # Remove boot symlinks
systemctl daemon-reload        # Reload unit files after
edits
systemctl is-active nginx       # Check if running (exit
code 0/1)
systemctl is-enabled nginx      # Check if enabled

# === List units ===
systemctl list-units            # All active units
systemctl list-units --all      # All units (including
inactive)
systemctl list-units --type=service # Only services
systemctl list-unit-files       # All unit files + enable
state
systemctl list-dependencies nginx # Show dependency tree

# === Targets (runlevels) ===
systemctl list-units --type=target # List targets
systemctl get-default              # Default boot target
systemctl set-default multi-user.target # Boot to multi-user
(no GUI)
systemctl isolate emergency.target # Switch to emergency
mode

# === Legacy runlevel mapping ===
# runlevel 0 → poweroff.target
# runlevel 1 → rescue.target
# runlevel 3 → multi-user.target
# runlevel 5 → graphical.target
# runlevel 6 → reboot.target
```

Writing a Service Unit File

A `.service` unit defines how to start, stop, and manage a daemon. **The [Unit] section describes dependencies, [Service] defines execution, and [Install] controls enable/disable behavior.** At FAANG, every internally developed service ships with a production-grade systemd unit file that includes resource limits, health checks, and graceful shutdown settings.

```
# Example: /etc/systemd/system/myapp.service
[Unit]
Description=My Production Service
After=network-online.target
Wants=network-online.target
Requires=postgresql.service
After=postgresql.service

[Service]
Type=simple                # Default: process doesn't fork
User=myapp
Group=myapp
WorkingDirectory=/opt/myapp
ExecStart=/usr/bin/myapp --config /etc/myapp/config.yaml
ExecReload=/bin/kill -HUP $MAINPID
ExecStop=/bin/kill -TERM $MAINPID
Restart=always             # Restart on crash
RestartSec=10              # Wait 10s before restart
TimeoutStartSec=30         # Max time to start
TimeoutStopSec=30         # Max time to stop gracefully

# Resource limits
LimitNOFILE=65536
LimitNPROC=4096
MemoryMax=512M
CPUQuota=50%

# Security
ProtectSystem=strict
ProtectHome=true
PrivateTmp=true
NoNewPrivileges=true
```

```

CapabilityBoundingSet=CAP_NET_BIND_SERVICE

[Install]
WantedBy=multi-user.target

# === Type variants ===
# Type=simple      - Main process is ExecStart (default)
# Type=forking     - Process forks (daemonizes); notify when
parent exits
# Type=oneshot     - Runs once (for setup/scripts);
RemainAfterExit=yes
# Type=notify      - Process calls sd_notify() when ready
# Type=dbus        - Process registers on D-Bus; wait for bus name

# === Environment ===
# Set environment variables:
[Service]
Environment=MYAPP_ENV=production
Environment=JAVA_HOME=/usr/lib/jvm/java-11-openjdk
EnvironmentFile=/etc/default/myapp    # File with KEY=VAL pairs

# === Override without editing original ===
sudo systemctl edit nginx            # Creates
/etc/systemd/system/nginx.service.d/override.conf
sudo systemctl cat nginx            # Show full unit with overrides
applied

```

journalctl: The systemd Logging System

systemd replaced syslog with **journald**, a binary logging system with structured metadata (priority, facility, PID, UID, boot ID).

journalctl queries the journal. **In production debugging, journalctl -xe is often the first command run during an incident.**

```
# === Basic journalctl usage ===
journalctl                                # All logs (pager)
journalctl -n 50                          # Last 50 lines
journalctl -f                             # Follow (like tail -f)
journalctl -u nginx                       # Logs for a specific unit
journalctl -u nginx -u postgresql        # Multiple units

# === Time-based filtering ===
journalctl --since "5 min ago"
journalctl --since today
journalctl --since "2024-01-01" --until "2024-01-02"
journalctl --since yesterday -u nginx

# === Priority levels ===
journalctl -p err                         # Only errors and above
journalctl -p warning                     # Warnings and above
# Levels: emerg(0), alert(1), crit(2), err(3), warning(4),
# notice(5), info(6), debug(7)

# === Output formats ===
journalctl -o verbose                     # All structured fields
journalctl -o json                       # JSON output (for jq / log
aggregation)
journalctl -o json-pretty

# === Disk usage ===
journalctl --disk-usage
sudo journalctl --vacuum-size=500M        # Keep only 500MB of logs
sudo journalctl --vacuum-time=7d          # Keep only 7 days

# === Boot-specific ===
journalctl -b -1                          # Previous boot (useful after
crash)
journalctl -b -2                          # Two boots ago
journalctl --list-boots                   # List boot numbers + IDs

# === Log a test message ===
logger "Test log message"                 # Write to systemd journal
logger -p user.err "Custom error"         # With priority

# === Config: /etc/systemd/journald.conf ===
```

```
# SystemMaxUse=1G
# Compress=yes
# ForwardToSyslog=no
```

Timers (Replacing Cron) & Socket Activation

systemd timers replace cron with fine-grained scheduling, dependency support, and logging. **Socket activation allows services to start on demand when a connection arrives — a lazy-start pattern used by many system services.**

```
# === Timer example ===
# /etc/systemd/system/backup.service
[Service]
Type=oneshot
ExecStart=/usr/local/bin/backup.sh

# /etc/systemd/system/backup.timer
[Unit]
Description=Daily Backup
Requires=backup.service

[Timer]
OnCalendar=daily
Persistent=true           # If missed (e.g., during downtime), run
                           immediately
RandomizedDelaySec=3600   # Random delay within 1 hour (avoid
                           thundering herd)

[Install]
WantedBy=timers.target

# === Manage timers ===
systemctl enable --now backup.timer
systemctl list-timers
# NEXT                LEFT                LAST
PASSED              UNIT
```

```
# Mon 2024-01-15 03:00:00    14h left    Sun 2024-01-14 03:00:01
9h ago
backup.timer

# === OnCalendar patterns ===
# daily          → Every day at 00:00:00
# hourly         → Every hour at :00
# Mon *-*- * 09:00:00 → Every Monday at 9 AM
# *-*-01 00:00:00  → First of every month
# *:0/15         → Every 15 minutes

# === Socket activation ===
# Listen on a port; start service on first connection
# /etc/systemd/system/echo.socket
[Socket]
ListenStream=2000

[Install]
WantedBy=sockets.target

# /etc/systemd/system/echo@.service
[Service]
ExecStart=/usr/local/bin/echo-server
StandardInput=socket

# === Benefits of socket activation ===
# 1. Lazy-start: service consumes resources only when needed
# 2. Resilience: if service crashes, socket survives, next request
restarts it
# 3. Parallel boot: services can start while socket is already
listening
```

System Analysis & Debugging

systemd provides powerful tools for analyzing boot performance, system state, and unit interactions. **At FAANG, these tools are used in periodic 'boot health' checks and capacity planning reviews.**

```
# === Boot analysis ===
systemd-analyze                # Total boot time
systemd-analyze blame          # Time taken by each unit
                                (sorted)
systemd-analyze critical-chain # Critical chain of slowest
dependencies
systemd-analyze plot > boot.svg # Generate a visualization

# === Unit dependency graph ===
systemctl list-dependencies nginx
systemd-analyze dot nginx | dot -Tsvg > nginx-deps.svg # Visual
graph

# === Failed units ===
systemctl --failed             # List failed units

# === Check if systemd is running ===
ps -p 1 -o comm=              # Should print 'systemd'

# === Emergency / Recovery ===
systemctl rescue              # Rescue mode (single-user,
minimal services)
systemctl emergency           # Emergency shell (PID 1 only,
root filesystem ro)

# === Common issues ===
# 1. 'Unit not found' – systemctl daemon-reload
# 2. 'Dependency failed' – check required units (systemctl list-
dependencies)
# 3. 'Start request repeated too quickly' – RestartSec is too short
# 4. 'Failed to connect to bus' – systemd is not PID 1 (e.g.,
inside container without
systemd)
```

FAANG Interview Q&A

Q: 'How does systemd start services in parallel during boot?'

A: 'systemd parses all unit files and builds a dependency graph. Units without inter-dependencies start in parallel using socket activation — even if a service isn't ready, its socket is listening, so dependent services don't block. The `After=` directive doesn't mean 'wait until it's started' — it means 'this unit should be in the dependency graph after the other.' Actual ordering is managed by `Type=notify` or `Requires=` with `After=`.'

Q: 'How would you debug a service that keeps crashing on startup?'

A: 'First, run `journalctl -u <service> -n 50 --no-pager` to see the logs. Then check `systemctl status <service>` for the exit code and recent journal entries. Common causes: missing environment variables, permission issues (check `journalctl` for 'Permission denied'), or dependency failures. I'd also check `systemctl list-dependencies` to ensure all required units are running. If the service restarts in a loop, check `RestartSec=` and increase it to avoid log flooding.'

Q: 'What's the difference between systemd and Docker?'

A: 'systemd is the init system for the host — it manages system services, mounts, sockets, and the boot process. Docker manages application containers. systemd can also manage containers (systemd-nspawn, podman with systemd integration), but their scope is different. In production, many FAANG services run inside

Docker containers on hosts that use systemd — the host systemd manages containerd/dockerd, and inside the container, the application might use a minimal init or no init at all.'

References & Further Reading

- **systemd Documentation** — Official man pages and guides
- **systemd Essentials** — DigitalOcean tutorial series
- **Linux systemd: A Practical Guide** — In-depth reference for system administration

FAANG Architect's Note: systemd is one of the most controversial pieces of Linux infrastructure — some love it, some hate it. But at FAANG, we don't care about the philosophical debate. We care that systemd provides a consistent, declarative, and dependency-aware way to manage services across tens of thousands of hosts. The ability to write a correct unit file, understand journald's structured logging, and debug boot issues is table stakes for any production engineer.

Production Tip: Always set `TimeoutStopSec=` in your service units. If your service doesn't stop within the timeout, systemd sends `SIGKILL`. Database services and stateful applications need extra time to flush buffers. A good pattern: set `TimeoutStopSec=120` for databases and 30 for stateless web services. Also, never forget `systemctl daemon-reload` after editing unit files — one of the most common 'why isn't my change working?' mistakes.

CHAPTER 11

Users, Groups & Authentication

Systems & Operations · 8 topics

Linux is a multi-user operating system from the ground up. Every process runs as a specific user, every file has an owner and group, and access decisions are made by the kernel based on user/group IDs. At FAANG, understanding user management, PAM (Pluggable Authentication Modules), and sudo policies is critical for security, compliance, and least-privilege deployments.

FAANG Linux users groups sudo PAM authentication security

User & Group Fundamentals

Users are identified by **UID** (User ID) and groups by **GID** (Group ID). The mapping is stored in `/etc/passwd`, `/etc/shadow` (passwords), `/etc/group`, and `/etc/gshadow`. **UID 0 is root** — the superuser with unrestricted access. At FAANG, human engineers almost never use root — they sudo to specific commands.

```
# === Key files ===
cat /etc/passwd      # User accounts:
username:x:UID:GID:comment:home:shell
cat /etc/shadow      # Password hashes (root only)
cat /etc/group        # Group definitions:
groupname:x:GID:member1,member2
cat /etc/gshadow      # Group passwords (rarely used)
```

```

# Understanding /etc/passwd format:
# kallol:x:1000:1000:Kallol:/home/kallol:/bin/bash
# [1] username
# [2] password placeholder (x = hash in /etc/shadow)
# [3] UID (User ID)
# [4] GID (Primary Group ID)
# [5] GECOS (user info, often empty)
# [6] Home directory
# [7] Login shell

# === Adding users ===
sudo useradd -m -s /bin/bash -G sudo,docker kallol
# -m create home directory
# -s login shell
# -G supplementary groups
# -c comment (full name)

sudo useradd --system --no-create-home --shell /usr/sbin/nologin
myapp
# System user (UID < 1000), no login, no home

sudo passwd kallol          # Set/change password
sudo usermod -aG docker kallol # Add to group (append, don't
forget -a!)
sudo userdel -r kallol      # Delete user + home + mail spool

# === Listing users ===
who                          # Currently logged in users
w                            # Detailed who (login time, idle, what
running)
users                        # Just usernames
id                           # Current user's UID, GID, groups
id kallol                   # Specific user's IDs
finger kallol               # User info (may need apt install finger)

# === Listing groups ===
groups                      # Current user's groups
groups kallol              # Specific user's groups

```

```
getent group sudo      # Members of 'sudo' group
lid -g sudo           # List members (may need apt install
libuser)
```

sudo: Delegated Privilege

sudo allows authorized users to run commands as another user (typically root). **At FAANG, sudo is the primary mechanism for privileged access — root login is disabled.** The sudoers file (`/etc/sudoers`) is a critical security boundary and must be edited with `visudo` to prevent syntax errors that could lock you out.

```
# === /etc/sudoers basics (edit with visudo!) ===

# Simple rules:
kallol ALL=(ALL:ALL) ALL           # kallol can do anything
kallol ALL=(ALL:ALL) NOPASSWD: ALL # No password prompt (dev VMs
only)

# Command-level restrictions:
kallol ALL=(root) /usr/bin/systemctl restart nginx
kallol ALL=(root) /usr/bin/journalctl

# Group-based:
%sudo ALL=(ALL:ALL) ALL           # sudo group members have
full access
%admin ALL=(ALL) ALL              # admin group

# === sudo command usage ===
sudo -l                           # List allowed commands for
current user
sudo -u www-data whoami           # Run as www-data
sudo -s                           # Root shell (not
recommended)
sudo -i                           # Root login shell (loads
root's profile)
sudo -E                           # Preserve environment
```

```

variables

# === /etc/sudoers.d/ - Drop-in files ===
# Better than editing /etc/sudoers directly:
# /etc/sudoers.d/deploy
%deploy ALL=(root) /usr/local/bin/deploy.sh

# === Security best practices ===
# 1. Defaults: !root_sudo          - root should use sudo too (for
audit trail)
# 2. Defaults: log_output          - log all sudo commands
# 3. Defaults: timestamp_timeout=5 - re-authenticate after 5 min
# 4. Cmnd_Alias for command groups

Cmnd_Alias NETWORKING = /sbin/ip, /sbin/ss, /sbin/tcpdump
Cmnd_Alias SERVICES = /usr/bin/systemctl *
kallol ALL=(root) NETWORKING, SERVICES

```

PAM: Pluggable Authentication Modules

PAM separates authentication logic from applications. When you SSH in, log in on a TTY, or run `sudo`, the application uses PAM modules to authenticate. **At FAANG, PAM is configured to enforce multi-factor authentication (MFA), LDAP/SSO integration, and hardware key requirements.**

```

# === PAM configuration files ===
ls /etc/pam.d/
# common-auth          - Authentication methods
# common-account       - Account restrictions (expiry, time)
# common-password      - Password quality rules
# common-session       - Session setup (environment, limits)

# === PAM module types ===
# auth      - Verify identity (password, MFA, SSH key)
# account   - Check account status (expired, locked, time-based)
# password  - Update authentication tokens

```

```
# session - Set up environment (open sessions, mount home)

# === Control flags ===
# requisite - Fail immediately if this module fails
# required - Fail after all modules run (don't reveal which
#           failed)
# sufficient - If this passes, skip remaining modules
# optional - Used only if no other module succeeds

# === Example: /etc/pam.d/common-auth ===
auth    [success=1 default=ignore]      pam_unix.so nullok
auth    requisite                       pam_deny.so
auth    required                         pam_permit.so

# === /etc/security/limits.conf (PAM limits) ===
# These apply to PAM sessions (SSH, login, etc.)
kallol soft nfile 65536
kallol hard nfile 65536
@deploy soft nproc 4096

# === Check PAM config ===
sudo pam-auth-update          # Ubuntu: enable/disable modules
pam_tally2 --user kallol      # Show failed login count
sudo pam_tally2 --user kallol --reset # Reset failed count
```

Service Accounts & Least Privilege

Every daemon should run as its own dedicated user with minimum required permissions. **At FAANG, 'root in containers' is a common security finding — each service must have a dedicated non-root user.** The principle of least privilege (PoLP) means: no process should have access rights beyond what it strictly needs.

```
# === Creating a service account (best practices) ===
sudo useradd \
    --system \
```

```

--no-create-home \
--shell /usr/sbin/nologin \
--user-group \
myapp

# Verify:
grep myapp /etc/passwd
# myapp:x:998:998::/nonexistent:/usr/sbin/nologin

# === Grant minimum permissions ===
# Give service account access to specific directories
sudo mkdir -p /var/log/myapp /var/lib/myapp /etc/myapp
sudo chown myapp:myapp /var/log/myapp /var/lib/myapp /etc/myapp
sudo chmod 750 /var/log/myapp /var/lib/myapp /etc/myapp

# === Capabilities (fine-grained root privileges) ===
# Instead of running as root, grant specific capabilities:
# CAP_NET_BIND_SERVICE - bind to ports < 1024
# CAP_SYS_TIME          - change system clock
# CAP_DAC_OVERRIDE      - bypass file permission checks

# In systemd unit:
[Service]
User=myapp
AmbientCapabilities=CAP_NET_BIND_SERVICE
CapabilityBoundingSet=CAP_NET_BIND_SERVICE

# === Docker: running as non-root ===
FROM node:22-alpine
RUN addgroup -S myapp && adduser -S myapp -G myapp
USER myapp
COPY --chown=myapp:myapp . /app

# === /etc/login.defs ===
# Global user creation defaults:
# UID_MIN 1000
# UID_MAX 60000
# SYS_UID_MIN 100
# SYS_UID_MAX 999
# PASS_MAX_DAYS 90

```

FAANG Interview Q&A

Q: 'You SSH into a server and get Permission denied (publickey). What do you check?'

A: 'First, verify my SSH key is in `~/.ssh/authorized_keys` on the server with correct permissions (600). Check `/etc/ssh/sshd_config` : PubkeyAuthentication yes, PasswordAuthentication no (usually). Check `~/.ssh` permissions (700) and that the user's home directory is not group-writable. Check `/etc/security/access.conf` and PAM config in `/etc/pam.d/sshd` . Also check SELinux/AppArmor status with `getenforce` . At FAANG, SSH key rotation is automated, but manual troubleshooting still requires knowing these checks.'

Q: 'What's the difference between su and sudo?'

A: ' `su` switches to another user completely (requires their password), starting a new shell. `sudo` runs a single command as another user (requires your password). `sudo` is preferred because: (1) it creates an audit trail, (2) it grants specific commands, not full access, (3) it doesn't require sharing the root password. At FAANG, `su` is effectively banned — all privilege elevation goes through `sudo` with centralized audit logging.'

Q: 'A user left the company. What steps do you take?'

A: 'Immediately disable their account: `sudo usermod -L username` (lock password) and remove SSH keys. Revoke sudo access: remove from `sudoers`. Kill their active sessions: `kill -u username` .

Archive their home directory for compliance. Remove the account after the compliance retention period. At FAANG, this is automated in the HR → IT pipeline — when an employee is terminated in Workday, their accounts are deprovisioned within minutes across all systems.'

References & Further Reading

- [passwd\(5\) man page](#) — Format of /etc/passwd
- [sudoers\(5\) man page](#) — Complete sudoers syntax reference
- [Arch Linux PAM Wiki](#) — Excellent reference on PAM modules and configuration

FAANG Architect's Note: User management is often overlooked by engineers focused on 'the fun stuff' — distributed systems, ML pipelines, Kubernetes. But as a production engineer, you will spend significant time on access control. Every service needs a service account. Every human needs just enough access to do their job. Every SSH key and IAM role must be auditable. The teams that ignore user management are the teams that get paged at 3 AM because someone accidentally ran 'rm -rf' as root on a shared filesystem.

Production Tip: Never, ever disable password expiry for human accounts. Use `chage -M 90 username` to enforce 90-day password rotation. For service accounts, use SSH keys with: (1) no passphrase (script-friendly), (2) `from=` restriction in `authorized_keys` (limit to specific source IPs), (3) `command=` restriction (limit to specific commands). Example:

```
from="10.0.0.0/8"
command="/usr/local/bin/deploy" ssh-rsa AAA...
```

CHAPTER 12

SSH: Secure Shell

Systems & Operations · 6 topics

SSH (Secure Shell) is the universal protocol for remote access to Linux systems. Despite the rise of 'no-SSH' architectures (immutable infrastructure, containers, session managers), SSH remains essential for incident response, debugging, and infrastructure management. At FAANG, SSH is typically gated behind bastion hosts, certificate-based auth, and session recording.

FAANG

Linux

SSH

security

tunneling

authentication

SSH Authentication: Keys, Certificates & Agents

Password-based SSH access is a security risk and is generally disabled in production. **Key-based authentication is the minimum standard. At FAANG, SSH certificate authorities (CAs) are used — users get short-lived certificates signed by a central CA.** The SSH agent handles key forwarding so you don't store private keys on jump boxes.

```
# === Key generation ===  
# ED25519 is the modern default (faster, smaller, more secure than  
RSA)  
ssh-keygen -t ed25519 -C "kallol@company.com"  
  
# RSA with 4096 bits (legacy compatibility)  
ssh-keygen -t rsa -b 4096 -C "kallol@company.com"
```

```
# === Key files ===
# ~/.ssh/id_ed25519      - Private key (NEVER share, chmod 600)
# ~/.ssh/id_ed25519.pub  - Public key (safe to share)
# ~/.ssh/authorized_keys - Public keys allowed to log in (on the
server)
# ~/.ssh/known_hosts     - Known server host keys (on the client)

# === Copy key to server ===
ssh-copy-id user@server          # Copy public key to
authorized_keys
# Manual equivalent:
cat ~/.ssh/id_ed25519.pub | ssh user@server "mkdir -p ~/.ssh && cat
>>
~/.ssh/authorized_keys"

# === SSH Agent (key forwarding) ===
eval "$(ssh-agent -s)"          # Start agent
ssh-add ~/.ssh/id_ed25519       # Add private key to
agent
ssh-add -l                     # List loaded keys
ssh-add -L                     # List loaded public
keys

# === SSH Agent Forwarding ===
# Connect to bastion and from there to internal hosts
ssh -A user@bastion             # -A forwards your agent
# WARNING: agent forwarding is powerful but risky – root on bastion
can use your agent
# Better: use ProxyJump (see below)

# === Certificate-based SSH (FAANG standard) ===
# Server-side: /etc/ssh/ca.pub added to TrustedUserCAKeys
# User gets short-lived cert signed by corporate CA
ssh-keygen -s user_ca -I user@company -n kali -V +52w user.pub
# Result: user-cert.pub (valid for 52 weeks, typically rotated more
often)
```

SSH Client Configuration (~/.ssh/config)

A well-organized SSH config is the hallmark of an experienced engineer. It eliminates repetitive flags, handles jump hosts, and manages keys per host. **At FAANG, your SSH config manages bastion hosts, different keys for different environments, and connection persistence.**

```
# ~/.ssh/config – Highly recommended

# Global defaults
Host *
    ServerAliveInterval 60
    ServerAliveCountMax 3
    StrictHostKeyChecking accept-new
    UserKnownHostsFile ~/.ssh/known_hosts
    AddKeysToAgent yes
    UseKeychain yes                # macOS: store passphrase
    in keychain

# Bastion host (jump box)
Host bastion
    HostName bastion.corp.example.com
    User kali
    IdentityFile ~/.ssh/id_ed25519
    ForwardAgent no                # Safer: don't forward
    agent on bastion

# Internal hosts (via bastion proxy)
Host bastion-proxy bastion.corp.example.com

Host *.internal.example.com
    User kali
    ProxyJump bastion              # Jump through bastion (no
    agent forwarding needed)
    IdentityFile ~/.ssh/id_ed25519

# Specific server with different key
Host prod-db-*
    User admin
    Port 2222
```

```

    IdentityFile ~/.ssh/prod-key
    ProxyJump bastion

# GitHub / GitLab
Host github.com
    HostName github.com
    User git
    IdentityFile ~/.ssh/id_ed25519
    IdentitiesOnly yes

# === Usage ===
ssh internal-web-01.internal.example.com # Automatically uses
bastion
ssh bastion                             # Direct bastion access

```

SSH Server Hardening

The SSH daemon (`sshd`) is the most exposed service on any Linux server. **At FAANG, SSH hardening is mandatory: no passwords, no root login, custom ports are discouraged (security by obscurity is not security), and all sessions are logged.**

```

# /etc/ssh/sshd_config - Security hardening

# Authentication
PubkeyAuthentication yes           # Key-based auth only
PasswordAuthentication no         # Disable passwords (must
have keys working first!)
PermitRootLogin without-password  # Or 'prohibit-password'
(disallow root passwords)
# PermitRootLogin no              # Even better: no root SSH at
all

# Keys
AuthorizedKeysFile .ssh/authorized_keys
TrustedUserCAKeys /etc/ssh/ca.pub  # If using SSH CA

```

```

# Network
Port 22                                # Using non-standard port
just annoys attackers
slightly
ListenAddress 0.0.0.0                 # Bind to specific IP if
possible

# Session
ClientAliveInterval 300                # 5 min keepalive
ClientAliveCountMax 0                 # Disconnect after 1 missed
keepalive
MaxAuthTries 3                        # Limit auth attempts
MaxSessions 10                        # Max concurrent sessions per
connection
LoginGraceTime 30                     # 30 seconds to authenticate

# Forwarding
AllowAgentForwarding no               # Disable on bastions
(security)
AllowTcpForwarding yes                # Enable only if needed
X11Forwarding no                     # Almost never needed

# Logging
LogLevel VERBOSE                      # Log fingerprints (for
audit)

# === After changes ===
sshd -t                               # Validate config before
restart
sudo systemctl restart sshd

# === Fail2ban (supplementary protection) ===
sudo apt install fail2ban
# /etc/fail2ban/jail.local
[sshd]
enabled = true
maxretry = 3
bantime = 3600

# === Check current SSH sessions ===

```

```
ss -tnp | grep :22          # Active SSH connections
who                          # Logged in users
sudo journalctl -u sshd -n 20 # Recent SSH auth logs
```

SSH Tunneling & Port Forwarding

SSH can tunnel arbitrary TCP connections through encrypted channels. This is essential for accessing internal services (databases, monitoring dashboards) without exposing them to the network. **At FAANG, SSH tunnels are a common pattern for 'break glass' database access.**

```
# === Local Port Forwarding ===
# Access internal service via local port
# Syntax: ssh -L local_port:target_host:target_port bastion
ssh -L 5432:internal-db.internal:5432 bastion
# Now connect to localhost:5432 → encrypted → internal-db:5432

# === Remote Port Forwarding ===
# Expose local port on remote server (reverse tunnel)
# Syntax: ssh -R remote_port:local_host:local_port bastion
ssh -R 8080:localhost:3000 bastion
# Now bastion:8080 → encrypted → localhost:3000

# === Dynamic Port Forwarding (SOCKS proxy) ===
# Create a SOCKS5 proxy through the SSH connection
ssh -D 1080 bastion
# Configure browser/system SOCKS proxy to localhost:1080
# Now all traffic routes through the bastion

# === Jump host (ProxyJump) ===
# Newer, simpler alternative to chaining SSH tunnels
ssh -J bastion internal-host

# === SSH through multiple hops ===
ssh -J bastion,internal-bastion target-host
```

```
# === Common tunnel patterns ===  
# Database access via bastion  
ssh -L 3306:prod-db-1.internal:3306 -N bastion &  
# -N = don't open a shell, just forward ports  
  
# RDP to Windows jump box behind bastion  
ssh -L 3389:windows-jump.internal:3389 bastion  
  
# === Auto-reconnecting tunnel (autossh) ===  
autossh -M 0 -o "ServerAliveInterval 60" -o "ServerAliveCountMax 3"  
-L 5432:db:5432  
bastion
```

SSH Troubleshooting & Debugging

When SSH doesn't work, the default error messages can be cryptic. **Verbose mode (-vvv) is your primary diagnostic tool.** At FAANG, SSH issues are a common on-call triage category — knowing how to diagnose quickly is essential.

```
# === Verbose mode ===  
ssh -vvv user@server # Detailed debug output  
  
# Common issues by verbose output analysis:  
# 1. "Permission denied (publickey)" – No matching key in  
authorized_keys  
# 2. "Connection refused" – sshd not running or port wrong  
# 3. "Connection timed out" – Firewall blocking (check security  
groups)  
# 4. "Host key verification failed" – Host key changed (MITM or  
server rebuild)  
# Fix: ssh-keygen -R "hostname" then reconnect  
# 5. "Too many authentication failures" – ssh is trying too many  
keys  
# Fix: ssh -o IdentitiesOnly=yes -i specific_key user@host  
  
# === Test connectivity ===
```

```

nc -zv server 22                # Check if port 22 is open
telnet server 22                # Raw connection test

# === Check host keys ===
ssh-keyscan server              # Get server's public host
key
ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub # Server host key
fingerprint

# === Check which key is being offered ===
ssh -v user@server 2>&1 | grep "Offering public key"

# === Known hosts issues ===
cat ~/.ssh/known_hosts          # List known hosts
ssh-keygen -F server            # Search known_hosts for a
host
ssh-keygen -R server            # Remove host from
known_hosts

# === Agent issues ===
ssh-add -l                      # Check which keys are loaded
ssh-add -D                      # Clear all keys
ssh -o IdentitiesOnly=yes -i ~/.ssh/custom_key user@host # Force
specific key

# === SCP & SFTP ===
scp file.txt user@server:/remote/path/
scp -r dir/ user@server:/remote/path/
rsync -avz -e ssh dir/ user@server:/remote/path/ # Better for
large transfers
sftp user@server                # Interactive file transfer

```

FAANG Interview Q&A

Q: 'How would you design SSH access for a fleet of 10,000 servers?'

A: 'No direct SSH. All access goes through bastion hosts. Bastions

are locked down — SSH only from corporate VPN, MFA required, no agent forwarding. Users authenticate with SSH certificates signed by a central CA — certificates expire after 4-8 hours. All sessions are recorded (script, tlog). Host keys are distributed via configuration management (Ansible/Chef). Servers have no individual user accounts — engineers authenticate via their certificate, which maps to a local account via `AuthorizedPrincipalsFile`. This gives us audit trails, revocability, and no key distribution problem.'

Q: 'You can't SSH into a production server. What do you check?'

A: 'Layer by layer: (1) Network: can I ping it? `nc -zv host 22` ? Check security groups/firewall rules. (2) Service: is sshd running? Check via monitoring or console access. (3) Authentication: am I using the right key? `ssh -vvv` . Check my certificate hasn't expired. (4) Server config: is sshd_config correct? `sshd -t` remotely via console. (5) PAM/SELinux: is SELinux blocking? `ausearch -m avc -ts recent` . At FAANG, 'console access' means AWS Systems Manager Session Manager or the equivalent — we always have a backup access method.'

Q: 'What's the difference between SSH tunneling and a VPN?'

A: 'An SSH tunnel is a single TCP port forwarded through an encrypted channel — it's application-specific. A VPN creates a virtual network interface and routes all (or selected) traffic through an encrypted tunnel — it's network-level. SSH tunneling is simpler and doesn't require admin privileges (just SSH access),

but it only works for TCP and requires manual setup per port. VPNs are transparent to applications but need client software and more infrastructure. At FAANG, both are used: VPN for general corporate access, SSH tunnels for granular 'break glass' access to specific services.'

References & Further Reading

- **SSH Protocol Details** — Comprehensive guide to the SSH protocol
- **ssh_config(5) man page** — All SSH client options
- **SSH Agent Explained** — Deep dive into SSH agent internals

FAANG Architect's Note: The 'no SSH' trend in infrastructure (immutable servers, containers, 'cattle not pets') is real — at FAANG, the vast majority of interactions with servers are automated through deployment pipelines, orchestration systems, and API-driven management. But when something goes wrong, SSH is still the emergency brake. Every senior engineer I know has stories of SSH'ing into a server at 3 AM to fix a critical issue. The difference at FAANG is that SSH access is audited, limited, and temporary — but it still exists, and knowing how to use it effectively is a survival skill.

Production Tip: Always test SSH config changes with a second session before closing the first. A common production outage scenario: an engineer edits `sshd_config`, restarts `sshd`, and the new config has a syntax error that kills all active sessions — locking everyone out. Keep an active 'emergency' session while testing. Better yet: use `sshd -t` to validate before restarting.

CHAPTER 13

Linux Networking

Systems & Operations · 7 topics

Linux networking is a vast topic that every infrastructure engineer must understand. From interface configuration to packet filtering, DNS resolution to socket debugging, the tools and concepts covered here are used daily at FAANG — whether you're debugging a production incident, tuning a load balancer, or designing a VPC topology.

FAANG Linux networking tcpdump DNS iptables troubleshooting

Network Interfaces & Configuration

Every Linux machine has at least one network interface. Modern Linux uses **ip** (from `iproute2`) instead of the deprecated **ifconfig** and **route**. **At FAANG, server networking is configured via `netplan`, `cloud-init`, or `networkd` — never by manually editing `/etc/network/interfaces` on ephemeral instances.**

Understanding interfaces, addresses, and routing is the foundation.

```
# === Modern: ip commands (iproute2) ===
ip addr                                # All interfaces + IPs
(equivalent to ifconfig)
ip addr show eth0                      # Single interface
ip -br addr                           # Brief output (compact)
ip link                               # Interface status (up/down,
```

```

MAC, MTU)
ip route                                # Routing table
ip route show default                  # Default gateway
ip neigh                               # ARP table (neighbor cache)

# === Interface management ===
sudo ip link set eth0 up               # Bring interface up
sudo ip link set eth0 down             # Bring interface down
sudo ip addr add 10.0.0.5/24 dev eth0  # Add IP address
sudo ip addr del 10.0.0.5/24 dev eth0  # Remove IP address
sudo ip route add default via 10.0.0.1 # Set default gateway
sudo ip route add 192.168.0.0/16 via 10.0.0.1 # Add static route

# === Legacy ===
ifconfig                               # Interface info (deprecated,
still present)
route -n                               # Routing table (deprecated)
netstat -rn                            # Routing table (deprecated)
arp -a                                 # ARP table (deprecated)

# === Persistent config (Ubuntu: netplan) ===
# /etc/netplan/00-installer-config.yaml
network:
  version: 2
  ethernets:
    eth0:
      addresses:
        - 10.0.0.5/24
      routes:
        - to: default
          via: 10.0.0.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]

# Apply:
sudo netplan apply                     # Apply netplan config

# === MTU (Maximum Transmission Unit) ===
# Standard: 1500 bytes. Jumbo frames: 9000 bytes.
# Check: ip link | grep mtu

```

```
# Set: sudo ip link set eth0 mtu 9000
# At FAANG, internal networks often use jumbo frames for storage
traffic.
```

Socket Diagnostics: ss, netstat & lsof

Understanding what ports are listening and what connections are active is essential for debugging. **ss (socket statistics) is the modern replacement for netstat — it's faster and shows more information.** At FAANG, these commands are used constantly: 'Is my service listening?', 'Who's connected?', 'Is there a port conflict?'

```
# === ss (Socket Statistics) ===
ss -tlnp                # TCP listening sockets (with
process names)
ss -ulnp                # UDP listening
ss -tanp                # All TCP connections
(including established)
ss -tup                # All TCP + UDP with processes
ss -s                  # Socket summary (counts by
state)
ss -tan state established # Only established connections
ss -4                  # Only IPv4
ss -6                  # Only IPv6

# === Key fields ===
# Recv-Q, Send-Q - Queue sizes (non-zero indicates backlog)
# Local Address:Port - What the socket is bound to
# Peer Address:Port - Who's connected

# === netstat (legacy, often requires apt install net-tools) ===
netstat -tlnp          # TCP listening
netstat -tanp          # All TCP
netstat -i             # Interface statistics
netstat -s             # Protocol statistics
```

```
# === lsof (list open files) ===
sudo lsof -i :80          # What's listening on port
80?
sudo lsof -i :443
sudo lsof -p <PID>        # Open files/sockets for a
process
lsof -i -P -n | grep LISTEN  # All listening processes

# === Socket states ===
# LISTEN      - Waiting for connection
# ESTABLISHED - Active connection
# TIME_WAIT   - Connection closing (waiting for remaining packets)
# CLOSE_WAIT  - Remote closed; local hasn't closed yet
# SYN_SENT    - Trying to connect
# FIN_WAIT1/2 - Closing sequence in progress

# === Common debugging ===
ss -tanp | grep -c ESTABLISHED  # Count active connections
ss -tanp | grep -c TIME_WAIT    # Count TIME_WAIT (high is
normal for proxies)
ss -tanp | head -20             # Check for weird connections
```

DNS Resolution on Linux

DNS is how hostnames become IP addresses. Linux uses

`/etc/resolv.conf` for nameserver configuration, `/etc/hosts` for local overrides, and `getent hosts` for queries. **At FAANG, internal DNS is critical — services are discovered by name, and DNS failures cause cascading outages.**

```
# === Name resolution order (nsswitch.conf) ===
grep hosts /etc/nsswitch.conf
# hosts: files dns
# This means: check /etc/hosts first, then DNS

# === /etc/resolv.conf ===
cat /etc/resolv.conf
```

```

# nameserver 127.0.0.53      # systemd-resolved stub
# nameserver 8.8.8.8
# search internal.example.com # Auto-append domain for short names
# options rotate            # Round-robin between nameservers
# options timeout:2         # 2-second timeout per query
# options attempts:3        # Retry 3 times before failing

# === DNS queries ===
getent hosts google.com      # System resolver (uses
nsswitch)
host google.com              # Simple DNS lookup
nslookup google.com          # Legacy query tool
dig google.com               # Detailed DNS query (best for
debugging)

# === dig examples ===
dig google.com               # A record (IPv4)
dig google.com AAAA          # IPv6 record
dig google.com MX            # Mail exchange record
dig google.com NS            # Nameserver records
dig google.com CNAME         # Canonical name (alias)
dig -x 8.8.8.8               # Reverse lookup (PTR)
dig @8.8.8.8 google.com      # Query specific nameserver

# === /etc/hosts ===
cat /etc/hosts
# 127.0.0.1 localhost
# ::1      localhost
# 127.0.0.1 dev-server.internal # Local override for development

# === systemd-resolved ===
resolvectl status            # DNS config via systemd-
resolved
resolvectl query google.com  # Query via resolved

# === Check which service is using DNS ===
sudo ss -tlnp | grep 53      # DNS listener (systemd-
resolved or dnsmasq)

# === DNS troubleshooting ===

```

```
dig +trace google.com      # Trace full resolution path
dig google.com +short      # Just the IP
whois 8.8.8.8              # IP ownership info
```

Packet Filtering: iptables & nftables

Linux has built-in packet filtering via Netfilter. iptables is the legacy interface; **nftables** is the modern replacement. At FAANG, cloud security groups handle most firewall rules at the hypervisor level, but host-level iptables/nftables rules are still used for node-local policies, traffic shaping, and Kubernetes network policies.

```
# === iptables (legacy, still widely used) ===

# List rules by chain
sudo iptables -L          # Filter table (INPUT,
FORWARD, OUTPUT)
sudo iptables -t nat -L   # NAT table (PREROUTING,
POSTROUTING)
sudo iptables -L -n -v    # With packet/byte counters

# === Common patterns ===

# Allow SSH
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

# Allow established connections
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j
ACCEPT

# Drop all other inbound
sudo iptables -P INPUT DROP
sudo iptables -P FORWARD DROP
sudo iptables -P OUTPUT ACCEPT

# Port forwarding (NAT)
sudo iptables -t nat -A PREROUTING -p tcp --dport 80 -j REDIRECT --
to-port 8080
```

```

# Rate limiting (anti-DoS)
sudo iptables -A INPUT -p tcp --dport 80 -m limit --limit
100/minute -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 80 -j DROP

# === nftables (modern) ===
# /etc/nftables.conf
#!/usr/sbin/nft -f

table inet filter {
    chain input {
        type filter hook input priority 0;
        policy drop;

        ct state established,related accept
        iif lo accept
        tcp dport 22 accept
        tcp dport {80,443} accept
        ip protocol icmp accept
    }
    chain forward {
        type filter hook forward priority 0;
        policy drop;
    }
    chain output {
        type filter hook output priority 0;
        policy accept;
    }
}

# nft commands
sudo nft list ruleset          # Show all rules
sudo nft add rule inet filter input tcp dport 8080 accept
sudo nft flush ruleset        # Clear all rules

# === Save/restore ===
sudo iptables-save > /etc/iptables/rules.v4 # Save iptables rules
sudo iptables-restore < /etc/iptables/rules.v4
sudo nft list ruleset > /etc/nftables.conf   # Save nftables
rules

```

Packet Capture: tcpdump & wireshark

When you need to see exactly what's on the wire, packet capture is the ultimate debugging tool. **tcpdump is the CLI standard — it's installed on virtually every Linux system.** At FAANG, tcpdump is used for: verifying TLS handshakes, debugging connection resets, analyzing protocol behavior, and security incident investigation.

```
# === Basic tcpdump ===
sudo tcpdump -i eth0          # Capture all traffic on
interface                     # interface
sudo tcpdump -i any           # Capture on all interfaces
sudo tcpdump -n               # Don't resolve hostnames
(faster, safer)
sudo tcpdump -nn              # Don't resolve ports either

# === Expresssions (filters) ===
sudo tcpdump -n host 10.0.0.1  # Traffic to/from a specific
IP
sudo tcpdump -n src 10.0.0.1   # Source IP
sudo tcpdump -n dst 10.0.0.1   # Destination IP
sudo tcpdump -n port 80        # Specific port
sudo tcpdump -n port 80 or port 443 # Multiple ports
sudo tcpdump -n tcp            # Only TCP
sudo tcpdump -n udp            # Only UDP
sudo tcpdump -n icmp           # Only ICMP (ping)

# === Output control ===
sudo tcpdump -c 100            # Stop after 100 packets
sudo tcpdump -s 0              # Capture full packet
(default: 68 bytes snaplen)
sudo tcpdump -w capture.pcap   # Save to file (for Wireshark
analysis)
sudo tcpdump -r capture.pcap   # Read from file

# === Advanced ===
sudo tcpdump -n 'tcp[tcpflags] & tcp-syn != 0 and tcp[tcpflags] &
```

```

tcp-ack == 0' # SYN
packets only
sudo tcpdump -n 'tcp port 443 and (tcp[13] & 0x02 != 0)' # HTTPS
SYN packets

# === One-liner for debugging ===
sudo tcpdump -ni any port 80 -c 100 | grep -i "HTTP/" # Show HTTP
requests

# === tcpdump + Wireshark ===
# Capture on server, analyze on laptop:
# Server: sudo tcpdump -w capture.pcap -i eth0
# Download: scp user@server:capture.pcap .
# Analyze: open in Wireshark

# === Other network debug tools ===
ping -c 5 google.com # Basic connectivity test
mtr google.com # traceroute + ping in real
time (my traceroute)
traceroute google.com # Show route path
tracepath google.com # Similar, no root needed
curl -v http://example.com # HTTP debug (verbose
headers)
wget --debug http://example.com # HTTP debug

# === Bandwidth testing ===
iperf3 -c server # Network throughput test
iperf3 -s # Server mode
nc -vz host 22 # TCP port connectivity test
(netcat)

```

FAANG Interview Q&A

Q: 'A user reports that 'network is slow.' How do you diagnose?'

A: 'Start broad, then narrow. Check `ping` for latency and packet loss. Check `mtr` for the path — is the bottleneck at a specific hop? Check `ss -s` for socket stats — are there many retransmits

or TIME_WAIT connections? Check `ip -s link` for interface errors (dropped packets, overruns). Check `sar -n DEV 1` for bandwidth utilization. If it's an application issue, use `tcpdump` to analyze the specific traffic. At FAANG, 'slow network' is almost never the network — it's usually application-level problems (slow queries, insufficient connections, serialization bottlenecks) or DNS issues.'

Q: 'What happens when you type google.com in a browser on Linux?'

A: '(1) Browser checks its cache, then OS DNS cache (systemd-resolved/nscd). (2) /etc/nsswitch.conf says 'files dns' — check /etc/hosts first (no match). (3) Query sent to nameserver in /etc/resolv.conf. (4) If nameserver is 127.0.0.53 (systemd-resolved stub), resolved handles caching and forwarding. (5) DNS resolution returns an IP (e.g., 142.250.80.4). (6) SYN → SYN-ACK → ACK (three-way handshake via `connect()` syscall). (7) TLS handshake. (8) HTTP request. This is a classic interview question — the depth of your answer shows your understanding of the stack.'

Q: 'What's the difference between iptables and security groups?'

A: 'Security groups are a cloud hypervisor-level firewall — traffic is filtered before it reaches the instance. iptables runs on the instance itself. Security groups are stateless (for Network ACLs) or stateful (for Security Groups in AWS), while iptables is stateful by default with conntrack. At FAANG, we use security groups for broad traffic policies (e.g., 'only allow HTTP from load balancers')

and iptables for fine-grained host-level policies (e.g., 'only allow this specific user to connect to this port'). They complement each other — defense in depth.'

References & Further Reading

- **ip(8) man page** — Complete iproute2 reference
- **Arch Linux Network Configuration** — Comprehensive networking guide
- **tcpdump man page** — Official tcpdump documentation

FAANG Architect's Note: Networking is the most common 'unknown unknown' for engineers moving to infrastructure roles. You can be a brilliant programmer, but if you don't understand how packets flow, what DNS does, or why TIME_WAIT matters, you'll struggle in production. My advice: learn tcpdump before you need it. Set up two VMs and watch traffic between them. Break things. Fix them. The muscle memory you build now will save you when you're on call at 2 AM during a major incident.

Production Tip: Always run `ss -t lnp` (not netstat) for socket debugging. It's faster and more reliable. And remember the 'check your own box first' principle: before blaming the network, check if your service is actually listening. I've lost count of how many 'network issues' were actually 'I forgot to start nginx.'

CHAPTER 14

Storage, Filesystems & Disk Management

Systems & Operations · 7 topics

Storage on Linux spans a hierarchy: physical disks → partitions → LVM (logical volumes) → filesystems → mount points. At FAANG, storage is abstracted through multiple layers: local NVMe SSD, EBS volumes, network filesystems (NFS/EFS), and distributed filesystems (Ceph, JuiceFS). Understanding the Linux storage stack is essential for capacity planning, performance tuning, and data integrity.

FAANG Linux storage filesystem LVM disk management monitoring

Disk Discovery & Partitioning

The first step in storage management is discovering what disks are available. Modern Linux uses `lsblk` and `fdisk / parted` for partitioning. **At FAANG, disks are typically provisioned via cloud APIs (EBS volumes, persistent disks) and formatted automatically by init scripts — manual partitioning is rare but understanding it is expected for system-level roles.**

```
# === Disk discovery ===
lsblk                                # Tree view of all block
devices                             # Include filesystem info
lsblk -f
```

```

(FSTYPE, UUID, LABEL)
lsblk -o NAME,SIZE,TYPE,MOUNTPOINT,FSTYPE,UUID # Custom columns

sudo fdisk -l # Detailed partition table
sudo parted -l # Alternative partition tool

# Device naming:
# /dev/sda, /dev/sdb, ... - SCSI/SATA disks (also virtio-blk, EBS)
# /dev/nvme0n1, /dev/nvme1n1, ... - NVMe SSDs
# /dev/vda, /dev/vdb, ... - VirtIO disks (KVM)
# /dev/mapper/* - LVM logical volumes

# === Partitioning with parted (GPT) ===
sudo parted /dev/sdb mklabel gpt # Create GPT
label
sudo parted /dev/sdb mkpart primary ext4 0% 100% # Use entire
disk
sudo parted /dev/sdb mkpart primary ext4 1MiB 10GiB # Specific
size
sudo parted /dev/sdb print # Show
partition table

# === Partitioning with fdisk (MBR) ===
sudo fdisk /dev/sdb # Interactive
# n → new partition, p → primary, w → write

# === Partition tables: MBR vs GPT ===
# MBR (DOS): max 2TB, 4 primary partitions
# GPT: up to 8ZiB, 128 partitions, redundant header
# Most modern systems use GPT

# === Wipe a disk ===
sudo wipefs -a /dev/sdb # Wipe all filesystem
signatures
sudo dd if=/dev/zero of=/dev/sdb bs=1M count=100 # Overwrite first
100MB

# === Kernel re-scan ===
sudo partprobe # Inform kernel of partition
table changes
echo 1 | sudo tee /sys/block/sdb/device/rescan

```

Filesystems: ext4, XFS & Modern Options

The filesystem is the layer that organizes data on disk. **ext4** is the default on Ubuntu/Debian — reliable and well-tested. **XFS** is the default on RHEL/Amazon Linux — excellent for large files and parallel I/O. At FAANG, the choice of filesystem depends on workload: XFS for large distributed databases, ext4 for general-purpose, and specialised FS for specific use cases (ZFS for data integrity, Btrfs for snapshots).

```
# === Creating filesystems ===
sudo mkfs.ext4 /dev/sdb1          # Format as ext4
sudo mkfs.ext4 -L mydata /dev/sdb1 # With label
sudo mkfs.xfs /dev/sdb1           # Format as XFS
sudo mkfs.xfs -L mydata /dev/sdb1 # XFS with label
sudo mkfs.btrfs /dev/sdb1         # Btrfs

# === ext4 tuning ===
sudo mkfs.ext4 -m 0 /dev/sdb1      # 0% reserved for root (was
5%, good for data
volumes)
sudo mkfs.ext4 -E lazy_itable_init=0 /dev/sdb1 # Initialize inode
table (faster first
mount? Actually slower now, faster later)
# tune2fs: tune an existing ext2/3/4 filesystem
sudo tune2fs -l /dev/sdb1          # List filesystem features
sudo tune2fs -m 1 /dev/sdb1        # Change reserved block
percentage

# === XFS tuning ===
# XFS is optimized for large files and parallel I/O
# Key mkfs.xfs options:
# -d agcount=4      - Allocation group count (more = better
parallelism)
# -f                - Force (overwrite existing FS)
# -m crc=1          - Enable metadata CRC (default since RHEL 7)
```

```
# === Filesystem features comparison ===
# Feature          ext4          XFS          Btrfs
# Max file size     16TB          8EB          16EB
# Max volume size   1EB           8EB          16EB
# Default fsck       e2fsck         xfs_repair    btrfs check
# Snapshots         no             no             yes
# Compression       no             no             yes
# Online defrag      no             yes (xfs_fsr) yes

# === Check filesystem ===
sudo fsck /dev/sdb1          # Check and repair (must be
                             unmounted!)
sudo fsck.ext4 -f /dev/sdb1  # Force check even if clean
sudo xfs_repair /dev/sdb1    # XFS repair (unmounted)
sudo xfs_check /dev/sdb1     # XFS check (unmounted)
```

Mounting & /etc/fstab

Mounting attaches a filesystem to a directory. The `/etc/fstab` file defines persistent mounts that are automatically mounted at boot. **A misconfigured fstab can prevent a server from booting — always test mount options and keep a backup.**

```
# === Mount basics ===
sudo mount /dev/sdb1 /mnt/data  # Mount device to directory
sudo umount /mnt/data          # Unmount
sudo mount -a                   # Mount everything in
/etc/fstab

# === /etc/fstab format ===
# <device>    <mountpoint>    <fstype>    <options>    <dump>
<pass>

# Examples:
UUID=abc123-... /                ext4    defaults,errors=remount-
ro 0          1
UUID=def456-... /boot            ext4    defaults
```

```

0      2
UUID=ghi789-... /mnt/data      ext4      defaults,noatime
0      2

# /dev/sdb1      /mnt/data      ext4      defaults,nofail
0      2
# ^ nofail means: if this fails to mount, still boot

# === Common mount options ===
# defaults      - rw, suid, dev, exec, auto, nouser, async
# noatime        - Don't update access time (big performance gain)
# nodirtime      - Don't update directory access times
# noexec         - Don't allow executables on this filesystem
# nosuid         - Ignore SUID bits
# ro             - Read-only
# nofail         - Don't fail boot if mount fails (important for
removable/optional
volumes)
# discard        - Enable TRIM for SSDs (or better: periodic
fstrim)

# === Bind mounts ===
# Mount a directory to another location
mount --bind /var/log /mnt/log-backup
# /etc/fstab:
# /var/log /mnt/log-backup none bind 0 0

# === tmpfs (RAM-backed filesystem) ===
mount -t tmpfs -o size=1G tmpfs /mnt/ramdisk
# /etc/fstab:
# tmpfs /mnt/ramdisk tmpfs size=1G 0 0

# === Find by UUID ===
blkid                                # List all UUIDs and LABELs
sudo mount UUID="abc123" /mnt/data  # Mount by UUID (preferred,
immune to device name
changes)

# === Check mounts ===
mount                                # List all mounted filesystems

```

```
df -hT                                # Disk free, human-readable +  
type  
findmnt                                # Tree view of mounts
```

LVM: Logical Volume Manager

LVM provides a flexible abstraction layer between physical disks and filesystems. **With LVM, you can resize volumes without repartitioning, take snapshots for backups, and span volumes across multiple disks.** At FAANG, LVM is used on on-premise servers but cloud instances typically use cloud block storage directly (EBS volumes) — though understanding LVM is still expected for infrastructure roles.

```
# === LVM Architecture ===  
# Physical Volumes (PVs) → Volume Group (VG) → Logical Volumes (LVs)  
  
# === Create LVM ===  
# 1. Mark disk as Physical Volume  
sudo pvcreate /dev/sdb /dev/sdc  
  
# 2. Create Volume Group  
sudo vgcreate vg_data /dev/sdb /dev/sdc  
  
# 3. Create Logical Volume  
sudo lvcreate -L 100G -n lv_data vg_data  
sudo lvcreate -l 100%FREE -n lv_data vg_data    # Use all free space  
  
# 4. Format and mount  
sudo mkfs.ext4 /dev/vg_data/lv_data  
sudo mkdir /mnt/data  
sudo mount /dev/vg_data/lv_data /mnt/data  
  
# === LVM commands ===
```

```

pvs                                # List physical volumes
pvs -o+pv_used                    # With usage info
vgs                                # List volume groups
vgs -o+vg_free                    # With free space
lvs                                # List logical volumes
lvdisplay                         # Detailed LV info

# === Resize LVM ===
# Extend PV (if underlying disk grew)
sudo pvresize /dev/sdb

# Extend VG (add new disk)
sudo vgextend vg_data /dev/sdd

# Extend LV (then resize filesystem)
sudo lvextend -L +50G /dev/vg_data/lv_data
sudo lvextend -l +100%FREE /dev/vg_data/lv_data

# Resize filesystem (ext4)
sudo resize2fs /dev/vg_data/lv_data

# Resize filesystem (xfs)
sudo xfs_growfs /mnt/data          # XFS grows online (must be
mounted)

# === LVM Snapshots ===
# Create a snapshot (for backup consistency)
sudo lvcreate -L 10G -s -n lv_data_snap /dev/vg_data/lv_data
# Mount snapshot read-only:
sudo mount -o ro /dev/vg_data/lv_data_snap /mnt/backup
# Remove snapshot:
sudo lvremove /dev/vg_data/lv_data_snap

# === Remove LVM ===
sudo umount /mnt/data
sudo lvremove /dev/vg_data/lv_data
sudo vgremove vg_data
sudo pvremove /dev/sdb /dev/sdc

```

Monitoring: I/O, Space & Inodes

Storage monitoring is critical for production reliability. **Out of disk space is the #1 cause of predictable production incidents — it's almost always a log file or database that wasn't rotated.**

At FAANG, disk monitoring is built into the observability stack, but you still need to know the command-line tools for ad-hoc debugging.

```
# === Disk space ===
df -h                                # Human-readable disk usage
df -hT                              # With filesystem type
df -i                                # Inode usage (filesystem can
be full even with free
space!)

# === Directory size ===
du -sh /var/log                     # Total size of directory
du -sh /*                           # Size of top-level
directories
du -sh /var/log/* | sort -rh | head -10 # Largest items in
/var/log

# === I/O performance ===
iostat -x 1                          # I/O stats by device (1s
intervals)
# Key fields:
# %util      - % of time device was busy (warning at >80%)
# await      - Average I/O latency (ms)
# r/s, w/s    - Read/write IOPS
# kB/s, kB/s  - Throughput

iostat -x 1 5                        # 5 samples, 1 second apart

# === I/O top (per-process) ===
iotop -oP                            # Show only processes doing
I/O
```

```
# === Find large files ===
find /var/log -type f -size +100M    # Files > 100MB
find / -xdev -type f -size +1G -exec ls -lh {} \; # Files > 1GB on
root FS

# === Find and remove old logs ===
find /var/log -name "*.log" -mtime +30 -delete # Delete logs > 30
days

# === TRIM for SSDs ===
sudo fstrim -v /                        # Manually TRIM a filesystem
sudo fstrim -av                        # TRIM all mounted FS that
support it
# Systemd timer: systemctl status fstrim.timer

# === SMART disk health ===
sudo apt install smartmontools
sudo smartctl -H /dev/sda              # Overall health status
sudo smartctl -a /dev/sda              # Full SMART data
sudo smartctl -t short /dev/sda        # Run short self-test

# === dd benchmark ===
dd if=/dev/zero of=/tmp/test bs=1M count=1000 oflag=direct # Write
test
dd if=/tmp/test of=/dev/null bs=1M count=1000 iflag=direct  # Read
test
# Replace with fio for production-grade benchmarks

# === fio (Flexible I/O Tester) ===
sudo apt install fio
fio --randrepeat=1 --ioengine=libaio --direct=1 --gtod_reduce=1 \
--name=test --bs=4k --iodepth=64 --size=1G --readwrite=randread
\
--filename=/tmp/fio-test
```

FAANG Interview Q&A

Q: 'The database server has no space, but df shows 50% free. What's wrong?'

A: 'Check `df -i` — the filesystem may be out of inodes, not space. This happens when millions of tiny files (like Docker overlay layers, mail spool files, or a runaway process writing temp files) exhaust the inode table. Inode count is set at filesystem creation and can't be changed without reformatting. The fix: find and remove the unnecessary small files, or create a new filesystem with more inodes using `mkfs.ext4 -N <count> .`'

Q: 'How do you choose between ext4 and XFS for a new filesystem?'

A: 'It depends on the workload. For general-purpose, ext4 is fine. For large files (>2TB), high concurrency of parallel I/O, or workloads with many large files (media, logs, databases), XFS performs better — especially because it supports online defragmentation (`xfsr`) and has better allocation groups for parallelism. For metadata-heavy workloads (many small files, like a mail server or git repository), ext4 is typically better. At FAANG, the standard is often XFS on RHEL/Amazon Linux for production and ext4 on Ubuntu for development.'

Q: 'You receive an alert: EBS volume is at 95% capacity. Walk through your response.'

A: 'First, acknowledge the alert. SSH into the instance (or use session manager). Run `df -h` to confirm which volume. Run `du -`

`sh /* | sort -rh | head -10` to find the largest directories. Identify the culprit (logs, database WAL files, temp files). If it's logs: rotate/compress with logrotate or delete old logs. If it's predictable growth, extend the EBS volume (increase size in cloud console), then `sudo growpart /dev/xvda 1` (extend partition) and `sudo resize2fs /dev/xvda1` (extend filesystem). File a ticket to increase monitoring thresholds or add auto-scaling. Document the incident.'

References & Further Reading

- **Arch Linux Filesystems Wiki** — Comprehensive filesystem comparison
- **LVM Administration (RHEL 9)** — Red Hat's official LVM guide
- **mkfs.ext4(8) man page** — All ext4 formatting options

FAANG Architect's Note: Storage management has changed dramatically with cloud computing. At FAANG, you rarely touch physical drives — you provision EBS volumes, create snapshots, and manage lifecycle policies through APIs. But the Linux storage concepts (partitioning, filesystems, mount points, LVM) directly translate to the cloud: EBS volumes are like virtual disks, EBS snapshots are like LVM snapshots, and filesystem tuning is even more important when your storage is network-attached with variable latency. Don't skip these fundamentals because 'everything is in the cloud now' — the cloud abstracts hardware, not concepts.

Production Tip: Set up monitoring on disk inode usage, not just space usage. I've seen production outages caused by 'disk space OK but inodes full' — Docker overlay filesystems and log rotation failures are common culprits. Add a `df -i` check at 80% inode usage to your monitoring thresholds. Also, always use `noatime` mount option for performance — updating access time on every read is rarely needed and adds unnecessary I/O.

PART 3

Advanced Topics

Deep dives into the Linux kernel, memory management, I/O stack, performance tuning, eBPF, observability, GPU computing, and build toolchains.

CHAPTER 15

Kernel Architecture & Syscalls

Advanced · 6 topics

The Linux kernel is the core of the operating system — it manages processes, memory, hardware, and provides the syscall interface that user-space programs use to interact with the system. At FAANG, kernel knowledge is essential for debugging production incidents, building high-performance systems, and understanding container isolation.

FAANG

Linux

kernel

syscalls

scheduler

kernel modules

Kernel Architecture: User Space vs Kernel Space

The kernel runs in a privileged mode (ring 0 on x86). User-space programs run in an unprivileged mode (ring 3). **System calls (syscalls) are the only way user-space can request kernel services.** The `/proc` and `/sys` filesystems expose kernel data structures to user space. At FAANG, engineers working on infrastructure must understand this boundary — performance issues, security vulnerabilities, and debugging often hinge on crossing the kernel-user boundary.

```
# === Check kernel version ===
uname -a                # Full kernel version and
system info
uname -r                # Just kernel release (e.g.,
6.8.0-1015-aws)
cat /proc/version       # Kernel version with gcc info
```

```

# === List loaded kernel modules ===
lsmod                                # All loaded modules
modinfo <module>                     # Module details (author,
license, params)

# === Load/unload modules ===
sudo modprobe <module>               # Load module with dependency
resolution
sudo insmod /path/to/module.ko      # Load module directly (no
deps)
sudo rmmod <module>                  # Unload module
sudo modprobe -r <module>           # Unload with dependency
cleanup

# === Kernel parameters (sysctl) ===
sysctl -a                           # List all kernel parameters
sysctl net.ipv4.tcp_tw_reuse         # Single parameter
sudo sysctl -w net.ipv4.ip_forward=1 # Set parameter (runtime,
non-persistent)
cat /etc/sysctl.conf                 # Persistent kernel parameters
sudo sysctl -p                       # Reload config from file

# === Key kernel subsystems visible via /proc ===
cat /proc/cpuinfo                   # CPU details (model, cache,
flags)
cat /proc/meminfo                   # Memory details (Total, Free,
Buffers, Cached)
cat /proc/zoneinfo                  # Memory zone info (NUMA)
cat /proc/interrupts                # IRQ distribution across CPUs
cat /proc/softirqs                  # Softirq stats
cat /proc/sched_debug               # Scheduler debug info

# === cgroups v2 (kernel resource control) ===
ls /sys/fs/cgroup/                  # Cgroup controllers
cat /sys/fs/cgroup/cpu.stat          # CPU accounting
cat /sys/fs/cgroup/memory.current   # Current memory usage

# === /sys - kernel objects ===
ls /sys/class/block/                # Block devices
ls /sys/class/net/                  # Network interfaces
cat /sys/class/net/eth0/device/vendor # NIC vendor ID

```

```
cat /sys/block/sda/queue/scheduler # I/O scheduler (mq-deadline,
none, kyber)

# === Kernel ring buffer ===
dmesg # Kernel log messages
dmesg -w # Follow kernel log (like tail
-f)
dmesg | grep -i error # Kernel errors
dmesg -T # Human-readable timestamps
```

System Calls Deep Dive

Syscalls are the API between user-space and the kernel. **Common syscalls: read, write, open, close, mmap, fork, execve, ioctl, epoll.** Understanding syscalls is critical for debugging performance issues and understanding what your program is actually doing. At FAANG, strace is one of the most valuable debugging tools for understanding 'why is this process slow?'

```
# === strace - trace system calls ===
strace ls # Trace all syscalls for 'ls'
strace -e openat,read ls # Filter specific syscalls
strace -p 1234 # Attach to running process
strace -c ls # Count syscalls (summary,
great for profiling)
strace -f -c nginx # Include child processes
strace -tt -T ls # Show timestamps + syscall
duration

# === Key syscalls to know ===
# Process:
# fork() - Create child process
# execve() - Replace process image
# exit() - Terminate process
# wait() - Wait for child to exit
ey
# clone() - Create thread (or process with CLONE_VM)
```

```

#
# File I/O:
#  open() / openat() - Open a file
#  read()             - Read from fd
#  write()            - Write to fd
#  close()            - Close fd
#  lseek()            - Reposition file offset
#  mmap()             - Memory-map a file
#
# Networking:
#  socket()           - Create endpoint
#  bind()             - Bind to address
#  listen()           - Listen for connections
#  accept()           - Accept connection
#  connect()          - Connect to remote
#  send()/recv()      - Send/receive data
#  epoll_create() / epoll_wait() - Event-driven I/O

# === Syscall count per process ===
strace -c -p $(pgrep -f nginx | head -1) 2>&1 | head -20

# === Check syscall availability (seccomp) ===
# List syscalls allowed for a process
cat /proc/$(pgrep -f nginx | head -1)/status | grep -i seccomp
# 0 = disabled, 1 = strict, 2 = filter (seccomp-bpf)

# === /proc/sys/kernel/ - tune syscall behavior ===
cat /proc/sys/kernel/pid_max          # Max PID (default 32768,
increase for large servers)
cat /proc/sys/kernel/threads-max       # Max threads system-wide
cat /proc/sys/fs/file-max              # Max open files system-wide

# === Syscall table ===
# Architecture-specific (x86_64 vs arm64 vs aarch64)
# List: /usr/include/x86_64-linux-gnu/asm/unistd_64.h
# Online:
https://github.com/torvalds/linux/blob/master/arch/x86/entry/syscalls/syscall\_64.tbl

```

The Kernel Scheduler & Context Switching

The Completely Fair Scheduler (CFS) allocates CPU time to threads. Each thread gets a proportion of CPU based on its **nice value** and **cgroup cpu.shares**. The scheduler runs every **context switch** — when one thread is de-scheduled and another is scheduled. **Context switches are not free** — each one has overhead (TLB flush, cache miss). At FAANG, high context-switch rates are a common performance issue in over-threaded applications.

```
# === Scheduler statistics ===
cat /proc/schedstat          # Raw scheduler stats
cat /proc/sched_debug        # Detailed scheduler info

# === Context switches per process ===
cat /proc/<PID>/status | grep -i "ctxt_switches"
# voluntary_ctxt_switches – process gave up CPU (I/O wait, sleep)
# nonvoluntary_ctxt_switches – kernel preempted process (time slice expired)

# === System-wide context switches ===
vmstat 1 5                   # cs column = context
                              switches/sec
# High context switches (>100k/sec per core) often indicate
threading issues

# === Check scheduler policy ===
chrt -p <PID>                # Show scheduling policy +
priority
# SCHED_OTHER (0) – Default fair scheduling
# SCHED_FIFO (1) – Real-time, first-in-first-out
# SCHED_RR (2)  – Real-time, round-robin
# SCHED_BATCH (3) – Batch scheduling (non-interactive)
# SCHED_IDLE (5) – Lowest priority (idle-like)

# === CPU affinity ===
```

```

taskset -p <PID>                # Show CPU affinity mask
taskset -cp <PID>                # Show CPU affinity (list)
sudo taskset -cp 0-3 <PID>       # Pin to CPUs 0-3

# === NUMA (Non-Uniform Memory Access) ===
numactl --hardware               # NUMA topology
numastat -p <PID>                # Memory allocation per NUMA
node
# On multi-socket servers: accessing memory on remote socket is
# slower
# FAANG pattern: pin processes + memory to same NUMA node for
# performance

# === IRQ affinity ===
# Distribute hardware interrupts across CPUs
cat /proc/irq/<IRQ>/smp_affinity  # CPU mask for this IRQ
# Set: echo 3 > /proc/irq/32/smp_affinity (send to CPUs 0+1)
# Or use irqbalance daemon for automatic distribution

# === /proc/loadavg ===
cat /proc/loadavg
# Load average = number of processes in running or uninterruptible
# state
# Over 1-minute, 5-minute, 15-minute windows
# Rule of thumb: load < #cores is normal; load > 2x#cores indicates
# overload

```

Kernel Modules & Device Drivers

Kernel modules are pieces of code that can be loaded/unloaded without rebooting. **Most hardware drivers, filesystems, and network protocols are modules.** At FAANG, custom kernel modules are rare (most use the standard distribution kernel), but understanding the module system is essential for infosec (kernel rootkits load as modules) and for debugging hardware issues.

```

# === Module management ===
lsmod                                # All loaded modules
# Format: Module, Size, Used by

# Module details
modinfo nvme                        # Show info (author,
description, params, licenses)
modinfo -p nvme                     # Just parameters

# Module dependency map
depmod                              # Build modules.dep
cat /lib/modules/$(uname -r)/modules.dep # Module dependencies

# === Blacklist a module ===
echo "blacklist nouveau" | sudo tee /etc/modprobe.d/blacklist-
nvidia.conf

# === Module parameters ===
cat /sys/module/nvme/parameters/    # Runtime module parameters

echo "options nvme_core io_timeout=100" | sudo tee
/etc/modprobe.d/nvme.conf

# === Check for kernel oops/crash ===
dmesg | grep -i "oops\|panic\|BUG"
ls /var/crash/                      # Crash dumps (kernel-core)

# === Kernel TLS / kTLS ===
lsmod | grep tls                    # Check if kTLS module loaded
# kTLS offloads TLS encryption to kernel (performance optimization)

# === Build a simple kernel module ===
# /usr/src/linux-headers-$(uname -r)/ must be installed
cat > hello.c << 'EOF'
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>

static int __init hello_init(void) {
    printk(KERN_INFO "Hello, FAANG!\n");
    return 0;
}

```

```

}

static void __exit hello_exit(void) {
    printk(KERN_INFO "Goodbye, FAANG!\n");
}

module_init(hello_init);
module_exit(hello_exit);
MODULE_LICENSE("GPL");
MODULE_AUTHOR("FAANG Engineer");
EOF

# Makefile:
obj-m += hello.o
all:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD)
modules
clean:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean

# Build & test
make
sudo insmod hello.ko
dmesg | tail                # See 'Hello, FAANG!'
sudo rmmod hello
dmesg | tail                # See 'Goodbye, FAANG!'

```

FAANG Interview Q&A

Q: 'What happens when you type a command like 'ls' in a shell?'

A: '(1) Shell reads the line and parses it. (2) Shell searches PATH for 'ls' binary (via `execve` syscall). (3) Shell calls `fork()` to create a child process. (4) Child calls `execve('/usr/bin/ls', ['ls'], envp)` — this replaces the child's process image with the ls binary. (5) The kernel loads the ELF binary, maps memory segments, sets up the

stack. (6) `ls` runs, calls `openat()/getdents64()` to read directory entries. (7) `ls` calls `write()` to print results. (8) `ls` calls `exit(0)`. (9) Shell's `wait()` returns with exit status. (10) Shell prints the next prompt.'

Q: 'What is a kernel panic and how do you debug it?'

A: 'A kernel panic is when the kernel encounters a fatal error it cannot recover from (e.g., null pointer dereference in kernel code, hardware error, or corrupted kernel data structure). The kernel prints a message to the console and either hangs or reboots. Debug via: (1) Check `dmesg` from the next boot — look for 'Oops' or 'panic' messages. (2) Enable `kdump/kexec` to capture a crash dump for analysis with `crash` utility. (3) Check `/var/crash/` for `vmcore` files. (4) Use 'crash' utility to analyze: `crash /usr/lib/debug/boot/vmlinux-$(uname -r) /var/crash/vmcore`. At FAANG, kernel panics are rare but critical — the response involves the Linux kernel team and hardware vendors.'

Q: 'What is the difference between a process and a thread in the kernel?'

A: 'In the Linux kernel, both processes and threads are implemented as 'tasks' (`struct task_struct`). The difference: threads share the same address space (`mm_struct`), file descriptors (`files_struct`), and signal handlers via the `clone()` syscall with `CLONE_VM | CLONE_FILES | CLONE_SIGHAND` flags. Processes have independent address spaces created via `fork()` or `clone()` without `CLONE_VM`. To the scheduler, they're all tasks —

the kernel doesn't distinguish between processes and threads. This is why 'lightweight processes' (LWPs) is an accurate description of Linux threads.'

References & Further Reading

- **Linux Kernel Documentation** — Official kernel docs (admin-guide, core-api, process)
- **Linux Kernel Development** — Robert Love, the best practical kernel guide
- **Linux Syscall Table** — Interactive x86_64 syscall reference

FAANG Architect's Note: You don't need to be a kernel developer to work at FAANG, but you need to understand what the kernel does. The most effective infrastructure engineers I've worked with have a mental model of the kernel — they know when a slow read is a disk issue vs a page cache miss, when high CPU is a scheduling problem vs a busy loop, and when a crash is a kernel bug vs a hardware failure. This understanding comes from experience and from using tools (strace, perf, /proc) to observe what's actually happening. Build that habit now: before blaming the hardware or the network, check what the kernel is doing.

Production Tip: Before deploying a new kernel version at scale, always test on a non-production instance first. Kernel upgrades can change behavior of the scheduler, memory manager, or network stack — all of which affect your application. At FAANG, kernel upgrades are rolled out in canary phases: 1% of hosts, then 10%, then 50%, then full rollout, with automated rollback on any metric regression.

CHAPTER 16

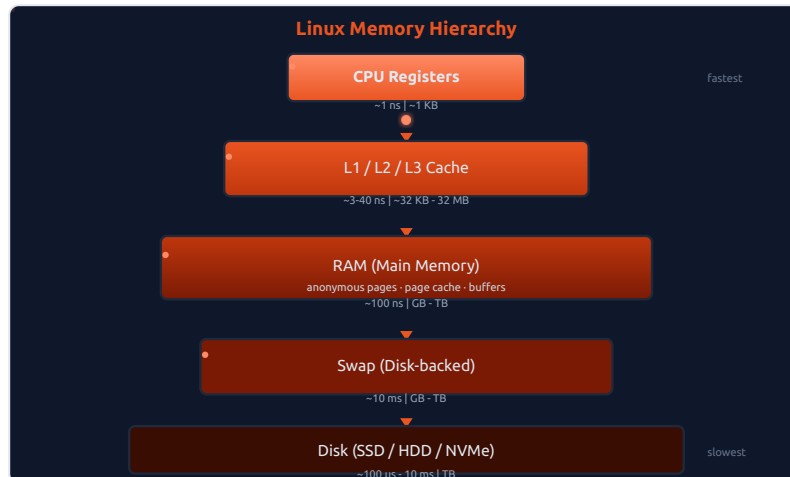
Memory Management & OOM Handling

Advanced · 7 topics

Memory management is one of the most critical subsystems in Linux. The kernel manages physical RAM, virtual address spaces, page caches, swapping, and out-of-memory (OOM) handling. At FAANG, memory-related issues (OOM kills, memory leaks, page cache pressure) are among the most common production incidents.

[FAANG](#)[Linux](#)[memory](#)[OOM](#)[page cache](#)[huge pages](#)[performance](#)

Virtual Memory & Address Spaces



Each process sees a **virtual address space** (typically 47 bits on x86_64 = 128TB). The kernel and hardware (MMU/TLB) translate virtual addresses to physical frames via **page tables**. This isolation is why one process can't read another's memory — and why `mmap` is so efficient (it maps files directly into the address space without copying).

```
# === Process memory map ===
cat /proc/<PID>/maps           # Full memory map of a process
# Format: address-range perms offset dev inode pathname

# Example interpretation:
# 55555554000-55555558000 r--p 00000000 fd:01 123456 /usr/bin/ls
# 55555558000-5555555c000 r-xp 00004000 fd:01 123456 /usr/bin/ls
# (code)
# 7ffff7d80000-7ffff7f80000 rw-p 00000000 00:00 0
# (heap)
# 7ffff7fde000-7ffff7fff000 rw-p 00000000 00:00 0
# (stack)

# === Memory regions ===
# r-xp - Code segment (read, execute)
```

```

# r--p - Read-only data
# rw-p - Read-write data (heap, global vars)
# rw-p [stack] - Thread stack
# rw-p [heap] - Heap (brk/mmap allocations)

# === /proc/<PID>/status memory fields ===
cat /proc/<PID>/status | grep -i vm
# VmPeak: peak virtual memory size
# VmSize: current virtual memory size
# VmLck: locked (mlock) memory
# VmPin: pinned memory
# VmHWM: peak resident set size
# VmRSS: current resident set size (= actual physical RAM used)
# RssAnon: anonymous memory (heap, stack, mmap without file)
# RssFile: file-backed memory (mmap of files, shared libraries)
# RssShmem: shared memory
# VmSwap: swapped memory

# === pmap - detailed process memory ===
pmap -x <PID> # Extended memory map with
RSS/DClean
pmap -XX <PID> # Very detailed (kernel flags)

# === Total memory usage ===
free -h # Total, used, free, shared,
buff/cache, available
cat /proc/meminfo # Detailed breakdown
# MemTotal: total physical RAM
# MemFree: completely free
# Buffers: block device cache
# Cached: page cache + tmpfs + shmem
# SwapTotal: total swap
# SwapFree: free swap
# Available: estimated memory available for starting new apps
# (= MemFree + reclaimable page cache + reclaimable
slab)

# === Slab allocator (kernel memory) ===
cat /proc/slabinfo # Kernel slab allocations
# Slab is the kernel's memory allocator for small objects (inodes,

```

```

dentries, etc.)
# High slab usage can indicate kernel memory leak
sudo slabtop -s c                                # Sort by cache size

```

Page Cache & Buffers

Linux uses all available free RAM for **page cache** (file data cached from disk) and **buffers** (block device metadata). **This is not 'wasted' memory — it improves performance dramatically.** When an application needs memory, the kernel reclaims page cache pages. At FAANG, a system with '99% memory used' is normal and healthy — what matters is the 'available' metric.

```

# === Page cache basics ===
# When you read a file, the kernel caches it in page cache
# Subsequent reads are served from RAM (fast) instead of disk (slow)

# === Drop caches (for benchmarking only!) ===
# NEVER do this in production - it destroys performance temporarily
sudo sync && echo 3 | sudo tee /proc/sys/vm/drop_caches
# 1 = free page cache, 2 = free dentries/inodes, 3 = free both

# === Page cache statistics ===
cat /proc/vmstat | grep -E "(pgfault|pgmajfault|pgfree)"
# pgfault    - Minor page faults (page was in memory, just needed mapping)
# pgmajfault - Major page faults (page had to be read from disk)
# High major faults = slow I/O, insufficient memory

# === File-backed memory vs anonymous ===
# file-backed: mmap'd files, executable code, shared libraries
# anonymous: heap, stack, anonymous mmap (malloc uses this)
# Key difference: file-backed can be reclaimed (write dirty pages to disk);
# anonymous must be swapped to disk (slow!) or the process must be killed

```

```
# === vmtouch - page cache analysis ===
sudo apt install vmtouch
vmtouch /var/log/syslog          # How much of a file is in
cache
vmtouch /usr/bin/*              # Page cache stats for
directory
# Files that are frequently accessed stay in cache (good)
# Cache can be 'poisoned' by a one-time large read (bad)

# === fincore - which pages are cached ===
# (part of linux-tools-common)
# fincore --pages=false --summarize /var/log/syslog

# === Tune page cache behavior ===
# /etc/sysctl.conf:
# vm.vfs_cache_pressure=100      # Default: reclaim slab at
same rate as page cache
#   Lower = keep more dentry/inode cache (good for many small
files)
#   Higher = reclaim slab more aggressively
# vm.min_free_kbytes=65536       # Reserve memory for critical
allocations
# vm.dirty_ratio=20              # Max dirty pages % before
blocking writers
# vm.dirty_background_ratio=10   # Background writeback starts
at this % dirty
# vm.swappiness=10               # Only swap when absolutely
necessary (default: 60)
```

OOM Killer: Out-Of-Memory Management

When the system runs out of memory, the kernel's **OOM killer** selects a process to kill to free memory. **Understanding OOM behavior is critical — it's often the root cause of mysterious**

service failures. At FAANG, OOM kills are monitored closely, and services are configured with cgroup memory limits to prevent a single service from starving the system.

```
# === OOM logs ===
dmesg | grep -i "oom_killer\|OOM"    # Find OOM events
# Output includes:
# - Total memory, free memory
# - Which process was killed (PID, name, score)
# - Memory footprint of top processes

# === OOM score ===
cat /proc/<PID>/oom_score              # OOM score (higher = more
likely to be killed)
# Score is based on: RSS + swap + page table + (total memory *
factor)
cat /proc/<PID>/oom_score_adj          # Adjust score (-1000 to 1000)
# Services you want to protect: echo -1000 >
/proc/<PID>/oom_score_adj
# Services you want killed first: echo 1000 >
/proc/<PID>/oom_score_adj

# === OOM in systemd units ===
[Service]
# Protect critical services:
OOMScoreAdjust=-500

# === OOM in cgroups ===
# cgroups v2 memory.max will trigger OOM kill within the cgroup,
# without affecting other cgroups. This is the PREFERRED approach
at FAANG.
echo "100M" > /sys/fs/cgroup/myapp/memory.max

# === How the OOM killer selects a victim ===
# 1. Calculates oom_score for each process
#   - Based on RSS, swap, page table size
#   - Root processes get a 3% bonus (less likely to be killed)
# 2. Selects the process with the highest score
# 3. Sends SIGKILL
# 4. If memory still insufficient, repeats
```

```
# === Prevent OOM: reserve swap ===
# Swap is slower than RAM but provides a safety buffer
# Without swap, OOM killer activates sooner
# At FAANG, swap is often disabled – memory limits are enforced via
cgroups
# But understanding swap is still essential:
swapon --show                # Check swap status
sudo swapon /swapfile        # Enable swap file

# === Memory overcommit ===
cat /proc/sys/vm/overcommit_memory
# 0 = heuristic overcommit (default – conservative)
# 1 = always overcommit (dangerous!)
# 2 = no overcommit (vm.overcommit_ratio controls %)
# At FAANG: usually 0 or 2, never 1

# === Check for OOM in systemd journal ===
journalctl -k | grep -i "oom\|killed"
journalctl -u myapp | grep -i "killed\|memory"
```

Huge Pages & Memory Performance

The default page size is 4KB. **Huge pages (2MB or 1GB)** reduce TLB misses and improve performance for memory-intensive applications (databases, JVMs, packet processing). At FAANG, databases and JVM-based services commonly use huge pages. There are two types: **Transparent Huge Pages (THP)** and **explicit Huge Pages (hugetlbfs)**.

```
# === Huge page sizes ===
cat /proc/meminfo | grep Huge
# HugePages_Total: 256          # 256 × 2MB = 512MB allocated
# HugePages_Free: 128
# Hugepagesize: 2048 kB

# === Configure explicit huge pages ===
```

```

echo 256 | sudo tee /proc/sys/vm/nr_hugepages
# Persistent: vm.nr_hugepages=256 in /etc/sysctl.conf

# === Mount hugetlbfs ===
sudo mkdir /mnt/huge
sudo mount -t hugetlbfs hugetlbfs /mnt/huge
# Now applications can mmap files in /mnt/huge with 2MB pages

# === Transparent Huge Pages (THP) ===
cat /sys/kernel/mm/transparent_hugepage/enabled
# [always] madvise never
# always – kernel tries to use 2MB pages automatically
# madvise – applications request huge pages via madvise()
# never – disable THP

# === THP and databases ===
# THP can cause latency spikes (background defragmentation)
# Many databases (MongoDB, PostgreSQL) recommend disabling THP

# To disable THP:
sudo sh -c 'echo never >
/sys/kernel/mm/transparent_hugepage/enabled'
sudo sh -c 'echo never >
/sys/kernel/mm/transparent_hugepage/defrag'

# === Per-process hugepages ===
# JVM: -XX:+UseLargePages -XX:LargePageSizeInBytes=2m
# PostgreSQL: huge_pages = on
# MongoDB: disable THP (uses its own WiredTiger cache management)

# === Memory access patterns ===
# Sequential access: page cache prefetches ahead (readahead)
# Random access: high TLB miss rate, benefits from huge pages
# NUMA-local access: always prefer allocating memory on the same
socket
numactl --membind 0 --cpunodebind 0 ./myapp # Bind memory + CPU to
node 0

```

FAANG Interview Q&A

Q: 'Free shows 99% memory used but the system is running fine. Is this a problem?'

A: 'Not necessarily. Linux uses free memory for page cache and buffers — this is not 'used' memory in the traditional sense. The key metric is 'available' (in `/proc/meminfo` or `free -h`), which estimates memory reclaimable without swapping. If 'available' is low and swap usage is growing, then you have a problem. At FAANG, we monitor 'available memory' rather than 'free memory' and set alerts when it drops below a threshold (e.g., 10% of total).'

Q: 'A process was killed by OOM. How do you investigate?'

A: 'Check `dmesg | grep -i oom` to find the OOM event — it shows the victim (PID, name), total memory, and the top memory consumers at the time. Check if the process had cgroup memory limits that were too low. Check if there was a memory leak using `ps aux --sort=-%mem | head` before and after restart. Check `/var/log/` for any evidence of the leak. If it's a Java process, analyze the heap dump. Root cause is almost always: (1) memory leak in application code, (2) cgroup limit set too low, or (3) traffic spike causing legitimate increased memory usage.'

Q: 'What is the difference between buffers and cache in `/proc/meminfo`?'

A: 'Buffers are metadata associated with block devices (e.g., filesystem metadata like superblock, inode tables). Cache is the

page cache — actual file data cached from disk. The practical difference: 'Buffers' is relatively small and stable, while 'Cached' can be gigabytes and fluctuates based on file access patterns. Both are reclaimable, which is why 'available' combines them with free memory. In modern kernels (5.x+), the distinction is less meaningful than it used to be, but the fields are kept for compatibility.'

References & Further Reading

- **Kernel VM Documentation** — All vm sysctl parameters explained
- **Understanding the Linux Kernel** — O'Reilly, chapters on memory management
- **Linux Memory Management (Mel Gorman)** — Classic deep-dive into the VM subsystem

FAANG Architect's Note: Memory management is the subsystem I've seen cause the most baffling production issues. 'My process uses 100MB, but the system has 64GB — why was it OOM-killed?' The answer is almost always: page cache. A backup process, log rotation, or a CI job read a large file and filled the page cache. When the application needed memory, the kernel tried to reclaim cache but couldn't keep up, so it invoked OOM. The solution: cgroup memory limits. At FAANG, *every* service runs in a cgroup with a memory limit that's 80-90% of its expected max. This prevents one service from destabilizing the system.

Production Tip: If you're using JVM, Go, or any garbage-collected runtime, always set both heap and cgroup memory limits. The JVM's max heap (-Xmx) should be set to ~75% of the cgroup memory limit to leave room for non-heap memory (stack, code cache, metaspace, native buffers). Go's GOMEMLIMIT should be similarly conservative. Failure to account for non-heap memory is the #1 cause of OOM kills in containerized JVM workloads.

CHAPTER 17

I/O Stack & Filesystem Performance

Advanced · 7 topics

The Linux I/O stack spans multiple layers: application → syscall → VFS → filesystem → block layer → I/O scheduler → device driver → disk hardware. At FAANG, I/O performance is critical for databases, log ingestion, and data pipelines. Understanding each layer helps you diagnose latency, tune throughput, and design storage systems that scale.

FAANG Linux I/O storage performance filesystem iostat
benchmarking

The Linux I/O Stack

When an application calls `read()` or `write()`, the request travels through: **1. VFS** (Virtual Filesystem Switch — dispatches to the right FS), **2. Filesystem** (ext4/XFS — translates file offset to block number), **3. Page Cache** (cache hit = return immediately; cache miss = read from disk), **4. Block Layer** (manages I/O requests, merging, sorting), **5. I/O Scheduler** (reorders requests for efficiency), **6. Device Driver** (NVMe, SATA, virtio), **7. Disk Hardware**. Each layer adds latency — total latency = sum of all layers.

```
# === I/O stack visibility ===  
# At each layer, you can observe with:  
# Application:      strace -e read,write,fsync
```

```

# VFS/Page Cache: /proc/meminfo (Cached), /proc/vmstat (pgfault)
# Filesystem: /proc/fs/<fstype>/, xfs_info/stat
# Block Layer: iostat -x, /sys/block/<dev>/stat
# I/O Scheduler: /sys/block/<dev>/queue/scheduler
# Device: smartctl, nvme-cli

# === I/O schedulers ===
cat /sys/block/sda/queue/scheduler # Current + available
schedulers
# [mq-deadline] – Good for SSDs with mixed workloads
# none – No scheduling (NVMe devices, each app manages its
own I/O)
# kyber – Latency-optimized for fast SSDs
# bfq – Desktop/soft real-time (fairness-focused)

# Change scheduler:
echo kyber | sudo tee /sys/block/nvme0n1/queue/scheduler

# === Block layer merging ===
# The block layer merges adjacent I/O requests for efficiency
cat /sys/block/sda/queue/nr_requests # Max I/O requests in queue
(default: 128)
cat /sys/block/sda/queue/max_sectors_kb # Max I/O size per request

# === Direct I/O vs Buffered I/O ===
# Buffered I/O – goes through page cache (default)
# dd if=bigfile of=/dev/null bs=1M count=1000 # Uses page
cache
# Direct I/O – bypasses page cache (database, O_DIRECT)
# dd if=bigfile of=/dev/null bs=1M count=1000 oflag=direct
# O_DIRECT is faster for large sequential access (no double-copy)
# O_DIRECT is worse for small random access (no cache benefit)

# === fsync – flushing to disk ===
# fsync() ensures data is on stable storage (crucial for databases)
# Each fsync has a cost – group commits reduce this
strace -e fsync mysql # Watch fsync calls
# For PostgreSQL: fsync=on (default)
# For MySQL/InnoDB: innodb_flush_log_at_trx_commit=1

# === I/O depth (queue depth) ===
# How many I/O requests can be in-flight at once

```

```
# High queue depth = more parallelism, higher latency per request
# Low queue depth = less parallelism, lower latency per request
cat /sys/block/nvme0n1/queue/nr_requests # Linux queue depth
(default: 128)
# NVMe drives can handle queue depth of 65536
# SATA SSDs usually saturate at queue depth ~32
```

Measure I/O Latency & Throughput

I/O performance has two dimensions: **latency** (how long each operation takes) and **throughput** (operations per second / bytes per second). **At FAANG, tail latency (p99, p99.9) is more important than average — a single slow I/O can hold up thousands of requests in a database.**

```
# === iostat - the essential I/O tool ===
iostat -x 1 # Extended stats, refresh
every second
# Key columns:
# %util - % of time device was busy servicing I/O (>80% =
degraded)
# r/s, w/s - Read/write IOPS
# kB/s, MB/s - Throughput
# await - Average I/O latency (service time + queue time) in ms
# svctm - Actual service time (iowait + hardware time)
# aqu-sz - Average queue length

# === ioping - measure latency ===
sudo apt install ioping
ioping -c 10 /var/lib/postgresql # Measure disk latency to
directory
ioping -RD /dev/sda # Measure random read latency

# Typical latencies:
# NVMe SSD: ~0.01-0.1 ms
# SATA SSD: ~0.1-0.5 ms
# HDD: ~5-15 ms
```

```
# Network block (EBS): ~0.5-5 ms (gp3), ~0.1-2 ms (io2)

# === fio - benchmark I/O ===
# Random read test (database-like)
fio --name=randread --ioengine=libaio --iodepth=32 --rw=randread \
    --bs=4k --direct=1 --size=1G --runtime=30 --filename=/tmp/test

# Sequential write test (log-like)
fio --name=seqwrite --ioengine=libaio --iodepth=64 --rw=write \
    --bs=1M --direct=1 --size=10G --runtime=60 --filename=/tmp/test

# === blktrace - block layer tracing ===
sudo blktrace -d /dev/nvme0n1 -o - | blkparse -i -
# Tracks every I/O request through the block layer
# Use for: identifying I/O merges, queue depth, latency breakdown

# === perf - I/O profiling ===
sudo perf stat -e block:block_rq_issue,block:block_rq_complete
sleep 10
# Count I/O requests issued vs completed

# === iowatcher - visualize I/O ===
# Creates a timeline graph of I/O activity
sudo blktrace -d /dev/sda -o trace
# Then: iowatcher -t trace/sda.blktrace.* -o io.svg
```

Filesystem Performance Tuning

Each filesystem has different performance characteristics. **ext4 is reliable and well-tested. XFS excels at large files and parallel I/O. Btrfs offers snapshots and checksums.** At FAANG, the choice depends on workload: XFS for large-scale databases on RHEL/Amazon Linux, ext4 for general-purpose Ubuntu instances.

```
# === Mount options for performance ===
# /etc/fstab performance optimizations:
# noatime - Don't update access times (biggest single perf win)
```

```

# nodiratime - Also skip directory access time updates
# nobarrier - Skip write barriers (DANGEROUS for data integrity!)
# data=ordered - ext4: write data before metadata (default, safe)
# data=writeback - ext4: no ordering (faster, crash-unsafe without
journal)

# Best practice mount:
# UUID=xxx /data ext4 defaults,noatime,errors=remount-ro 0 2

# === ext4 tuning ===
# Reserve blocks for root (default 5% - wasted on data volumes)
sudo tune2fs -m 0 /dev/sdb1          # 0% reserved on data volume

# Enable flex_bg (flexible block groups) - better large-file perf
sudo mkfs.ext4 -O flex_bg /dev/sdb1

# === XFS tuning ===
# Allocation groups - more = better parallelism for multi-threaded
I/O
sudo mkfs.xfs -d agcount=8 /dev/sdb1

# XFS log size - larger = better for write-heavy workloads
sudo mkfs.xfs -l size=256m /dev/sdb1

# === Filesystem fragmentation ===
# ext4: check fragmentation
sudo fsck.ext4 -fn /dev/sdb1          # -n = no modify, -f = force

# XFS: check and defragment
xfs_db -c frag -r /dev/sdb1           # Fragmentation report
sudo xfs_fsr /dev/sdb1                # Online defragmentation

# === Write cache control ===
# For SSD: enable write cache (performance), ensure power loss
protection
cat /sys/block/sda/queue/write_cache # 'write back' or 'write
through'
# write back: faster, risk of data loss on power failure
# write through: slower, safer

# === I/O priority (ionice) ===
# CFQ/bfq scheduler only

```

```

ionice -c 2 -n 0 -p <PID>          # Best-effort, highest priority
ionice -c 3 -p <PID>              # Idle I/O (only when no other
I/O)
# Real-time I/O class (use with caution!):
ionice -c 1 -n 0 ./heavy_io_program

```

Storage Performance at FAANG Scale

At FAANG, storage is abstracted through multiple layers: **local NVMe** (fast, ephemeral), **EBS** (persistent, network-attached), **EFS/NFS** (shared filesystem), and **distributed filesystems** (Ceph, JuiceFS, Luster). Each has different performance profiles and failure modes. Understanding the Linux I/O stack helps you make the right choice and debug issues at any layer.

```

# === Local NVMe ===
# /dev/nvme0n1, /dev/nvme1n1, ...
# Latency: ~10-100µs
# Throughput: 3-7 GB/s per device
# ✗ Ephemeral – data lost on instance stop/termination
# ✓ Best for: caches, temp data, high-throughput processing

# === EBS (Elastic Block Store) ===
# Network-attached block devices
# gp3: 3000 IOPS base, 16K IOPS burst, 125 MB/s per volume
# io2: up to 256K IOPS, 1000 MB/s, 99.999% durability
# ✓ Persistent – survives instance stop/termination
# ✓ Best for: databases, any persistent data
# ✗ Network latency: ~0.5-5ms (vs NVMe's 0.01-0.1ms)

# === EBS-optimized instances ===
# Dedicated network bandwidth for EBS traffic
# Check: cat /sys/block/xvda/device/queue_depth
# EBS Nitro: uses NVMe interface, no more xvda

# === NFS (Network File System) ===
# Shared filesystem accessed via network

```

```

mount -t nfs4 -o
vers=4.2,rsz=1048576,wsz=1048576,hard,timeo=600,retrans=2 \
nfs-server:/export /mnt/nfs

# NFS tuning:
# rsize/wsize: 1MB for modern networks (1048576)
# hard: retry indefinitely until server responds (vs soft: return
error on timeout)
# timeo: timeout in deciseconds (600 = 60s before retransmit)
# retrans: 2 retries before major timeout

# === Check I/O errors ===
dmesg | grep -i "I/O error\|buffer I/O"
cat /sys/block/sda/device/state      # 'running' = healthy
sudo smartctl -H /dev/sda           # SMART health check

# === /sys/block stats ===
cat /sys/block/sda/stat
# Field: read I/Os, read merges, read sectors, read ticks,
#        write I/Os, write merges, write sectors, write ticks,
#        in-flight I/Os, I/O ticks, time in queue

# Calculate average latency:
# latency_ms = (read_ticks + write_ticks) / (read_ios + write_ios)
/ 1000

```

FAANG Interview Q&A

Q: 'The database is running slow. iostat shows 99% util on the data disk. What do you check?'

A: '99% util means the device is saturated — I/O requests are queuing. First, check `iostat -x 1` for await and aqu-sz. If await is high but svctm is low, the bottleneck is in the queue (too many concurrent requests), not the device. If both are high, the device itself is slow. Next, check which process is generating the I/O:

`iostat -oP`. Look at the query plan in the database — a missing index causing a full table scan is the most common cause. If the workload is legitimate, either: (1) increase IOPS (if EBS), (2) add read replicas, (3) optimize the query, or (4) add caching (Redis, page cache).'

Q: 'What's the difference between I/O wait and I/O util?'

A: 'CPU iowait (%iowait in top) is the percentage of time the CPU is idle while waiting for I/O. It's a CPU-centric metric. I/O util (%util in iostat) is the percentage of time the device was busy — it's a device-centric metric. High iowait with low util means processes are waiting on a device that's not fully utilized (e.g., contention on a slow file, or a network filesystem). High util with low iowait means the device is busy but the CPUs are doing other work (parallelism). At FAANG, we monitor both: high iowait + high util = device bottleneck; high iowait + low util = contention or NFS issue.'

Q: 'A developer says 'the filesystem is slow.' How do you systematically debug this?'

A: 'Rule out layers from bottom to top. (1) Check SMART stats — is the disk failing? (2) Run `ioping` to measure raw device latency. (3) Run `fio` to benchmark device throughput. (4) Check filesystem mount options — maybe data=writeback was accidentally set? (5) Check `iostat -x` for queue depth and await. (6) Check page cache hit rate with `cachestat` (from bcc/bpfttrace). (7) Check application behavior with `strace -e`

`pread64,pwrite64,fsync -p <PID>` . In 90% of cases, the 'slow filesystem' is either an application pattern (too many fsyncs, no index, serial I/O) or a saturated device.'

References & Further Reading

- **Block Layer Stats** — Understanding `/sys/block/*/stat` fields
- **Red Hat: I/O Performance Tuning** — RHEL storage performance guide
- **Linux Storage Stack Deep Dive** — Tristan Hume's excellent visual guide

FAANG Architect's Note: I/O performance is the most common bottleneck in data-intensive applications. CPU and memory are rarely the limiting factor — it's almost always I/O. The reason: Moore's Law has increased CPU speed by orders of magnitude, but disk latency has improved much more slowly (HDD: 10ms in 1990 → 5ms today; SSD: 0.1ms; NVMe: 0.01ms; network: 0.5-5ms). The gap between CPU and storage speed is the fundamental constraint in system design. Every engineer should understand the I/O stack well enough to identify where latency is coming from.

Production Tip: Always benchmark your storage before putting it into production. Use fio to simulate your workload pattern (random 4K for databases, sequential 1MB for logs, mixed for general purpose). Know your baseline latency and throughput — then you'll know when something is wrong. Also, set up monitoring on per-disk metrics (await, queue depth, IOPS) not just aggregate system metrics. A single busy disk in a RAID/JBOD array can degrade performance for everything on that controller.

CHAPTER 18

Performance Tuning & Profiling

Advanced · 7 topics

Performance tuning is the art of identifying and eliminating bottlenecks. Linux provides powerful tools — perf, flamegraphs, ftrace, and BPF — for profiling CPU, memory, I/O, and network. At FAANG, performance engineering is a dedicated role, and every infrastructure engineer should be able to use perf to find the bottleneck in any system.

FAANG Linux perf profiling performance ftrace
flamegraphs

perf: The Universal Profiler

The `perf` tool (part of `linux-tools-common`) uses hardware performance counters and kernel tracepoints to profile system performance with minimal overhead. **perf can answer: 'Where is my program spending CPU time?', 'Why are there cache misses?', 'What is the kernel doing right now?'** At FAANG, perf is the first tool reached for CPU performance investigations.

```
# === Install perf ===
sudo apt install linux-tools-common linux-tools-$(uname -r)

# === CPU sampling ===
# Record CPU samples for a running process (30 seconds):
sudo perf record -F 99 -p <PID> --sleep 30
# -F 99: sample at 99 Hz (avoids aliasing with 100Hz scheduler)
```

```

ticks)
# -p <PID>: target process

# View the profile:
sudo perf report                # Interactive TUI report
sudo perf report --stdio        # Text output

# === Call graph (stack traces) ===
sudo perf record -F 99 -p <PID> -g -- sleep 30 # -g = call graphs
sudo perf report -g                # Show call graph

# === Live profiling ===
sudo perf top                    # Real-time hottest functions
(system-wide)
sudo perf top -p <PID>           # Real-time per process
sudo perf top -K                 # Kernel functions only
sudo perf top -U                 # User-space functions only

# === Static tracing ===
# Count specific events:
sudo perf stat -e cycles,instructions,cache-misses,cache-references
ls
# IPC (instructions per cycle) > 1 is good; < 0.5 indicates
inefficiency

# === Perf events ===
# Hardware events: cpu-cycles, instructions, cache-misses,
branches, branches-misses
# Software events: context-switches, page-faults, cpu-migrations
# Tracepoint events: sched:sched_switch, block:block_rq_complete

# === perf for flamegraphs ===
sudo perf record -F 99 -a -g -- sleep 60      # System-wide call
graph
sudo perf script > out.perf                  # Export for
flamegraph script
# Then use Brendan Gregg's FlameGraph tools:
# git clone https://github.com/brendangregg/FlameGraph
# stackcollapse-perf.pl out.perf > out.folded
# flamegraph.pl out.folded > flamegraph.svg

# === perf diff – compare profiles ===

```

```
# After deploying a code change or config change:
sudo perf record -F 99 -p <PID> -g -o perf.before -- sleep 30
# (make change)
sudo perf record -F 99 -p <PID> -g -o perf.after -- sleep 30
perf diff perf.before perf.after      # Compare profiles
```

Brendan Gregg's USE Method & Methodology

The **USE Method** (Utilization, Saturation, Errors) is a systematic approach to performance analysis. **For every resource (CPU, memory, disk, network), check: utilization (how busy?), saturation (how much queue?), errors (any failures?).** This method, developed by Brendan Gregg, is the standard approach at FAANG for performance investigations.

```
# === CPU ===
# Utilization:
mpstat -P ALL 1                # Per-CPU utilization
# Saturation:
vmstat 1                      # 'r' column = run queue
                              (processes waiting for CPU)
# Rule: r < #cores is normal; r > 2x#cores means CPU saturation
# Errors:
cat /proc/stat | grep -i "cpu"  # Look for strange counters

# === Memory ===
# Utilization:
free -h                      # Total, used, available
cat /proc/meminfo            # Detailed breakdown
# Saturation:
vmstat 1                      # 'si' = swap in, 'so' = swap
                              out (>0 = swapping!)
# Errors:
cat /proc/vmstat | grep -E "(oom|fail|error)"
dmesg | grep -i oom           # OOM kills

# === Disk ===
```

```

# Utilization:
iostat -x 1                                # %util > 80% = degraded
# Saturation:
iostat -x 1                                # 'aqu-sz' > 0 = requests
queued
# Errors:
iostat -x 1                                # Look at 'rrqm/s', 'wrqm/s'
- merges
cat /sys/block/sda/device/state            # 'running' = ok

# === Network ===
# Utilization:
sar -n DEV 1                               # Bandwidth usage vs
interface capacity
# Saturation:
ip -s link                                 # 'overruns', 'dropped',
'backlog'
ss -s                                       # Socket stats
# Errors:
ip -s -s link                              # Errors, dropped, overruns
netstat -i                                # Interface error counters

# === Quick triage: 60-second checklist (Brendan Gregg) ===
# 1. uptime                               - Load averages
# 2. dmesg | tail                          - Kernel errors
# 3. vmstat 1                             - Run queue, swapping,
context switches
# 4. mpstat -P ALL 1                       - Per-CPU balance
# 5. pidstat 1                             - Per-process CPU
# 6. iostat -xz 1                          - Disk I/O
# 7. free -m                              - Memory
# 8. sar -n DEV 1                          - Network
# 9. sar -n TCP,ETCP 1                     - TCP stats
# 10. top                                  - Overview

```

ftrace: Kernel Function Tracing

ftrace is the kernel's built-in tracer. It can trace function calls, interrupts, scheduling, and latency events with **zero overhead when disabled** and **very low overhead when enabled**. It's available on every Linux system without installing additional tools.

```
# === ftrace basics ===
# Mount tracefs:
mount -t tracefs tracefs /sys/kernel/tracing

# === Available tracers ===
cat /sys/kernel/tracing/available_tracers
# function      - Trace function calls (high overhead)
# function_graph - Trace with call graph (nested function calls)
# preemptirqsoff - Trace preemption and IRQ-off latency
# wakeup        - Trace scheduling latency
# nop           - No tracing (default)

# === Function tracing ===
# Trace a specific function:
echo function > /sys/kernel/tracing/current_tracer
echo do_sys_open > /sys/kernel/tracing/set_ftrace_filter
echo 1 > /sys/kernel/tracing/tracing_on
# ... run your test ...
echo 0 > /sys/kernel/tracing/tracing_on
cat /sys/kernel/tracing/trace          # View trace output

# === Latency tracing ===
# Measure preemption-off latency:
echo preemptirqsoff > /sys/kernel/tracing/current_tracer
echo 0 > /sys/kernel/tracing/tracing_max_latency
# Wait for events...
cat /sys/kernel/tracing/tracing_max_latency # Show max latency (µs)
cat /sys/kernel/tracing/trace              # Show what caused it

# === trace-cmd (front-end for ftrace) ===
sudo apt install trace-cmd
trace-cmd record -p function_graph -g do_sys_open ls
```

```

trace-cmd report                                # View recorded trace

# === wakeup latency ===
# Measures how long it takes after a task is woken up for it to
actually run
echo wakeup > /sys/kernel/tracing/current_tracer
echo <PID> > /sys/kernel/tracing/set_ftrace_pid

```

CPU & Memory Profiling in Practice

When an application is slow, profiling identifies exactly where time is spent. **CPU profiling** samples what functions are on the CPU. **Off-CPU profiling** (missing in traditional profilers) shows what the process is waiting for. **At FAANG, we use both on-CPU and off-CPU analysis to understand the full picture.**

```

# === On-CPU profiling ===
# Where is the CPU spending time?
perf record -F 99 -p <PID> -g -- sleep 30
perf report -g --stdio

# === Off-CPU profiling (why is the process blocked?) ===
# Use offcputime from bcc/BPF:
sudo offcputime -p <PID> 30          # Show stack traces + blocked
time

# === Memory profiling ===
# /usr/bin/time -v -- detailed memory usage
/usr/bin/time -v myprogram 2>&1 | grep -E "(Maximum
resident|Minor|Major)"
# Maximum resident set size -- peak memory
# Minor page faults -- page already in cache
# Major page faults -- had to read from disk (BAD for performance)

# === Cache misses ===
perf stat -e cache-misses,cache-references,LLC-loads,LLC-load-
misses ./myapp

```

```
# Cache miss rate > 5% may indicate poor data locality

# === Branch mispredictions ===
perf stat -e branches,branch-misses ./myapp
# High branch miss rate (>5%) can indicate unpredictable code paths

# === IPC (Instructions Per Cycle) ===
perf stat -e cycles,instructions ./myapp
# IPC > 2: excellent (highly optimized)
# IPC 1-2: good
# IPC < 1: stalled (cache misses, branch mispredicts, memory latency)

# === Context switch profiling ===
perf stat -e context-switches,cpu-migrations,page-faults ./myapp
# High context switches: over-threaded, bad thread pool sizing
# High cpu-migrations: scheduler moving threads between CPUs (bad cache locality)

# === GDB (GNU Debugger) for performance ===
# If a process is stuck:
sudo gdb -p <PID> -batch -ex "thread apply all bt" 2>&1
# Shows all thread stack traces – great for deadlock detection
```

FAANG Interview Q&A

Q: 'A service is using 100% CPU but throughput is low. What do you do?'

A: 'Run `perf top -p` to see which function is consuming CPU. This usually tells you the problem immediately. Common causes: (1) busy-waiting/polling loop, (2) inefficient algorithm ($O(n^2)$ instead of $O(n \log n)$), (3) excessive string operations (regex, JSON parsing), (4) thread contention (spinlocks, mutex contention). If perf points to a library function, look at the call chain with `perf record -g`. A flamegraph makes this even clearer. At FAANG, we

run perf in staging before every major deployment to catch performance regressions.'

Q: 'How would you profile a 10-second latency spike that happens once per hour?'

A: 'This is the hardest profiling problem — an intermittent issue. Approach: (1) Set up continuous profiling (perf with rotating buffers, or BPF tools) that captures profiles and keeps them in a ring buffer. When the latency spike occurs, save the buffer. (2) Use latency tracing via ftrace to capture the longest kernel preemption-off interval during the spike. (3) Correlate with other metrics — I/O latency, network latency, GC logs. (4) The cause is often: cron job (logrotate, backup), GC pause, network retransmission, or kernel interference (kswapd, journal commit). At FAANG, we use distributed tracing (Jaeger, Zipkin) that captures individual request latency breakdowns across services.'

Q: 'Your application uses more memory than expected. How do you find the leak?'

A: 'First, confirm it's a memory leak and not just normal growth. Run `ps aux --sort=-%mem` over time. Use `/usr/bin/time -v` to track peak RSS. For heap analysis: (1) JVM: enable `-XX:+HeapDumpOnOutOfMemoryError` and analyze the .hprof file with Eclipse MAT. (2) Node.js: use `--inspect` and Chrome DevTools heap snapshot. (3) Python: use tracemalloc or objgraph. (4) Native code: use Valgrind (memcheck) for precise leak detection, or `perf`

`mem` for sampling. At FAANG, memory profiling is part of the CI pipeline — we track RSS trends across builds and alert on regressions.'

References & Further Reading

- **Brendan Gregg's Linux Performance** — The definitive resource (tools, methodologies, talks)
- **Systems Performance: Enterprise and the Cloud** — Brendan Gregg, the bible of performance analysis
- **perf Wiki** — Official perf documentation and examples

FAANG Architect's Note: Performance analysis is a superpower. Most engineers can tell you something is slow; few can tell you exactly why. The engineers who master perf, flamegraphs, and the USE method are the ones who get called in when things break at scale. The good news: you don't need to know everything at once. Start with `perf top` and `perf stat`. Then learn flamegraphs. Then learn off-CPU profiling. Each tool adds another dimension to your understanding. The key insight: the computer is not a black box — every cycle, every byte, every instruction is measurable.

Production Tip: Never profile in production with default sampling rates on the first try. perf's default sampling rate can add measurable overhead on high-CPU systems. Start with `-F 99` (99 Hz) instead of the default; even lower (e.g., `-F 49`) on heavily loaded systems. For always-on profiling at FAANG, we use lightweight profilers like BPF-based tools that sample at very low rates (<10 Hz) and run continuously. When a specific issue is identified, we then turn up the sampling rate for targeted investigation.

CHAPTER 19

BPF & eBPF Deep Dive

Advanced · 9 topics

BPF (Berkeley Packet Filter), evolved into eBPF (extended BPF), is a revolutionary kernel technology that allows running sandboxed programs in the kernel without changing kernel code or loading modules. eBPF enables safe, efficient, and programmable observability, networking, and security. At FAANG, eBPF powers critical infrastructure: Cilium (Kubernetes networking), Falco (security), Pixie (observability), and custom monitoring tools.

FAANG

Linux

eBPF

BPF

bpfttrace

bcc

XDP

observability

networking

eBPF Architecture & Concepts

eBPF programs are written in a restricted C, compiled to BPF bytecode, verified by the kernel (safety checks: no loops with unbounded iterations, valid memory access, max instruction count), and attached to events (syscalls, tracepoints, kprobes, network interfaces). **The in-kernel verifier is what makes eBPF safe — it rejects any program that could crash or hang the kernel.** At FAANG, eBPF is used for: performance monitoring (bcc/bpfttrace tools), network observability (Cilium), security auditing (Falco), and custom container monitoring.

```
# === Check eBPF support ===
cat /proc/sys/kernel/unprivileged_bpf_disabled # 0 = enabled, 1 =
disabled
# Check if BPF syscall is available:
grep bpf /proc/kallsyms | head -5

# === eBPF program types ===
# Kprobes      - Dynamic kernel function tracing
# Tracepoints  - Static kernel tracepoints
# Perf events  - PMC-based sampling
# XDP          - eXpress Data Path (network driver level)
# TC          - Traffic Control (network stack level)
# cgroup       - cgroup operations (device, socket, sysctl)

# === The eBPF lifecycle ===
# 1. Write BPF program (C / Rust / Python / Lua)
# 2. Compile to BPF bytecode (LLVM/clang)
# 3. Load into kernel (bpf() syscall)
# 4. Kernel verifier checks safety
# 5. JIT compiler translates to native code
# 6. Attach to event (kprobe, tracepoint, etc.)
# 7. Program runs on event, writes to BPF maps
# 8. User-space reads maps via bpf() syscall

# === BPF maps (data structures) ===
# Hash maps, arrays, per-CPU arrays, stacks, queues, ring buffers
# Shared between kernel BPF program and user-space
# Used for: aggregated metrics, configuration, state

# === Limitations ===
# - Max 4096 instructions (can be extended to 1M with tail calls)
# - No loops until recently (bounded loops added in Linux 5.17)
# - No kernel function calls (must be inlined) until BPF helpers
# - Stack limited to 512 bytes
```

bpftool: High-Level eBPF Tracing

bpfttrace is a high-level tracing language for Linux eBPF. It's like awk for kernel tracing — you write one-liners to observe kernel and user-space events. At FAANG, bpfttrace is the go-to tool for ad-hoc performance investigations and root cause analysis.

```
# === Install bpfttrace ===
sudo apt install bpfttrace

# === File open tracing ===
bpfttrace -e 'tracepoint:syscalls:sys_enter_openat { printf("%s\n", comm,
str(args->filename)); }'
# Shows every file opened, by which process

# === New process tracing ===
bpfttrace -e 'tracepoint:syscalls:sys_enter_execve { printf("%s ->
%s\n", comm,
str(args->filename)); }'

# === Kill tracing (who is sending SIGKILL?) ===
bpfttrace -e 'tracepoint:syscalls:sys_enter_kill { printf("%s -> PID
%d SIG %d\n", comm,
args->pid, args->sig); }'

# === OOM tracing ===
bpfttrace -e 'kprobe:oom_kill_process { printf("OOM killed %s
(%d)\n",
str(args->task->comm), args->task->pid); }'

# === Count syscalls by process ===
bpfttrace -e 'tracepoint:syscalls:sys_enter_* { @[comm] = count();
}'

# === Block I/O tracing ===
bpfttrace -e 'kprobe:blk_account_io_start { printf("I/O: %s %s\n",
comm, args->rwb); }'

# === TCP connection tracing ===
bpfttrace -e 'kprobe:tcp_connect { printf("%s connected to %s:%d\n",
```

```

comm,
ntop(args->sk->__sk_common->skc_daddr), args->sk->__sk_common->skc_dport); }'

# === One-shot latency histogram (read syscall) ===
bpftrace -e 'kprobe:vfs_read { @start[tid] = nsecs; }
kretprobe:vfs_read /@start[tid]/ {
@us = hist((nsecs - @start[tid]) / 1000); delete(@start[tid]); }'

# === Profile: CPU sampling ===
bpftrace -e 'profile:hz:99 { @[kstack, comm] = count(); }' #
Kernel stack
bpftrace -e 'profile:hz:99 { @[ustack, comm] = count(); }' # User
stack

# === Built-in tools ===
sudo bpftrace -l                # List all probes
sudo bpftrace -l 'tracepoint:syscalls:*' # List syscall
tracepoints
# Many pre-built tools come with bpftrace:
ls /usr/share/bpftrace/tools/
# opensnoop.bt, killsnoop.bt, execsnoop.bt, tcplife.bt, ...

# Run a pre-built tool:
sudo bpftrace /usr/share/bpftrace/tools/opensnoop.bt

```

bcc: The BPF Compiler Collection

bcc (BPF Compiler Collection) provides Python- and Lua-based tools for eBPF tracing. It's more flexible than bpftrace (full programming language support) but also more complex. **bcc tools are production-ready — many are used daily at FAANG for performance monitoring.**

```

# === Install bcc ===
sudo apt install bpfcc-tools # Ubuntu package name
# Tools are prefixed with -bpfcc:

```

```
# === Essential bcc tools ===
# opensnoop: trace file opens
sudo opensnoop-bpfcc

# execsnoop: trace new processes
sudo execsnoop-bpfcc

# biotop: block I/O by process
sudo biotop-bpfcc

# tcplife: TCP connection lifecycle
sudo tcplife-bpfcc

# tcptop: TCP throughput by process
sudo tcptop-bpfcc

# cpuunclaimed: CPU not claimed by scheduler
sudo cpuunclaimed-bpfcc

# offcputime: trace blocked time (off-CPU analysis)
sudo offcputime-bpfcc -K 30

# runqlat: scheduler run-queue latency
sudo runqlat-bpfcc

# profile: CPU sampling with frames
sudo profile-bpfcc -af 30

# cachestat: page cache hit rate
sudo cachestat-bpfcc

# llcstat: Last-level cache misses
sudo llcstat-bpfcc

# === Custom bcc Python script (minimal example) ===
cat > trace_open.py << 'EOF'
from bcc import BPF

# BPF C program
bpf_program = """
int kprobe__do_sys_openat2(struct pt_regs *ctx, int dfd, const char
```

```
__user *filename, int
flags) {
    bpf_trace_printk("openat: %s\\n", filename);
    return 0;
}
"""

b = BPF(text=bpf_program)
b.trace_print()
EOF

sudo python3 trace_open.py
```

XDP & eBPF Networking

XDP (eXpress Data Path) is an eBPF-based packet processing framework that runs at the network driver level — before the kernel's network stack processes the packet. This enables DDoS mitigation, load balancing, and packet filtering at line rate (millions of packets per second). At FAANG, XDP is used for: DDoS protection (Cloudflare), Kubernetes service load balancing (Cilium), and custom network filtering.

```
# === XDP program flow ===
# 1. Packet arrives at NIC
# 2. XDP program runs (driver level, before skb allocation)
# 3. Actions: XDP_DROP, XDP_PASS, XDP_TX, XDP_REDIRECT
# 4. Result: drop packet (fast), pass to kernel stack (normal),
#           transmit back out (hairpin), or redirect to another
NIC

# === XDP vs kernel bypass (DPDK) ===
# XDP: kernel is still in control, safe, programmable, no userspace
changes
# DPDK: bypasses kernel entirely, userspace driver, fastest
possible,
```

```
# but requires dedicated CPUs and application changes

# === Cilium (Kubernetes CNI powered by eBPF) ===
# Cilium uses eBPF for:
# - Service load balancing (replacing kube-proxy)
# - Network policies (L3/L4/L7)
# - Observability (Hubble)
# - Transparent encryption

# === Check if XDP is supported ===
ip link show eth0 # Check features: xdp
# Enable XDP on interface:
sudo ip link set dev eth0 xdp obj my_xdp_prog.o sec xdp

# === XDP example: DDoS blacklist ===
# Simple XDP program to drop packets from blacklisted IPs
# (Conceptual – real XDP uses BPF maps for dynamic blacklists)

# === eBPF for security: Falco ===
# Falco uses eBPF tracepoints/kprobes to detect:
# - Container escape attempts
# - Unauthorized file access
# - Suspicious network connections
# - Privilege escalation

# Install Falco (using eBPF probe):
# falco driver-loader bpf
# falco --bpf
```

FAANG Interview Q&A

Q: 'How does eBPF differ from a kernel module?'

A: 'A kernel module can access any kernel function, data structure, or memory — it can crash the entire system. eBPF programs are verified by the kernel's BPF verifier before loading — the verifier checks for: (1) valid memory access (no null pointers), (2) bounded

loops (termination guaranteed), (3) maximum instruction count (4096 default, up to 1M with tail calls), (4) valid helper function calls. If verification fails, the program is rejected. This safety guarantee is what makes eBPF deployable in production — a buggy eBPF program can't crash the kernel like a buggy module can.'

Q: 'What's a real-world use case for eBPF at FAANG?'

A: 'One example: Cilium at scale. At a FAANG company using Kubernetes with 10,000+ nodes, Cilium uses eBPF to replace kube-proxy for service load balancing — instead of iptables rules (which become $O(n)$ with number of services), eBPF programs are updated per-service and run at the XDP or TC layer. This eliminates the iptables scaling bottleneck. Another example: Datadog's continuous profiler uses eBPF to sample stack traces without any application changes — no agent code, no recompilation, no JVM flags.'

Q: 'What are the limitations of eBPF?'

A: '(1) Instruction limit: 4096 by default (can be extended). (2) Stack limit: 512 bytes — you can't allocate large local data structures. (3) No unbounded loops (before Linux 5.17). (4) Limited helper functions — you can call only a fixed set of BPF helpers, not arbitrary kernel functions. (5) Verifier complexity — the verifier can be slow for complex programs. (6) No floating-

point arithmetic. (7) Kernel version dependence — newer BPF features require newer kernels. At FAANG, we run custom kernels (or at least Linux 5.10+) specifically to get modern eBPF features.'

References & Further Reading

- **What is eBPF?** — Official eBPF foundation introduction
- **bcc GitHub** — Tools and examples for BPF Compiler Collection
- **bpfttrace GitHub** — One-liners, tools, and documentation
- **Cilium** — Kubernetes CNI powered by eBPF

FAANG Architect's Note: eBPF is the most important Linux kernel technology of the last decade. It's transforming observability, networking, and security — without eBPF, you'd need kernel modules (dangerous) or ptrace (slow) for most of these use cases. At FAANG, eBPF is not a 'nice to have' — it's infrastructure. Cilium, Falco, Pixie, and Datadog's continuous profiler all rely on eBPF. If you're working on infrastructure at FAANG, learning eBPF (even just the tooling like bpfttrace and bcc) will give you debugging superpowers that traditional tools can't match.

Production Tip: bpfttrace is safe for production use — it's designed to never crash the system. However, certain bcc tools (especially those using kprobes at high frequency) can add measurable overhead. Start with the pre-built tools (execsnoop, opensnoop, etc.) — they're optimized for low overhead. For custom BPF programs, test in a staging environment first and always monitor CPU usage of the BPF program itself (visible via `bpftool prog show`).

CHAPTER 20

Monitoring & Observability

Advanced · 7 topics

Monitoring and observability are how you understand what your systems are doing. **Monitoring tells you something is wrong; observability lets you ask why.** At FAANG, every service emits metrics, logs, and traces. This guide covers the Linux-native tooling for system monitoring and the concepts that scale from single-server debugging to fleet-wide observability.

FAANG

Linux

monitoring

observability

Prometheus

logging

SRE

The Three Pillars: Metrics, Logs, Traces

Metrics are numeric measurements (CPU, memory, requests per second, error rates) — best for alerting and dashboards. **Logs** are discrete events with timestamps — best for debugging specific incidents. **Traces** follow a request across services — best for understanding distributed system behavior. At FAANG, the 'three pillars' are served by specialized systems: Prometheus (metrics), Elasticsearch/Splunk (logs), Jaeger (traces). On a single Linux system, the same concepts apply with local tools.

```
# === CPU Metrics ===  
cat /proc/stat           # Overall CPU stats (user,  
nice, system, idle,
```

```

iowait, irq, softirq, steal)
mpstat -P ALL 1                # Per-CPU breakdown
sar -u 1 5                     # Historical CPU stats (from
sysstat package)

# === Memory Metrics ===
cat /proc/meminfo              # Detailed memory state
free -h                       # Quick summary
sar -r 1 5                     # Historical memory stats

# === Disk Metrics ===
iostat -x 1 5                  # Device-level I/O metrics
sar -b 1 5                     # Historical I/O stats
sar -d 1 5                     # Per-device I/O stats

# === Network Metrics ===
sar -n DEV 1 5                 # Per-interface bandwidth
sar -n TCP,ETCP 1 5            # TCP stats (connections,
retransmits, segments)
ip -s link                     # Interface errors, drops,
overruns

# === System Load ===
cat /proc/loadavg               # Load average (1/5/15 min)
# Load is NOT CPU % – it's the number of processes in running +
uninterruptible state
# On an 8-core system: load 8 = fully utilized (each core has 1
task)

# === Top/htop ===
top -b -n 1 | head -20        # Batch mode (non-
interactive)
htop                           # Interactive with better UX

# === Collect all metrics with a script ===
cat > quick_health.sh << 'EOF'
#!/bin/bash
echo "=== UPTIME ==="
uptime
echo "=== CPU ==="
grep 'cpu ' /proc/stat | awk '{usage=($2+$4)*100/($2+$4+$5)} END
{print usage "%"}'
```

```
echo "=== MEMORY ==="
free -h | grep Mem
echo "=== DISK ==="
df -h / /var/lib /data 2>/dev/null
echo "=== TOP PROCESSES ==="
ps aux --sort=-%cpu | head -6
echo "=== NETWORK ==="
ip -s link | grep -E "(eth|ens|enp)" -A 2
EOF
```

Prometheus & Node Exporter

Prometheus is the industry standard for metric collection and alerting. It scrapes HTTP endpoints and stores time-series data.

Node Exporter exposes Linux system metrics in Prometheus format. At FAANG, Prometheus is the default metrics system — every service exposes a /metrics endpoint, and node_exporter runs on every server for hardware/OS metrics.

```
# === Node Exporter ===
# Install:
sudo apt install prometheus-node-exporter
# Or download from https://prometheus.io/download/

# Node exporter exposes metrics at :9100/metrics
# Key metric groups:
# node_cpu_seconds_total      - CPU time by mode
# node_memory_MemTotal_bytes - Memory stats
# node_disk_io_time_seconds_total - Disk I/O
# node_network_receive_bytes_total - Network
# node_filesystem_avail_bytes - Disk space
# node_load1, node_load5, node_load15 - Load averages

# === Check node exporter metrics ===
curl -s localhost:9100/metrics | head -30

# === PromQL (Prometheus Query Language) examples ===
```

```

# CPU utilization (1m average):
# 100 - (avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}
[1m])) * 100)

# Memory utilization:
# (1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)
* 100

# Disk space > 80%:
# node_filesystem_avail_bytes{mountpoint="/" } /
node_filesystem_size_bytes{mountpoint="/" }
< 0.2

# === Prometheus configuration (prometheus.yml) ===
# scrape_configs:
#   - job_name: 'node'
#     static_configs:
#       - targets: ['localhost:9100', 'server1:9100',
'server2:9100']

# === Installation ===
sudo apt install prometheus
# Web UI at localhost:9090

# === Alertmanager rules ===
# groups:
#   - name: node_alerts
#     rules:
#       - alert: DiskSpaceLow
#         expr: node_filesystem_avail_bytes{mountpoint="/" } /
node_filesystem_size_bytes{mountpoint="/" } < 0.1
#         for: 5m
#         labels: { severity: critical }
#         annotations: { summary: "Disk space < 10% on {{
$labels.instance }}" }

```

Logging: journald, syslog & Logrotate

Linux has two logging systems: the legacy **syslog** (text files in `/var/log/`) and the modern **journald** (binary, structured). At FAANG, logs are shipped to centralized systems (Splunk, Elasticsearch, Datadog) — but the local management of logs (rotation, size limits, compression) is still critical to prevent filling the disk.

```
# === journald ===
# Centralized logging with structured metadata
# Config: /etc/systemd/journald.conf

# Key settings:
# SystemMaxUse=1G          - Limit total journal disk usage
# MaxRetentionSec=2week    - Auto-delete old logs
# Compress=yes             - Compress old journal files
# ForwardToSyslog=no       - Don't duplicate to syslog

# === logrotate ===
# Rotate, compress, and delete old log files
# Config: /etc/logrotate.conf + /etc/logrotate.d/

# Example: /etc/logrotate.d/nginx
/var/log/nginx/*.log {
    daily
    missingok
    rotate 14          # Keep 14 days
    compress
    delaycompress      # Don't compress newest file
    notifempty
    create 0640 www-data adm
    sharedscripts
    postrotate
        systemctl reload nginx # Signal nginx to reopen logs
    endscript
}

# Test logrotate:
sudo logrotate -d /etc/logrotate.d/nginx # Dry-run (debug)
sudo logrotate -f /etc/logrotate.d/nginx # Force run
```

```
# === /var/log/ key files ===
# /var/log/syslog          - General system log (Debian/Ubuntu)
# /var/log/messages       - General system log (RHEL/CentOS)
# /var/log/auth.log       - Authentication logs (SSH, sudo)
# /var/log/kern.log       - Kernel messages
# /var/log/dmesg          - Kernel ring buffer
# /var/log/nginx/         - Web server logs
# /var/log/mysql/        - Database logs

# === Centralized logging ===
# rsyslog - forward logs to central server
# /etc/rsyslog.conf:
# *.* @logserver.example.com:514    # Send all logs via UDP
# *.* @@logserver.example.com:514   # Send all logs via TCP
# (reliable)
# Use TLS for production:
# *.* @@logserver.example.com:6514

# === Log format for machine parsing ===
# JSON logs are preferred at FAANG (easier to parse)
# Example: logger
# '{"event":"deploy","service":"myapp","status":"ok"}'

# === Structured logging in applications ===
# KEY=value format for grep-ability:
# 2024-01-15T10:30:00 [INFO] user_id=abc123 action=login
# duration_ms=42

# Journald structured fields:
# journalctl -o verbose | grep -E "
# (MESSAGE=|SYSLOG_IDENTIFIER=|_PID=)"
```

Distributed Tracing & Observability Concepts

On a single server, you track a request through the OS. In a distributed system, you track it across services. **Distributed tracing** propagates a trace ID across service boundaries, allowing

you to reconstruct the full path of a request. At FAANG, tracing is a first-class concern — every framework and infrastructure component propagates trace context.

```
# === Single-server tracing (low-level) ===
strace -f -e trace=network -p <PID>      # Trace network syscalls
perf trace -p <PID>                      # Trace syscalls with perf

# === OpenTelemetry (industry standard) ===
# OpenTelemetry provides:
# - APIs and SDKs for generating traces, metrics, logs
# - Exporters to send data to backends (Jaeger, Prometheus,
  Datadog)
# - Context propagation (W3C TraceContext, Baggage)

# === Trace context propagation ===
# Trace ID is passed via HTTP headers:
# traceparent: 00-0af7651916cd43dd8448eb211c80319c-
  b7ad6b7169203331-01
#   version-trace_id-span_id-trace_flags

# === Key metrics from tracing ===
# Latency: time spent in each service
# Error rate: failed requests / total requests
# Saturation: utilization of each service
# Apdex: user satisfaction score based on latency thresholds

# === RED Method (for service monitoring) ===
# Rate: requests per second
# Errors: failed requests per second
# Duration: latency (average, p50, p95, p99)

# === USE Method (for resource monitoring) ===
# Utilization: % busy
# Saturation: queue length
# Errors: error count

# === Four Golden Signals (Google SRE) ===
# 1. Latency: time to serve a request
```

```
# 2. Traffic: demand on the system
# 3. Errors: rate of failed requests
# 4. Saturation: how 'full' the system is
```

FAANG Interview Q&A

Q: 'You're on call and get paged for high CPU on a production server. Walk through your response.'

A: 'Acknowledge the page. SSH in (or use session manager). Run `uptime` to see load averages — is it a spike or sustained? `mpstat -P ALL 1` — is one core pegged or all cores? `ps aux --sort=-%cpu | head` — which process is using the CPU? `perf top -p <PID>` — which function is hot? If it's a known pattern (e.g., GC in Java, V8 garbage collection in Node), check the application logs. If it's a new pattern, capture a perf record and create a flamegraph. Check if a deploy just happened — rollback if it's a regression. If the process is legitimately busy (traffic spike), add capacity (horizontal scaling) or check if upstream throttling was removed. File a postmortem ticket.'

Q: 'How do you set up monitoring for a new Linux service?'

A: 'First, identify the four golden signals. Install `node_exporter` to collect OS-level metrics. Add a `/metrics` endpoint to the application exposing RED metrics (rate, errors, duration). Set up Prometheus to scrape both. Create a Grafana dashboard with: (1) request rate and latency, (2) error rate by status code, (3) CPU/memory/disk of the service process, (4) database connection pool usage if applicable. Set alerts for: p99 latency > 500ms for 5

minutes, error rate > 1% for 5 minutes, disk > 80% full. Test the alerts by simulating the failure conditions. At FAANG, every service has a 'service level objective' (SLO) — a target for availability or latency — and monitoring measures against that SLO.'

Q: 'What's the difference between monitoring and observability?'

A: 'Monitoring is tracking known failure modes — CPU > 90%, disk full, process not running. It answers 'what is broken?'

Observability is the ability to ask arbitrary questions about your system without knowing the answers in advance — 'why is this specific request slow?', 'what is this process waiting for?'. Monitoring requires predefined dashboards and alerts.

Observability requires structured data (traces, structured logs, high-cardinality metrics) and tools to explore it. At FAANG, we invest heavily in observability because the most impactful outages are rarely the things you thought to monitor in advance.'

References & Further Reading

- **Prometheus Documentation** — Official Prometheus docs and best practices
- **OpenTelemetry Documentation** — Industry standard for observability
- **Google SRE Book: Monitoring** — The four golden signals and monitoring philosophy

FAANG Architect's Note: The most important skill in monitoring is knowing what to monitor. At FAANG, we don't monitor everything — we monitor the things that tell us whether the system is healthy. For each service, define: 'If this metric goes bad, will I be woken up at 3 AM?' If not, don't alert on it. Put it on a dashboard if it's useful for debugging, but don't page people for things they can't act on. The best monitoring setup is the one where pages are rare (because the system is stable) and actionable (because when a page comes, you know exactly what to do).

Production Tip: Never alert on average latency. Average hides outliers — a 1% error rate and 99% good requests can still have a 'good' average. Always alert on percentiles: p99 latency (the worst 1% of requests) and error rate. At FAANG, we monitor p50 (typical), p95 (slow), p99 (very slow), and p99.9 (outliers). A request that takes 10 seconds for 0.1% of users can be invisible in the average but devastating for those users. Set your SLOs on p99, build dashboards on p50, and debug with p99.9.

CHAPTER 21

GPU Computing on Linux

Advanced · 8 topics

GPU computing on Linux is the backbone of ML/AI at FAANG. Understanding the GPU driver stack — from kernel modules to CUDA runtime to container integration — is essential for ML engineers, infrastructure engineers, and anyone deploying GPU workloads at scale. The Linux ecosystem for GPUs spans NVIDIA (dominant), AMD (ROCm), and Intel (oneAPI).

FAANG Linux GPU CUDA NVIDIA MIG
machine learning profiling

GPU Stack: From Kernel to Application

The GPU software stack has multiple layers: **kernel drivers** (nvidia.ko, nvidia-uvm.ko), **user-space libraries** (libcuda.so, libcudart.so), **compute frameworks** (CUDA, cuDNN, TensorRT), and **orchestration** (NVIDIA container toolkit, MIG). At FAANG, this stack is baked into custom AMIs and container images — engineers don't install it manually, but understanding it is essential for debugging.

```
# === Check GPU hardware ===  
lspci | grep -i nvidia          # List NVIDIA GPUs  
lspci -v -s 01:00.0            # Detailed GPU info  
  
# === NVIDIA driver modules ===  
lsmod | grep nvidia            # nvidia, nvidia_uvm,
```

```

nvidia_drm, nvidia_modeset
# nvidia.ko      - Core driver
# nvidia-uvm.ko  - Unified Virtual Memory (CUDA unified memory)
# nvidia-drm.ko  - Direct Rendering Manager (display)
# nvidia-modeset.ko - Mode setting (display)

# === nvidia-smi (the single most important GPU command) ===
nvidia-smi          # Summary: GPUs, processes,
memory, temp
nvidia-smi -L        # List all GPUs with UUIDs
nvidia-smi --query-
gpu=index,name,temperature.gpu,utilization.gpu,memory.used,memory.t
otal
--format=csv
nvidia-smi pmon -c 1      # Per-process GPU usage
nvidia-smi dmon -c 1      # Per-device monitoring
nvidia-smi topo -m        # GPU topology (NVLink, PCIe)

# === GPU health ===
nvidia-smi -q -d TEMPERATURE # Temperature details
nvidia-smi -q -d POWER        # Power draw and limits
nvidia-smi -q -d CLOCK        # Clock speeds (graphics,
memory, SM)

# === ECC (Error Correcting Code) ===
nvidia-smi -q -d ECC          # ECC errors (memory bit
flips)
# ECC is ON by default on datacenter GPUs (A100, H100, V100)
# ECC OFF on consumer GPUs (RTX 4090) - trades reliability for
speed

# === GPU persistence daemon ===
sudo nvidia-persistenced --verbose # Keep GPU initialized
between processes
# Without it: first CUDA call on each GPU takes ~1 second (driver
init)
# With it: driver stays loaded, zero startup latency

# === Reset GPU (if hung) ===
sudo nvidia-smi --gpu-reset 0      # Reset GPU 0 (only on
Tesla/Quadro)

```

GPU Performance & Profiling

GPU profiling identifies bottlenecks: is your kernel compute-bound (SM utilization high), memory-bound (memory bandwidth high), or latency-bound (kernel launch overhead)? **At FAANG, every ML training run is profiled — it's part of the CI pipeline for model training.** Tools: nvidia-smi, NSight Systems, NSight Compute, and PyTorch profiler.

```
# === Real-time GPU monitoring ===
watch -n 0.5 nvidia-smi          # Refresh every 500ms

# === GPU utilization patterns ===
# 0-30%: Underutilized (small batch size, CPU bottleneck, data
#        loading issue)
# 30-70%: Moderate (typical for distributed training with comm
#        overhead)
# 70-95%: Good utilization (efficient training)
# 95-100%: Saturated (GPU is the bottleneck – ideal for throughput)

# === Key metrics to watch ===
# Volatile GPU-Util – SM utilization (not a perfect metric, hides
# underutilization)
# Memory-Usage – How much GPU memory is used
# Temperature – Throttling starts at ~85°C for most GPUs
# Power Draw – If below TDP, GPU may be power-limited or
# underutilized

# === CUDA-MEMCHECK (memory errors in GPU kernels) ===
cuda-memcheck ./my_cuda_program    # Check for out-of-bounds,
# misaligned access

# === NSight Systems (system-level profiling) ===
# Profile CPU + GPU interactions:
nsys profile --stats=true -o profile python train.py
# Shows: CPU kernel time, GPU kernel time, CUDA API calls, memory
# operations
```

```
# === NSight Compute (kernel-level profiling) ===
ncu --target-processes all python train.py # Profile individual
CUDA kernels

# === PyTorch profiler ===
# python -m torch.utils.bottleneck train.py
# Or:
# with torch.profiler.profile(activities=[...]) as prof:
#     run_training()
# print(prof.key_averages().table(sort_by="cuda_time_total"))

# === NCCL (NVIDIA Collective Communications Library) ===
# Used for multi-GPU and multi-node communication
# Debug: NCCL_DEBUG=INFO, NCCL_DEBUG_SUBSYS=ALL
# Performance: NCCL_ALGO=Ring, NCCL_PROTO=Simple
nccl-tests/build/all_reduce_perf -b 128M -e 8G -f 2 -g 8 #
Benchmark NCCL

# === CUDA graphs (reduce kernel launch overhead) ===
# Captures a sequence of GPU operations and replays them
# Reduces CPU launch overhead – critical for small-batch inference
# Enabled by default in PyTorch 2.x with torch.compile
```

GPU Multi-Tenancy: MIG, Time-Slicing & MPS

Modern datacenter GPUs support sharing a physical GPU across multiple workloads. **MIG (Multi-Instance GPU)** partitions A100/H100 into up to 7 isolated instances with dedicated memory and compute. **Time-slicing** shares GPU time (no memory isolation). **MPS** shares CUDA context across processes. At FAANG, MIG is used for cost-efficient inference serving — partitioning a single A100 into 3x20GB instances for 3 models.

```
# === MIG (Multi-Instance GPU) – A100/H100 only ===
# List MIG capabilities:
nvidia-smi mig -i 0 # Check MIG support for GPU 0
```

```

# Enable MIG on GPU 0:
sudo nvidia-smi -mig 1 -i 0
sudo nvidia-smi mig -cgi 19,19,19,19 # Create 4 compute instances,
20GB each
# 19 = 'MIG 3g.20gb' profile (3 GPU slices, 20GB) on A100
# Profiles: 1g.10gb, 2g.20gb, 3g.40gb, 4g.40gb, 7g.80gb

# List MIG instances:
nvidia-smi mig -lgi

# Assign MIG device to container:
docker run --gpus '"device=MIG-<UUID>"' nvidia/cuda:12.0-runtime

# === Time-slicing (Kubernetes) ===
# GPU sharing without memory isolation
# Each pod gets GPU time but can use all GPU memory
# Best for: burstable inference workloads
# Configure via NVIDIA device plugin with 'time-slicing' config

# === MPS (Multi-Process Service) ===
# Enables CUDA context sharing across processes
# Better utilization for many small inference requests
export CUDA_MPS_PIPE_DIRECTORY=/tmp/mps_pipe
export CUDA_MPS_LOG_DIRECTORY=/tmp/mps_log
nvidia-cuda-mps-control -d # Start MPS daemon
# All CUDA processes now share a single CUDA context
# echo "quit" | nvidia-cuda-mps-control # Stop MPS

# === GPU memory bandwidth contention ===
# Multiple processes sharing GPU memory bandwidth can conflict
# MIG isolates memory bandwidth; time-slicing does not
# At FAANG: inference workloads use MIG; training gets full GPU

# === GPU oversubscription (CUDA Unified Memory) ===
# GPU can access more memory than physical VRAM via page faults
# Slow! Only use for prototyping
# Production: allocate within GPU memory limits

```

NVIDIA Container Toolkit & Kubernetes GPU

GPU access in containers requires the **NVIDIA Container Toolkit** (nvidia-docker2 / nvidia-container-toolkit). It mounts GPU drivers and CUDA libraries into containers. At FAANG, GPU pods are scheduled via the **NVIDIA Kubernetes Device Plugin**, which advertises GPU count per node and handles MIG partitioning.

```
# === NVIDIA Container Toolkit ===
# Install:
sudo apt install nvidia-container-toolkit
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker

# Test:
docker run --gpus all nvidia/cuda:12.0-runtime nvidia-smi

# === How it works ===
# The toolkit mounts:
#   /usr/lib/x86_64-linux-gnu/libcuda.so - CUDA driver library
#   /usr/lib/x86_64-linux-gnu/libnvidia-* - NVIDIA libraries
#   /proc/driver/nvidia/                - Driver information
#   /dev/nvidia*                        - GPU device files
# Without toolkit: container can't access /dev/nvidia0

# === Kubernetes GPU scheduling ===
# NVIDIA device plugin registers GPU resources
# kubectl describe node | grep nvidia.com/gpu
# Pod requests GPU:
#   resources:
#     limits:
#       nvidia.com/gpu: 1
#
# Device plugin also handles MIG profiles:
#   nvidia.com/mig-1g.10gb: 2

# === CUDA version compatibility ===
# nvidia-smi shows CUDA Version (driver API version)
```

```
# Container can use any CUDA version <= driver version
# Driver: 545.23.08 = supports CUDA 12.3 and older
# Container can run: CUDA 12.3, 12.2, 12.1, ..., 11.x, 10.x, etc.
# This is why FAANG uses: driver on host, CUDA toolkit in container

# === GPU topology-aware scheduling ===
# NVLink-connected GPUs communicate faster than PCIe
# nvidia-smi topo -m shows connectivity
# Kubernetes: use 'nvidia.com/gpu.nvlink' label for topology-aware
scheduling

# === GPU monitoring in containers ===
# Inside container:
dcgmi group -c all                # Create DCGM group
dcgmi stats -g 1 -s true          # Enable stats collection
dcgmi dmon -e 100,101,102,200,201 # Key metrics: GPU util, mem
util, temp, power,
PCIe

# Prometheus + DCGM Exporter:
# https://github.com/NVIDIA/dcgmi-exporter
# Exposes: DCGM_FI_DEV_GPU_UTIL, DCGM_FI_DEV_MEM_COPY_UTIL, etc.
```

FAANG Interview Q&A

Q: 'Your training job is running at 30% GPU utilization. What do you check?'

A: 'Check nvidia-smi to confirm utilization. Then identify the bottleneck: is it data loading (CPU/IO bound), compute (kernel inefficient), or communication (NCCL sync). Use nvidia-smi dmon for per-GPU metrics, NSight Systems to profile the full pipeline. Common causes: (1) Data loader bottleneck — increase num_workers, use NVMe storage, enable pinned memory. (2) Small batch size — GPU finishes compute quickly, waits for next

batch. (3) Gradient accumulation steps are inefficient. (4) Model has many small kernels with launch overhead — use `torch.compile` or CUDA graphs. (5) AllReduce communication overhead in multi-GPU — check NCCL settings and network bandwidth. At FAANG, we have a standard 'GPU health check' script that profiles the full stack before every training run.'

Q: 'How do you choose MIG vs time-slicing vs full GPU for production inference?'

A: 'MIG provides hard isolation — guaranteed memory and compute, no interference. Use MIG when you need QoS guarantees (e.g., a latency-sensitive model sharing GPU with a throughput-oriented model). Time-slicing provides compute sharing but no memory isolation — a memory leak in one container can crash others. Use time-slicing for burstable workloads where occasional interference is acceptable. Full GPU is for large models that need all memory (LLMs, large Vision models) or for training. At FAANG, we use MIG for production inference serving and time-slicing for development/staging environments where cost efficiency matters more than isolation.'

Q: 'An ML engineer says 'CUDA out of memory' on an A100 with 80GB, but the model should fit in 40GB. What do you check?'

A: 'Several possibilities: (1) Memory fragmentation — allocate largest tensors first, use memory pools. (2) Intermediate memory — activation memory can exceed parameter memory for large batch sizes. Reduce batch size or use gradient checkpointing. (3)

Memory leak — tensors not freed between epochs (Python references holding GPU memory). Use `torch.cuda.empty_cache()` and check for reference cycles. (4) The engineer is not using MIG and another process is using the remaining memory — check `nvidia-smi` for other processes on that GPU. (5) NCCL buffers — NCCL allocates scratch memory for AllReduce. Set `NCCL_BUFFSIZE` or use environment variables. At FAANG, we run `nvidia-smi` and then do a memory allocation test to find the exact available memory before launching training.'

References & Further Reading

- **nvidia-smi Documentation** — Complete reference for GPU monitoring and management
- **NVIDIA Container Toolkit** — GPU access in Docker and Kubernetes
- **NVIDIA Deep Learning Documentation** — CUDA, cuDNN, TensorRT, and NGC catalog

FAANG Architect's Note: GPU computing is where Linux infrastructure meets ML engineering. The most common failure mode I see: engineers treating GPUs as 'magic accelerators' rather than understanding the driver stack. When a training job is slow, the instinct is to blame the GPU — but 90% of the time, the bottleneck is elsewhere: data loading (CPU/IO), framework overhead (Python GIL, tensor copies), or network communication (NCCL sync). The engineers who stand out are those who can profile the entire pipeline: they watch `nvidia-smi`, `strace` the data loader, NCCL debug the communication, and `NSight-profile` the kernels. Build this skill systematically — each training job is an opportunity to understand the stack better.

Production Tip: Always set `CUDA_LAUNCH_BLOCKING=1` when debugging GPU hangs. This forces synchronous kernel launches — instead of queuing all kernels and then hanging, CUDA executes them one at a time, and you'll see exactly where it hangs. In production, remove this (synchronous launches kill performance). Also: never use `torch.cuda.manual_seed_all()` in production unless you need deterministic results — it slows down initial kernel launches on some GPU architectures.

CHAPTER 22

Linux Build & Toolchain

Advanced · 8 topics

Every engineer compiles code — but understanding the toolchain (compiler, linker, loader, libraries, build systems) separates those who debug build issues in minutes from those who fight them for hours. At FAANG, build toolchain knowledge is essential for: debugging linker errors, optimizing binary performance, reducing container image size, and working with hermetic build systems like Bazel.

FAANG Linux build toolchain compiler linker Bazel
optimization

The C Compilation Pipeline

C/C++ compilation is a multi-stage pipeline: **preprocessing** (handle `#include`, `#define`, `#ifdef`), **compilation** (translate to assembly), **assembly** (translate to machine code/object file), **linking** (combine object files + libraries into executable). Understanding each stage helps you debug build errors and optimize build times.

```
# === The four stages ===  
# Stage 1: Preprocessing  
gcc -E hello.c -o hello.i          # Preprocessed output  
(expanded macros, includes)  
  
# Stage 2: Compilation
```

```

gcc -S hello.i -o hello.s           # Assembly output (human-
readable)

# Stage 3: Assembly
gcc -c hello.s -o hello.o           # Object file (machine code,
not executable)

# Stage 4: Linking
gcc hello.o -o hello                 # Executable (linked with
libc, crt0, etc.)

# Or all at once:
gcc -Wall -O2 -o hello hello.c

# === Object file inspection ===
file hello.o                         # ELF 64-bit LSB relocatable,
x86-64
nm hello.o                           # Symbol table (functions,
globals)
nm -C hello.o                        # Demangled C++ names
objdump -d hello.o                   # Disassembly
objdump -t hello.o                   # Symbol table
objdump -x hello.o                   # All headers, sections,
symbols

# === Shared library dependencies ===
ldd /usr/bin/python3                 # List shared library
dependencies
# linux-vdso.so.1 – Virtual Dynamic Shared Object (kernel-provided)
# libpython3.10.so.1.0 – Python runtime
# libc.so.6 – GNU C Library (glibc)
# /lib64/ld-linux-x86-64.so.2 – Dynamic linker/loader

# === ELF sections ===
readelf -S hello.o                   # List sections (.text,
.data, .bss, .rodata)
readelf -h hello.o                   # ELF header (architecture,
entry point, flags)
readelf -l hello.o                   # Program headers (segments
for loading)
# .text – executable code
# .data – initialized global/static variables

```

```
# .bss - uninitialized global variables (zero-initialized)
# .rodata - read-only data (string literals, const)
# .symtab - symbol table

# === Strip debugging symbols ===
strip hello                    # Remove symbols (smaller
binary, harder to debug)
strip --strip-debug hello      # Keep global symbols, remove
debug info
# FAANG production: binaries are stripped, debug symbols are
archived separately
```

Shared Libraries: SONAME, RPATH & LD_LIBRARY_PATH

Shared libraries (.so files) are loaded at runtime by the dynamic linker (`ld-linux.so`). **The linker searches: RPATH (hardcoded in binary) → LD_LIBRARY_PATH → /etc/ld.so.cache → /lib, /usr/lib.** Mismatched library versions cause the infamous 'symbol not found' errors. At FAANG, RPATH is preferred over LD_LIBRARY_PATH for production — it's baked into the binary and can't be accidentally overridden.

```
# === SONAME (Shared Object Name) ===
# Libraries have three names:
# 1. Real name: libcrypto.so.3.0.0 (full version)
# 2. SONAME: libcrypto.so.3 (compatible ABI version)
# 3. Linker name: libcrypto.so (no version, for -lcrypto flag)

# Check SONAME:
objdump -p /usr/lib/x86_64-linux-gnu/libcrypto.so.3 | grep SONAME
# SONAME is embedded in the library file
# ldconfig creates symlinks from SONAME to real name

# === RPATH vs RUNPATH ===
```

```

# RPATH: search path embedded in binary, searched BEFORE
LD_LIBRARY_PATH
# RUNPATH: similar but searched AFTER LD_LIBRARY_PATH
# Prefer RUNPATH (allows overriding with LD_LIBRARY_PATH for
debugging)

gcc -Wl,-rpath,'$ORIGIN/../lib' -o myapp myapp.o
# $ORIGIN expands to the directory containing the executable
# This is how portable app bundles work (e.g., AppImage)

# Check RPATH:
readelf -d myapp | grep -E "(RPATH|RUNPATH)"

# === LD_LIBRARY_PATH (use for development, not production) ===
export LD_LIBRARY_PATH=/opt/custom/lib:$LD_LIBRARY_PATH
./myapp                                # Searches custom path
first

# === ldconfig - update library cache ===
sudo ldconfig                          # Rebuild /etc/ld.so.cache
sudo ldconfig -p | grep nvda           # Search cached libraries by
name

# === ld.so.conf - system-wide library paths ===
cat /etc/ld.so.conf                    # Include
/etc/ld.so.conf.d/*.conf
ls /etc/ld.so.conf.d/
# Each .conf file adds a search path

# === Library versioning best practices ===
# libfoo.so.1 -> libfoo.so.1.0.0 (major.minor.patch)
# Major version change = ABI break
# Minor/patch = backward-compatible
# Never replace a library with incompatible ABI without changing
SONAME

# === Static vs shared libraries ===
# Static (.a): linked into executable, larger binary, no runtime
deps
# Shared (.so): loaded at runtime, smaller binary, memory sharing

```

```
# FAANG production: mostly shared (container images share library
pages)
# For small utilities: static (Go is fully static by default)
```

Build Systems: Make, CMake, Bazel & Hermetic Builds

At FAANG, build systems are **hermetic** — isolated from the developer's machine, producing identical outputs from identical inputs. **Bazel** (Google), **Buck** (Meta), and **Pants** (Twitter) enforce dependency tracking, remote caching, and incremental builds. Understanding build system concepts — dependency graph, caching, sandboxing, remote execution — is essential for infrastructure engineers.

```
# === Make ===
# Simple but limited: no dependency tracking, timestamp-based
# Every FAANG engineer should be able to read a Makefile

# === CMake ===
# Generates build files for different systems (Make, Ninja, Xcode,
VS)
# CMakeLists.txt example:
# cmake_minimum_required(VERSION 3.20)
# project(myapp CXX)
# add_executable(myapp main.cpp util.cpp)
# target_link_libraries(myapp PRIVATE fmt)

cmake -B build -G Ninja .          # Configure (generate Ninja
build files)
cmake --build build                # Build

# === Bazel (hermetic build system – Google's standard) ===
# BUILD file:
# cc_binary(
```

```
#   name = "myapp",
#   srcs = ["main.cc", "util.cc"],
#   deps = [":mylib", "@fmt//:fmt"],
# )

bazel build //:myapp           # Build (hermetic, cached,
                                distributed)
bazel test //:tests           # Run tests
bazel query 'deps//:myapp)'    # Show dependency graph

# Hermetic build principles:
# 1. All inputs declared (source, tools, environment)
# 2. No network access during build
# 3. No access to /usr, /opt, etc.
# 4. Outputs depend only on inputs (no timestamps, no hostname)
# 5. Remote caching: rebuild is instant if inputs haven't changed

# === Ninja (fast incremental builds) ===
ninja -C build                 # Faster than make (parallel
                                by default)

# === ccache (compiler cache) ===
export CCACHE_DIR=~/.ccache
export PATH=/usr/lib/ccache:$PATH # Transparent compiler caching
# Second build of same file: instant (cache hit)
```

LTO, PGO & Binary Optimization

Modern compilers offer profile-guided and link-time optimizations that can improve performance by 10-30%. **LTO (Link-Time Optimization)** enables cross-module inlining and optimization. **PGO (Profile-Guided Optimization)** uses runtime profiling data to guide optimization decisions. At FAANG, PGO is standard for critical services — the performance gain in compute-heavy workloads justifies the build complexity.

```

# === LTO (Link-Time Optimization) ===
# Traditional: compile each .c file separately, then link
# LTO: compile to intermediate representation (IR), optimize at
link time

gcc -flto -O2 -o myapp file1.c file2.c # LTO enabled
# Benefits: cross-module inlining, devirtualization, dead code
elimination
# Cost: slower linking (~2x), more memory during link

# === PGO (Profile-Guided Optimization) ===
# Step 1: Compile with instrumentation
gcc -fprofile-generate -O2 -o myapp.prof myapp.c

# Step 2: Run with representative workload
./myapp.prof < representative_input > /dev/null
# Generates .gcda files with profiling data

# Step 3: Recompile with profile data
gcc -fprofile-use -O2 -o myapp myapp.c
# Now: branch prediction, inlining, and loop optimization use real
profiling data

# AutoFDO (Linux 5.x+): use perf data instead of instrumented
binary
# perf record -b ./myapp # Branch-recorded profiling
# create_gcov --profile=perf.data --gcov=myapp.gcov ./myapp
# gcc -fprofile-use=myapp.gcov -O2 -o myapp myapp.c

# === Size optimization ===
gcc -Os -o small_binary myapp.c # Optimize for size
(embedded, containers)
# Compare: -O2 (speed) vs -Os (size) – typical difference is 30-50%

# === Debug symbols ===
gcc -g -o myapp.debug myapp.c # Full debug symbols
gcc -g1 -o myapp.debug myapp.c # Minimal debug symbols
(line numbers only)
objcopy --only-keep-debug myapp.debug myapp.dbg # Extract debug
info
objcopy --strip-debug myapp.debug myapp # Strip for production

```

```
# FAANG pattern: separate debug symbols (myapp.dbg) uploaded to
artifact store

# === Binary size analysis ===
size myapp                # text, data, bss, total
bloaty myapp              # Symbol-level size
breakdown (install: apt
install bloaty)
nm myapp | sort -k2,2 | uniq -c | sort -rn | head # Largest symbol
categories
```

FAANG Interview Q&A

Q: 'Your program compiles fine on your machine but fails with 'cannot open shared object file' on the server. Why?'

A: 'Missing shared library or wrong search path. Check with `ldd myapp` to find the missing library. Common fixes: (1) Install the library on the server (`apt install`). (2) Set `LD_LIBRARY_PATH`. (3) Link with `RPATH/RUNPATH` embedded in the binary. (4) The library version on the server is too old — check `SONAME` compatibility. At FAANG, this is solved by containers — the binary and all its shared libraries are in the container image, guaranteed to match.'

Q: 'What's the difference between dynamic and static linking, and when would you use each?'

A: 'Dynamic linking loads libraries at runtime — smaller binaries, memory sharing between processes, and library updates without recompiling. Downside: dependency hell (wrong library version = crash). Static linking embeds everything into the binary — no

runtime dependencies, fully portable, but larger binaries and no library updates without rebuilding. At FAANG: most services use dynamic linking within containers (container = static deployment unit, libraries inside are fixed). Go uses fully static linking by default. For C/C++, static linking is preferred for small command-line tools and for Alpine-based containers (musl libc vs glibc compatibility issues).'

Q: 'How would you set up a build system for a monorepo with 1000 microservices?'

A: 'Use a hermetic build system like Bazel. Key requirements: (1) Explicit dependency declarations — every build target declares its deps. (2) Remote caching — builds reuse cached outputs from CI, not just local. (3) Remote execution — distribute build across hundreds of machines. (4) Incremental builds — changing one service rebuilds only that service and its dependents. (5) Sandboxed builds — no access to /usr, no network during build. (6) Multi-language support — Java, C++, Python, Go, Rust all in one repo. (7) CI integration — build and test only what changed (affected target detection). At FAANG, this scale of build system is a dedicated team's responsibility — the build infrastructure is as critical as the production infrastructure.'

References & Further Reading

- **Linkers & Loaders** — John Levine, the definitive reference on linking
- **Bazel Build System** — Google's build system documentation

- **GCC Optimization Options** — Official GCC optimization documentation

FAANG Architect's Note: The build toolchain is one of the most underappreciated areas of systems knowledge. I've seen senior engineers stumble on basic linker errors, not understand why 'it works on my machine', or ship binaries without LTO/PGO that could have been 20% faster. At FAANG, the build system is a force multiplier: a well-optimized build pipeline means faster iteration cycles, smaller deployment artifacts, and better runtime performance. Every hour spent understanding the toolchain pays back tenfold in reduced debugging time and improved efficiency.

Production Tip: Always check `ldd` on your binary before deploying. A 'not found' library in production causes immediate crash on startup — one of the easiest bugs to catch in CI but one of the most common. Also: set `-Wl,--as-needed` in your `LDFLAGS`. This removes unnecessary library dependencies, reducing startup time and preventing ABI conflicts from unused libraries. In containers, always run `ldd /app/myapp | grep 'not found'` as a build step — if it passes, the container will start.

PART 4

Containers & SRE

Container internals (cgroups & namespaces), Docker, Kubernetes for Linux engineers, production troubleshooting, system recovery, security hardening, FAANG-scale operations, and interview preparation.

CHAPTER 23

cgroups & Namespaces

Containers & SRE · 7 topics

Control groups (cgroups) and namespaces are the two Linux kernel features that make containers possible. **cgroups limit resource usage (CPU, memory, I/O, PID count); namespaces isolate process views (filesystem, network, PID, user, mount).** At FAANG, every process runs inside cgroups — from individual containers to batch jobs to user shells in shared environments.

FAANG

Linux

cgroups

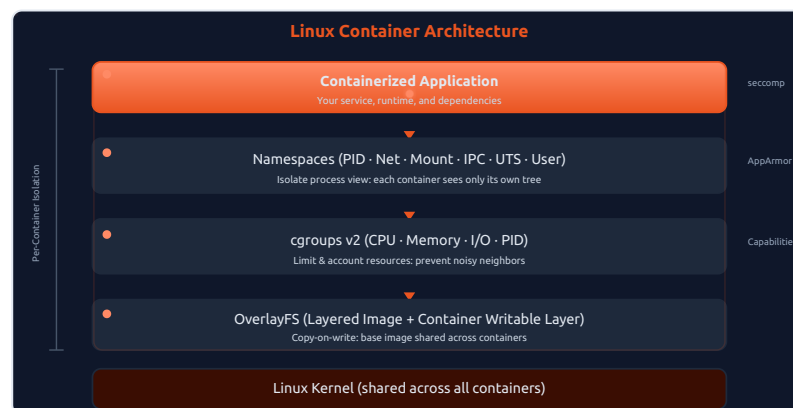
namespaces

containers

Docker

Kubernetes

Namespaces: The Illusion of Isolation



Namespaces give each process its own view of the system. **A process in a mount namespace sees only its own filesystem. A process in a network namespace sees only its own network interfaces. A process in a PID namespace sees only its own process tree.** There are 8 namespace types. Docker creates a new set of namespaces for each container.

```
# === Namespace types ===
# Mount (mnt)      - Isolate filesystem mount points
# PID (pid)        - Isolate process IDs
# Network (net)    - Isolate network interfaces, routes, iptables
# IPC (ipc)        - Isolate System V IPC and POSIX message queues
# UTS (uts)        - Isolate hostname and domain name
# User (user)      - Isolate UID and GID mappings
# Cgroup (cgroup) - Isolate cgroup root
# Time (time)      - Isolate system time (Linux 5.6+)

# === Check namespaces of a process ===
ls -l /proc/1/ns/          # Namespace inodes for PID 1
# lrwxrwxrwx 1 root root 0 ... mnt -> mnt:[4026531840]
# lrwxrwxrwx 1 root root 0 ... net -> net:[4026531840]
# Same inode numbers = same namespace

# === Compare namespaces ===
ls -l /proc/<PID1>/ns/net /proc/<PID2>/ns/net # Same? Same network
namespace

# === Create a new namespace (unshare) ===
# New mount namespace (isolated filesystem):
sudo unshare --mount --fork /bin/bash
# Inside: mount is isolated - changes don't affect host

# New PID namespace:
sudo unshare --pid --fork --mount-proc /bin/bash
# Inside: only sees processes in this namespace

# New network namespace:
sudo ip netns add myns          # Create network namespace
sudo ip netns exec myns ip addr # Run command inside namespace
```

```

sudo ip netns delete myns          # Delete namespace

# === Network namespace use: per-process networking ===
# Create veth pair to connect namespaces:
sudo ip link add veth0 type veth peer name veth1
sudo ip link set veth1 netns myns
sudo ip addr add 10.0.0.1/24 dev veth0
sudo ip netns exec myns ip addr add 10.0.0.2/24 dev veth1
sudo ip link set veth0 up
sudo ip netns exec myns ip link set veth1 up
# Now traffic flows between host and namespace

# === List all network namespaces ===
sudo ip netns list
# Each Docker container gets its own network namespace

# === Enter a container's namespace ===
# Get container PID:
docker inspect <container> | grep Pid
# Enter namespaces:
sudo nsenter -t <PID> -n /bin/bash  # Enter network namespace
sudo nsenter -t <PID> -p /bin/bash  # Enter PID namespace
sudo nsenter -t <PID> -a /bin/bash  # Enter ALL namespaces

```

cgroups v2: Resource Accounting & Limiting

cgroups v2 (unified hierarchy) is the modern cgroup implementation. **Every process belongs to a cgroup, and the kernel enforces resource limits at the cgroup level.** At FAANG, cgroups v2 is the standard on modern Linux distros (Ubuntu 22.04+, Amazon Linux 2023, RHEL 9+). cgroups v1 is legacy but still present for backward compatibility.

```

# === cgroups v2 hierarchy ===
# Mounted at /sys/fs/cgroup/
ls /sys/fs/cgroup/
# cpu.max, cpu.stat          - CPU limits

```

```

# memory.max, memory.current – Memory limits
# io.max, io.stat           – I/O limits
# pids.max                  – PID limit

# === Create and configure a cgroup ===
# On systemd-based systems, use systemd:
sudo systemd-run --unit=testapp --scope -p CPUQuota=50% -p
MemoryMax=256M sleep 300

# Or direct cgroup manipulation:
sudo mkdir /sys/fs/cgroup/myapp

# CPU limit: 50% of one core (50,000/100,000 µs)
echo "50000 100000" | sudo tee /sys/fs/cgroup/myapp/cpu.max

# Memory limit: 256 MB
echo "268435456" | sudo tee /sys/fs/cgroup/myapp/memory.max

# Memory + swap limit: 512 MB
echo "536870912" | sudo tee /sys/fs/cgroup/myapp/memory.swap.max

# PID limit: 100
echo "100" | sudo tee /sys/fs/cgroup/myapp/pids.max

# I/O limit: 10MB/s write
echo "8:0 wbps=10485760" | sudo tee /sys/fs/cgroup/myapp/io.max
# 8:0 = major:minor of the block device

# === Add process to cgroup ===
echo $PID | sudo tee /sys/fs/cgroup/myapp/cgroup.procs

# === Check cgroup stats ===
cat /sys/fs/cgroup/myapp/cpu.stat
# usage_usec – CPU time used
# nr_periods – number of enforcement periods
# nr_throttled – periods where tasks were throttled
# throttled_usec – total throttled time

cat /sys/fs/cgroup/myapp/memory.current
# Current memory usage in bytes

cat /sys/fs/cgroup/myapp/memory.events

```

```
# low, high, max, oom, oom_kill – event counters
# If 'max > 0', the cgroup hit its memory limit
# If 'oom_kill > 0', the OOM killer was triggered

# === Pressure Stall Information (PSI) ===
cat /sys/fs/cgroup/cpu.pressure      # CPU pressure
cat /sys/fs/cgroup/memory.pressure   # Memory pressure
cat /sys/fs/cgroup/io.pressure       # I/O pressure
# Format: some avg10=0.00 avg60=0.00 avg300=0.00 total=0
# 'some' – % of time at least one task was stalled
# 'full' – % of time all tasks were stalled
# PSI is the key metric for resource contention at FAANG
```

Docker & Containers: cgroups + Namespaces in Practice

Docker combines cgroups (for resource limits) and namespaces (for isolation). **When you run 'docker run -m 256M --cpus=0.5 nginx', Docker creates a new cgroup in /sys/fs/cgroup/system.slice/docker-*scope* and configures the limits. Understanding this mapping is essential for debugging container resource issues.**

```
# === Docker cgroup hierarchy ===
# Docker containers live under:
/sys/fs/cgroup/system.slice/docker-<container-id>.scope/

# Inside a container:
cat /proc/1/cgroup                # Shows container's cgroup
path
# 0::/system.slice/docker-abc123.scope

# From host, inspect container cgroup:
CONTAINER_ID=$(docker ps -q --no-trunc | head -1)
cat
/sys/fs/cgroup/system.slice/docker-$CONTAINER_ID.scope/memory.curre
```

```

nt

# === Docker resource limits vs cgroup ===
docker run --memory=256M --cpus=0.5 --pids-limit=100 nginx
# Maps to:
# memory.max = 268435456
# cpu.max = 50000 100000
# pids.max = 100

# === Common container resource issues ===
# 1. Container sees host memory (cat /proc/meminfo shows host, not
cgroup)
#   Fix: Use cgroup-aware tools or read
/sys/fs/cgroup/memory.current
#
# 2. Java/Golang apps ignore cgroup limits
#   Fix: Use JDK 10+ (cgroup-aware), Go 1.19+ (GOMEMLIMIT)
#
# 3. OOM inside container doesn't show in host dmesg
#   Fix: Check memory.events inside container cgroup

# === cgroups v2 in Kubernetes ===
# Kubernetes 1.24+ uses cgroups v2 by default
# kubelet creates cgroups under /sys/fs/cgroup/kubepods/
ls /sys/fs/cgroup/kubepods/           # All pod cgroups
ls /sys/fs/cgroup/kubepods/burstable/ # Best-effort pods
ls /sys/fs/cgroup/kubepods/guaranteed/ # Guaranteed QoS pods

# === systemd resource control ===
# systemd services run as scopes under:
/sys/fs/cgroup/system.slice/<service-name>.service/

# Example: nginx service cgroup
cat /sys/fs/cgroup/system.slice/nginx.service/cpu.max
cat /sys/fs/cgroup/system.slice/nginx.service/memory.max

```

Debugging Resource Issues with cgroups

When a container or service is hitting resource limits, cgroup statistics tell you exactly what's happening. **At FAANG, cgroup-level metrics are the primary source of truth for resource usage — more reliable than /proc/meminfo (which shows host-level data).**

```
# === Is the process being throttled? ===
cat /sys/fs/cgroup/system.slice/myapp.service/cpu.stat
# nr_throttled > 0 = CPU throttled (hitting CPU limit)
# throttled_usec = total time spent throttled

# === Is the process hitting memory limit? ===
cat /sys/fs/cgroup/system.slice/myapp.service/memory.events
# low: 0          - hitting memory.low threshold
# high: 0         - hitting memory.high (reclaiming aggressively)
# max: 5          - hit memory.max (OOM kill threshold!)
# oom: 5          - OOM events
# oom_kill: 5     - processes killed by OOM

# === Current memory breakdown ===
cat /sys/fs/cgroup/system.slice/myapp.service/memory.stat
# anon - anonymous (heap, stack)
# file - file-backed (page cache)
# kernel - kernel memory (slab, page tables)

# === Is the process hitting PID limit? ===
cat /sys/fs/cgroup/system.slice/myapp.service/pids.current
cat /sys/fs/cgroup/system.slice/myapp.service/pids.max
# If pids.current == pids.max, the process is hitting PID limits
# Common for: forking servers, thread leaks

# === Is I/O being throttled? ===
cat /sys/fs/cgroup/system.slice/myapp.service/io.stat
# rbytes, wbytes - bytes read/written
# rios, wios - I/O operations

# === PSI (Pressure Stall Info) ===
cat /sys/fs/cgroup/system.slice/myapp.service/memory.pressure
# some avg10=5.00 - 5% of time at least one task was stalled on
```

```
memory
# If avg10 > 10%, you're under memory pressure

# === One-liner: Check all limits for a container ===
CONTAINER_CGROUP=$(docker inspect --format '{{.Id}}' $CONTAINER |
cut -c1-12)
CGROUP_PATH="/sys/fs/cgroup/system.slice/docker-$CONTAINER_CGROUP.s
cope"
echo "=== CPU ==="
cat $CGROUP_PATH/cpu.max
echo "=== Memory ==="
cat $CGROUP_PATH/memory.current
cat $CGROUP_PATH/memory.events
echo "=== PIDs ==="
cat $CGROUP_PATH/pids.current
cat $CGROUP_PATH/pids.max
```

FAANG Interview Q&A

Q: 'Explain how Docker containers work under the hood.'

A: 'Docker containers use Linux namespaces for isolation and cgroups for resource limits. When Docker starts a container: (1) It creates new namespaces (mount, PID, network, IPC, UTS, user) via the clone() syscall. (2) It sets up the container's filesystem using a layered image (overlayfs) and chroots into it (pivot_root). (3) It creates cgroups under /sys/fs/cgroup/system.slice/ and sets CPU, memory, I/O, and PID limits. (4) It sets up the network (veth pair → bridge or overlay). (5) It applies seccomp and LSMA (AppArmor/SELinux) profiles. (6) It execs the container's CMD inside these namespaces. The container doesn't 'contain' anything — it's just a regular Linux process that can't see anything outside its namespaces and can't use more resources than its cgroup

allows.'

Q: 'A container is running out of memory, but `free -h` inside the container shows plenty of free memory. Why?'

A: 'Because `free` and `/proc/meminfo` inside a container show host-level memory, not the cgroup limit — unless the container runtime correctly mounts the cgroup filesystem. On older Docker/Cgroup v1 setups, `/proc/meminfo` is not virtualized. The container is hitting the cgroup memory limit even though `/proc/meminfo` says there's free memory. Always check `/sys/fs/cgroup/memory.current` and `/sys/fs/cgroup/memory.events` inside the container for the real story. Modern runtimes (containerd, cgroups v2) handle this better, but it's still a common pitfall.'

Q: 'How do you troubleshoot a Node.js container that's hitting CPU throttling?'

A: 'Check `cat /sys/fs/cgroup/cpu.max` to see the CPU limit. Check `cat /sys/fs/cgroup/cpu.stat` for `nr_throttled > 0`. If throttled, either increase the CPU limit or optimize the application. Use `perf top` inside the container to identify hot functions. Check if the event loop is blocked — Node.js runs on a single thread, so CPU throttling directly impacts throughput. If the container is a sidecar or batch job, consider that its burstable CPU might be competing with other containers on the same pod/node. At FAANG, we use cgroup-aware CPU metrics for autoscaling decisions — not host CPU metrics.'

References & Further Reading

- **cgroups v2 Documentation** — Kernel.org official documentation
- **namespaces(7) man page** — Complete namespace reference
- **Container Architectures (InfoQ)** — Jérôme Petazzoni's deep dive on container internals

FAANG Architect's Note: cgroups and namespaces are the most important Linux features you've probably never directly used. If you work with containers (and everyone at FAANG does), you depend on these features every day. Understanding them at the kernel level transforms how you debug container issues: you stop thinking of a container as a 'lightweight VM' and start thinking of it as 'a process with namespaces and cgroups.' This distinction matters when you're debugging a memory issue (check the cgroup, not `/proc/meminfo`), a network issue (check the namespace), or a performance issue (check the cgroup throttling).

Production Tip: Always set memory limits on containers and services. An unlimited container can destabilize the entire host — one container's memory leak can OOM-kill all containers on the node. Set memory limits to ~80-90% of what you expect the process to need, and set up monitoring on cgroup `memory.events` for the 'max' counter. If 'max > 0', the container is hitting its limit and you need to either increase the limit or find the leak. At FAANG, we never run containers without resource limits — ever.

CHAPTER 24

Docker & Container Lifecycle

Containers & SRE · 6 topics

Docker is the most widely used container runtime, built on Linux cgroups and namespaces. At FAANG, Docker (or its OCI-compatible alternatives like containerd) is the standard unit of deployment. Understanding the container lifecycle — from building to running to debugging — is essential for any infrastructure engineer.

[FAANG](#)[Linux](#)[Docker](#)[containers](#)[OCI](#)[container lifecycle](#)

Container Images: Layers & Build Caching

A Docker image is a stack of read-only layers, each representing a change to the filesystem. **Layers are cached and shared between images — this is why pulling updates is fast (only new layers are downloaded).** At FAANG, image layer optimization is critical for deployment speed: minimizing layers, caching dependencies, and using distroless base images reduce both build time and attack surface.

```
# === Dockerfile best practices ===  
  
# 1. Use specific base image versions (not 'latest')  
FROM ubuntu:22.04@sha256:abc123...  
  
# 2. Combine RUN commands to minimize layers  
RUN apt-get update && apt-get install -y \\  

```

```

python3 python3-pip \\\
&& rm -rf /var/lib/apt/lists/*

# 3. Use .dockerignore (like .gitignore for builds)
# .dockerignore:
# node_modules
# .git
# *.log

# 4. Multi-stage builds (separate build deps from runtime)
FROM node:22 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production

FROM node:22-alpine AS runtime
COPY --from=builder /app /app
USER node
CMD ["node", "server.js"]

# 5. Use distroless or alpine for smaller images
# Distroless: gcr.io/distroless/nodejs22-debian12
# Alpine: node:22-alpine (much smaller, but uses musl libc)

# 6. Don't run as root inside the container
RUN addgroup -S myapp && adduser -S myapp -G myapp
USER myapp

# === Build optimization ===
# Cache layers that change infrequently (package.json → deps first,
# then source)
COPY package*.json ./
RUN npm ci                # This layer only rebuilds when package.json
                           # changes
COPY . .                  # This layer rebuilds every time (frequent
                           # changes)

# === Layer inspection ===
docker history nginx      # Show layer sizes and commands
docker inspect nginx      # Detailed image metadata
(layers, env, ports)

```

```
# === Image size commands ===
docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}"
docker system df                # Total disk usage of Docker
```

Container Runtime & Lifecycle

When Docker runs a container, it goes through: **pull image** → **create container** → **create namespaces** → **set up cgroups** → **mount rootfs** → **apply seccomp** → **exec CMD**. Understanding this sequence helps you debug startup failures and performance issues. At FAANG, the container runtime is usually containerd (the industry-standard OCI runtime) with runc as the low-level runtime.

```
# === Container lifecycle commands ===
# Build:
docker build -t myapp:latest .      # Build image from Dockerfile
docker build --no-cache -t myapp .  # Force rebuild (ignore cache)

# Run:
docker run -d --name myapp -p 8080:80 myapp:latest
docker run -it --rm ubuntu:22.04 bash # Interactive, auto-remove

# Stop/start:
docker stop myapp                # SIGTERM + wait + SIGKILL
docker start myapp                # Restart stopped container
docker restart myapp             # Stop + start
docker kill myapp                # SIGKILL immediately

# Inspect:
docker ps                        # Running containers
docker ps -a                    # All containers (including
                                # stopped)
docker logs -f myapp            # Follow logs
docker logs --tail 100 myapp    # Last 100 lines
docker inspect myapp            # Full container metadata
```

(JSON)

```
# Execute in running container:
docker exec -it myapp bash          # Start shell in container
docker exec myapp ps aux            # Run command in container

# === Resource limits ===
docker run -d --name db \
  --memory=512M \
  --memory-reservation=256M \
  --cpus=0.5 \
  --pids-limit=200 \
  --ulimit nofile=65536:65536 \
  postgres:16

# === Health checks ===
# In Dockerfile:
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD curl -f http://localhost:8080/health || exit 1

# Manual check:
docker inspect --format='{{.State.Health.Status}}' myapp

# === Copy files ===
docker cp myapp:/var/log/app.log ./local.log
docker cp ./config.yaml myapp:/etc/app/config.yaml

# === Cleanup ===
docker rm myapp                    # Remove container
docker rmi myapp:latest            # Remove image
docker system prune -a            # Remove ALL unused
containers, images, networks
```

Networking & Storage in Docker

Docker provides several network drivers (**bridge** for single-host, **overlay** for multi-host, **host** for performance, **macvlan** for direct MAC assignment) and storage options (**volumes** for persistent

data, **bind mounts** for host directories, **tmpfs** for temporary memory-backed storage). At FAANG, overlay networks (Cilium, Flannel, Calico) handle container networking, and cloud block storage (EBS volumes) is mounted via bind mounts.

```
# === Network drivers ===
# Bridge (default): containers connect via docker0 bridge
docker network create mynet          # Create custom bridge network
docker run --network=mynet nginx

# Host: container uses host's network stack (no isolation, best
performance)
docker run --network=host nginx

# Overlay: multi-host networking (Docker Swarm or Kubernetes)
docker network create -d overlay myoverlay

# === Port mapping ===
docker run -p 8080:80 nginx          # host:8080 → container:80

# === DNS within Docker ===
# Containers on same user-defined bridge can resolve each other by
name
# docker run --name web --network=mynet nginx
# docker run --network=mynet alpine ping web # Works!

# === Volumes ===
docker volume create mydata          # Create volume
docker run -v mydata:/var/lib/data postgres # Mount volume

# Bind mount:
docker run -v /host/path:/container/path nginx

# tmpfs (RAM storage):
docker run --tmpfs /tmp:size=100M nginx

# === Backup volume ===
docker run --rm -v mydata:/source -v $(pwd):/backup alpine tar czf
/backup/mydata.tar.gz
```

```
-C /source .

# === Storage drivers ===
# overlay2 (default, modern) – Copy-on-Write, fast
# aufs (legacy) – Older Ubuntu default
# devicemapper – RHEL legacy (slow, avoid)
docker info | grep Storage

# === Container resource inspection ===
docker stats                                # Live resource usage
# CPU%, MEM%, MEM USAGE/LIMIT, NET I/O, BLOCK I/O, PIDS

docker stats --no-stream --format "table
{{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"
```

Debugging Containers

Containers introduce debugging challenges: you're running in an isolated environment with limited tools. **At FAANG, debugging containers requires understanding both the application inside and the host outside.** Key tools: `docker exec` for inside, `nsenter` for entering namespaces from host, and `cgroup` inspections for resource issues.

```
# === Inside-container debugging ===
docker exec -it myapp bash                # Shell inside container

# Install debugging tools (temporary container):
docker run --rm -it --net=container:myapp --pid=container:myapp \
  nicolaka/netshoot:latest                # Network troubleshooting
  toolbox

# === Host-level debugging ===
# Find container PID:
CONTAINER_PID=$(docker inspect --format '{{.State.Pid}}' myapp)
echo $CONTAINER_PID
```

```
# Enter container namespaces from host:
sudo nsenter -t $CONTAINER_PID -n ip addr # Network namespace
sudo nsenter -t $CONTAINER_PID -p ps aux  # PID namespace
sudo nsenter -t $CONTAINER_PID -a bash    # All namespaces

# Check container cgroup:
CGROUP_PATH=$(cat /proc/$CONTAINER_PID/cgroup | grep -oP
'/system.slice/docker-.*\.scope')
cat /sys/fs/cgroup$CGROUP_PATH/memory.events

# === Container logs ===
docker logs myapp # All logs
docker logs --tail 50 --follow --timestamps myapp

# === Check container filesystem ===
docker diff myapp # Show changed files
(A=added, C=changed, D=deleted)

# === Export container filesystem ===
docker export myapp > container.tar # Export as tar archive
docker import container.tar # Import as image

# === Debug entrypoint/cmd ===
# Override entrypoint to debug:
docker run -it --entrypoint sh myapp:latest

# === Common issues ===
# 1. Container exits immediately: check CMD/ENTRYPOINT and logs
# 2. Port already in use: change host port mapping
# 3. Out of disk space: docker system prune -a
# 4. Permission denied inside: check USER directive and volume
permissions
# 5. DNS resolution fails: check --dns or /etc/docker/daemon.json
```

FAANG Interview Q&A

Q: 'Your containerized application is running slow in production. How do you debug?'

A: 'First, rule out resource limits. Check `docker stats` or the cgroup metrics: `cat /sys/fs/cgroup/cpu.stat` (throttling), `memory.events` (OOM), `io.stat` (I/O throttling). If the container is hitting limits, increase them or optimize the app. Next, check the host: `top` for CPU, `iostat` for disk, `free -h` for memory. If the host is fine, debug inside the container: `docker exec -it myapp bash`, then use standard Linux tools (`top`, `strace`, `lsof`). If the container lacks tools, use `nsenter` from the host. At FAANG, we use ephemeral debug containers — a sidecar with network and PID namespace sharing that contains all debugging tools, without modifying the production container.'

Q: 'How would you design a container image for production?'

A: 'Production container image requirements: (1) Use a specific base image version, pinned by digest, not tag. (2) Multi-stage build: builder stage with build tools, runtime stage with only the binary and minimal dependencies. (3) Non-root user — never run as root. (4) HEALTHCHECK instruction. (5) No secrets in the image — use Docker build secrets or runtime secrets injection. (6) Minimal size: use distroless or Alpine base, combined RUN layers, clean package manager cache. (7) Reproducible: exact same image for the same git commit. (8) Signed: use Docker Content Trust or cosign for image signing. At FAANG, image compliance is enforced by admission controllers in the container registry — images without a HEALTHCHECK, running as root, or with known vulnerabilities are rejected.'

Q: 'What's the difference between CMD and ENTRYPOINT in a

Dockerfile?'

A: 'ENTRYPOINT defines the executable that always runs. CMD provides default arguments to ENTRYPOINT. `ENTRYPOINT ["nginx"]` + `CMD ["-g", "daemon off;"]` = runs `nginx -g 'daemon off;'`. If you override CMD at runtime (`docker run nginx -t`), it becomes `nginx -t` (CMD replaced). If you override ENTRYPOINT (`docker run --entrypoint bash nginx`), it runs bash instead of nginx. Best practice: use ENTRYPOINT for the fixed binary and CMD for the default arguments — this makes the container behavior predictable while allowing argument customization.'

References & Further Reading

- **Dockerfile Best Practices** — Official Docker documentation
- **Docker Compose** — Multi-container orchestration (development & testing)
- **containerd documentation** — Industry-standard container runtime used in Kubernetes

FAANG Architect's Note: Docker is the 'hello world' of containers — most FAANG environments use containerd or CRI-compatible runtimes directly, not the Docker CLI. But the concepts are identical: Docker taught the industry how to think about containers. The Dockerfile format, image layering, and the run/build model are universal. Learn Docker first; the transition to containerd/nerdctl/buildkit is straightforward because the underlying principles are the same.

Production Tip: Never run 'docker build' on a production server. Build images in your CI/CD pipeline, push them to a registry, and pull them on production nodes. This ensures that: (1) builds are reproducible (same commit = same image), (2) build tools aren't on production servers, (3) images are scanned for vulnerabilities before deployment, (4) rollbacks are instant (just point to the previous image tag). At FAANG, image building and deployment are entirely separate stages — building happens in a hermetic build environment, deployment pulls from a registry.

CHAPTER 25

Kubernetes for Linux Engineers

Containers & SRE · 9 topics

Kubernetes is a Linux cluster operating system. Every Kubernetes node runs Linux, and every container is a Linux process. Understanding Kubernetes from the Linux perspective — kubelet, CRI, CNI, CSI, cgroups, namespaces, eBPF — transforms how you debug cluster issues and design resilient systems. At FAANG, this Linux-first understanding is what separates infrastructure engineers who 'manage YAML' from those who 'operate clusters'.

FAANG Linux Kubernetes kubelet CRI CNI CSI
containers eBPF

Kubelet: The Node Agent

The **kubelet** is the primary node agent — it registers the node with the control plane, watches for pod assignments, and ensures containers are running. It interacts with the container runtime via **CRI (Container Runtime Interface)**. At FAANG, the kubelet is configured with custom flags for node-level resource reservation, pod eviction thresholds, and garbage collection — all critical for cluster stability.

```
# === Kubelet on the node ===  
ps aux | grep kubelet          # Check kubelet process and  
flags
```

```

# Key kubelet flags:
# --node-ip=10.0.0.5                # Node's internal IP
# --container-runtime-
endpoint=unix:///run/containerd/containerd.sock
# --kubeconfig=/var/lib/kubelet/kubeconfig
# --register-node=true
# --node-status-update-frequency=10s
# --eviction-hard=memory.available<100Mi,nodefs.available<10%
# --system-reserved=cpu=200m,memory=512Mi # Resources reserved for
system
# --kube-reserved=cpu=200m,memory=512Mi   # Resources reserved for
kubelet itself

# === Kubelet logs ===
journalctl -u kubelet -n 100 --no-pager # Recent kubelet logs
journalctl -u kubelet -f                # Follow kubelet logs

# === Kubelet state directory ===
ls /var/lib/kubelet/
# pod-manifests/      - Static pod YAML files
# device-plugins/     - GPU/device plugin sockets
# pods/               - Pod sandbox directories (one per pod)
# cpu_manager_state   - CPU manager state (static CPU pinning)

# === Pod sandbox on disk ===
ls /var/lib/kubelet/pods/<POD_UID>/
# containers/         - container logs and state
# volumes/            - mounted volumes
# etc-hosts           - pod-level /etc/hosts

# === Check node capacity ===
kubectl describe node <node> | grep -A 10 "Capacity"
# cpu, memory, pods, ephemeral-storage, hugepages
# Also: nvidia.com/gpu, nvidia.com/mig-1g.10gb (device plugin
resources)

# === Kubelet garbage collection ===
# Image GC: remove unused images when disk > threshold
# Container GC: remove terminated containers
# Eviction: kill pods when node resources are low
# Kubelet flags:

```

```
# --image-gc-high-threshold=85      # Start GC when disk > 85%
# --image-gc-low-threshold=80       # Stop GC when disk < 80%
# --maximum-dead-containers-per-container=5
```

CRI: Container Runtime Interface

The CRI abstracts the container runtime behind a gRPC API.

containerd is the default runtime (used by Docker, EKS, GKE, AKS). **cri-o** is a Kubernetes-native runtime optimized for CRI. **runc** is the low-level OCI runtime. At FAANG, containerd is standard — but some workloads use alternative runtimes for security (gVisor, Kata Containers) or performance (crun in Rust).

```
# === Check container runtime ===
kubectl get nodes -o wide | awk '{print $1, $NF}'
# Shows container-runtime version for each node

# === containerd (most common) ===
# containerd socket:
ls /run/containerd/containerd.sock

# containerd commands:
ctr images list           # List images in containerd
ctr containers list       # List containers
ctr tasks list            # List running tasks
ctr image pull docker.io/library/nginx:latest

# === crictl (CRI debug tool - ESSENTIAL) ===
crictl pods               # List pods (CRI-level)
crictl ps                 # List containers
crictl images             # List images
crictl inspect <CONTAINER_ID> # Detailed container info
crictl logs <CONTAINER_ID>   # Container logs (bypasses
                             # kubectl)
crictl exec -it <CONTAINER_ID> sh # Exec into container
crictl stats              # Resource usage (CPU,
                             # memory)
```

```
# === crictl vs docker ===
# crictl is the CRI-native replacement for docker CLI
# Use crictl when docker CLI is not available (containerd-only
nodes)
# Same concepts: pods=, containers=, images=

# === Low-level OCI runtime ===
# runc: the reference OCI runtime
# Creates cgroups + namespaces + mounts for each container
# Check which runtime:
runc --version

# === Runtime alternatives ===
# crun (Rust) – faster than runc, used by Red Hat/OpenShift
# gVisor (Google) – user-space kernel, stronger isolation than runc
# Kata Containers – lightweight VM per container, hardware
isolation
# Check node:
kubectl get nodes -o json | jq
'.items[].status.nodeInfo.containerRuntimeVersion'

# === CRI-O ===
# Red Hat's Kubernetes-native runtime
# Socket: /run/crio/crio.sock
# Use crictl with: crictl --runtime-endpoint
unix:///run/crio/crio.sock
```

CNI: Container Networking Interface

CNI is the plugin model for Kubernetes networking. **Each pod gets its own network namespace. The CNI plugin creates a veth pair, attaches one end to the pod namespace and the other to a bridge/overlay, and assigns an IP address.** At FAANG, choices include: Cilium (eBPF-based, fastest growing), Calico (policy-rich, widely used), and AWS VPC CNI (native VPC IPs, no overlay).

```

# === Pod networking (Linux view) ===
kubectl exec <pod> -- ip addr          # Inside pod: network
namespace
kubectl exec <pod> -- ip route          # Inside pod: routing table
kubectl exec <pod> -- cat /etc/resolv.conf # DNS config (inside
pod)

# === Node-side: pod network namespace ===
# Find pod's network namespace:
POD_UID=$(kubectl get pod <pod> -o jsonpath='{.metadata.uid}')
# Enter pod's netns from node:
ln -sf /proc/1/ns/net /var/run/netns/default # (on some systems)
# Better: use nsenter

# === CNI plugins directory ===
ls /opt/cni/bin/                        # Installed CNI plugins
# bridge, host-local, loopback, portmap, bandwidth, tuning

# === CNI config ===
cat /etc/cni/net.d/10-aws.conflist      # CNI configuration (varies
by plugin)

# === Check which CNI is in use ===
kubectl get pods -n kube-system | grep -E "
(cilium|calico|flannel|weave|aws-node)"

# === Cilium (eBPF-based CNI) ===
# Cilium replaces kube-proxy entirely with eBPF
# No iptables rules needed – eBPF programs handle service load
balancing
# Check Cilium status:
kubectl -n kube-system exec daemonset/cilium -- cilium status
kubectl -n kube-system exec daemonset/cilium -- cilium endpoint
list

# === Cilium Hubble (network observability) ===
# Shows pod-to-pod traffic with L3/L4/L7 visibility
# Requires no application changes – eBPF captures all traffic

# === Network policy ===

```

```
# CNI plugins enforce Kubernetes NetworkPolicy
# (default: all pods can communicate – you must add policies)
kubectl get networkpolicies --all-namespaces
```

CSI: Container Storage Interface

CSI provides a standard API for storage providers (EBS, EFS, GCE PD, Azure Disk). **When a pod requests a PVC, the CSI driver provisions the underlying storage (e.g., an EBS volume), attaches it to the node, mounts it into the pod's filesystem.** At FAANG, understanding CSI is essential for debugging volume mount failures, performance issues, and capacity planning.

```
# === Pod storage (Linux view) ===
kubectl exec <pod> -- df -h          # Mounted volumes inside pod
kubectl exec <pod> -- mount          # All mounts inside pod
kubectl exec <pod> -- ls -la /dev/    # Block devices inside pod

# === Node-side: volume mounts ===
# Host path for mounted volumes:
ls /var/lib/kubelet/pods/<POD_UID>/volumes/kubernetes.io~csi/
# Or for in-tree volumes:
ls /var/lib/kubelet/pods/<POD_UID>/volumes/kubernetes.io~aws-ebs/

# === Check CSI driver ===
kubectl get csidrivers                # Installed CSI drivers
kubectl get nodes -o json | jq '.items[0].status.volumesAttached'
# Shows which volumes are attached to the node

# === EBS CSI driver debugging ===
kubectl logs -n kube-system -l app=ebs-csi-node # Node-side driver logs

# Find EBS volume ID for a PVC:
PV_NAME=$(kubectl get pvc <pvc> -o jsonpath='{.spec.volumeName}')
VOLUME_ID=$(kubectl get pv $PV_NAME -o jsonpath='{.spec.csi.volumeHandle}')
```

```

echo $VOLUME_ID # This is the EBS volume ID

# === Node-level: check attached volumes ===
lsblk           # Check if EBS volume is
attached
sudo nvme list   # NVMe device list (Nitro
instances)
# Modern AWS Nitro instances expose EBS as NVMe devices

# === Filesystem inside volume ===
# If a volume can't mount, check:
# 1. Is the volume attached? (lsblk)
# 2. Is there a filesystem? (file -s /dev/nvme1n1)
# 3. Can it be mounted manually? (mount /dev/nvme1n1 /mnt)

# === Storage classes ===
kubectl get storageclass
# Each SC defines: provisioner, reclaim policy, filesystem type,
IOPS, etc.
# Example: gp3 EBS with 5000 IOPS

```

FAANG Interview Q&A

Q: 'A pod is stuck in 'ContainerCreating' state. How do you debug from the Linux node perspective?'

A: 'SSH into the node (or use kubectl node shell). Check kubelet logs: `journalctl -u kubelet --since '5 min ago'`. Common causes: (1) Image pull failure — check network connectivity to the registry. (2) Volume mount failure — check if the CSI driver created the volume, if it's attached (lsblk), if the filesystem exists. (3) Resource pressure — the node may be out of CPU/memory/disk. Run `kubectl describe node` for conditions. (4) CNI error — pod network setup failed. Check CNI daemon logs. (5) Admission webhook timeout — sometimes a webhook hangs and

prevents pod creation. At FAANG, we have automation that detects 'ContainerCreating' > 2 minutes and pages the on-call engineer with the most likely cause based on patterns.'

Q: 'How does kube-proxy work, and why is Cilium replacing it?'

A: 'Kube-proxy implements Service load balancing via userspace proxy, iptables, or IPVS. The iptables mode creates iptables rules for every Service endpoint — on a 10,000-service cluster, this means 100K+ iptables rules. Every packet traverses all these rules (O(n) cost), causing latency spikes during rule updates. Cilium replaces this with eBPF — a BPF program for each Service that runs at the XDP or TC layer, with O(1) lookup. Updates are atomic (bpf map update, no rule churn). Result: lower latency, better scalability, and no iptables overhead. At FAANG, any cluster with > 1000 services has either moved to Cilium or IPVS.'

Q: 'A node is reporting 'DiskPressure'. What's happening at the Linux level?'

A: 'DiskPressure means the node's available disk space or inodes are below the kubelet's eviction threshold. Check `df -h` and `df -i` on the node. Common culprits: (1) Container logs not rotated — `/var/log/pods/` can grow large. (2) Docker/containerd images — `docker system prune -a` or `cricctl rmi --prune`. (3) Ephemeral volume usage — pods writing to emptyDir volumes. (4) System logs — `journalctl --vacuum-size=500M`. Kubelet will start evicting

pods when DiskPressure is true, starting with the lowest priority pods. At FAANG, we set aggressive kubelet GC thresholds and monitor node disk usage to prevent this.'

References & Further Reading

- **Kubernetes Architecture** — Official docs on nodes, kubelet, control plane
- **CRI Specification** — Container Runtime Interface deep dive
- **Cilium: eBPF for Kubernetes** — How Cilium replaces kube-proxy with eBPF

FAANG Architect's Note: Kubernetes is often taught as a YAML abstraction — you write manifests, you get clusters. But every Kubernetes abstraction maps to a Linux primitive: Pod → cgroups + namespaces, Service → iptables/eBPF rules, Volume → mount + filesystem, Node → Linux server with kubelet. The engineers who truly understand Kubernetes are those who can cross the abstraction boundary — they know what 'ContainerCreating' means at the CRI level, what 'DiskPressure' means at the filesystem level, and what 'CrashLoopBackOff' means at the process level. If you understand Linux, you already understand 80% of Kubernetes — you just need to learn the orchestration layer on top.

Production Tip: Every Kubernetes engineer should know `crictl`. When `kubectl` fails (network issues, API server down, authentication problems), `crictl` gives you direct access to the container runtime on the node. You can see all pods, their state, their logs, and their resource usage without depending on the Kubernetes API. This is your 'break glass' tool for cluster debugging. Install it on every node and add it to your incident response runbook.

CHAPTER 26

Production Troubleshooting

Containers & SRE · 6 topics

Production troubleshooting is the art of diagnosing and resolving issues in live systems under pressure. At FAANG, the expectation is that you can SSH into any server (or use equivalent remote access) and systematically identify the root cause of an issue — whether it's a performance degradation, a crash, a network partition, or a resource exhaustion. This guide covers the systematic approach and essential techniques.

FAANG

Linux

troubleshooting

incident response

debugging

SRE

Incident Response: The First 5 Minutes

When a page comes in, the first 5 minutes set the tone for the entire incident. **Stay calm, follow the runbook, and gather data before making changes.** At FAANG, the incident response protocol is: acknowledge → assess → assemble → act. The cardinal rule: **don't make things worse.**

```
# === The 60-second triage (Brendan Gregg's checklist) ===

# 1. uptime — load averages
echo "=== UPTIME ==="
uptime

# 2. dmesg — kernel errors
```

```

echo "=== KERNEL ==="
dmesg | tail -20

# 3. vmstat – system-level CPU/memory/swap
echo "=== VMSTAT ==="
vmstat 1 3

# 4. mpstat – per-CPU balance
echo "=== PER-CPU ==="
mpstat -P ALL 1 1

# 5. pidstat – per-process CPU
echo "=== TOP CPU PROCESSES ==="
pidstat 1 3 | tail -10

# 6. iostat – disk I/O
echo "=== DISK I/O ==="
iostat -xz 1 2

# 7. free -m – memory
echo "=== MEMORY ==="
free -h

# 8. sar -n DEV – network
echo "=== NETWORK ==="
sar -n DEV 1 2 | grep -E "(Interface|eth|ens|bond)"

# 9. sar -n TCP,ETCP – TCP stats
echo "=== TCP ==="
sar -n TCP,ETCP 1 1

# 10. top – overview
echo "=== TOP ==="
top -b -n 1 | head -20

# === Before making any change ===
# 1. Confirm the issue: is this an alert false positive?
# 2. Check if a recent deploy/change caused it
# 3. Is this a known issue? Check the runbook
# 4. Communicate: 'I'm investigating, ETA unknown'
# 5. If risky: 'What's the rollback plan?'

```

Diagnosing High CPU & Memory Issues

CPU and memory issues are the most common production problems. **The key question: is this a traffic increase (legitimate high usage) or a code issue (infinite loop, memory leak)?** At FAANG, we use perf, heap dumps, and continuous profiling to answer this question in minutes, not hours.

```
# === High CPU ===
# 1. Identify the process:
ps aux --sort=-%cpu | head -10

# 2. Check threads:
top -H -p <PID>

# 3. Sample the hot function:
perf top -p <PID>

# 4. Capture stack samples:
perf record -F 99 -p <PID> -g --sleep 30
perf report -g --stdio | head -40

# 5. Check for common patterns:
# - Infinite loop: check application logs for repeated log messages
# - GC storm (Java/Go): check GC metrics
# - Event loop blocked (Node/Python): check event loop lag
# - Contention: check thread dumps (jstack, pstack, gdb)

# === Java thread dump (blocked threads) ===
jstack <PID> # Full thread dump
jstack <PID> | grep -A 20 "BLOCKED" # Find blocked threads

# === Memory leak ===
# 1. Confirm leak (RSS grows over time):
ps -p <PID> -o rss,vsz,pmem

# 2. Check /proc/<PID>/status for memory breakdown:
cat /proc/<PID>/status | grep -E "(VmRSS|VmSize|VmPeak|VmSwap)"
```

```
# 3. Check page fault rate (major faults = swapping):
cat /proc/<PID>/status | grep -i fault

# 4. Java heap analysis:
# Capture heap dump:
jmap -dump:live,format=b,file=heap.hprof <PID>
# Analyze with Eclipse MAT or jhat

# 5. Native memory leak (Valgrind):
valgrind --tool=memcheck --leak-check=full ./myapp

# 6. Python memory profiling:
pip install memory_profiler
python -m memory_profiler myapp.py

# === Node.js memory ===
# Take heap snapshot (need --inspect):
kill -USR2 <PID> # Toggle inspector
# Then connect Chrome DevTools or use node --inspect

# === Go memory ===
curl http://localhost:6060/debug/pprof/heap > heap.out
go tool pprof heap.out
```

Diagnosing Disk & I/O Issues

Disk issues manifest as: 'disk full', 'slow I/O', 'filesystem errors', or 'process stuck in D state'. **The most common cause of disk-full incidents is log files that weren't rotated.** At FAANG, disk monitoring is automated but engineers still need to diagnose root cause when the alert fires.

```
# === Disk full ===
df -h # Check space
df -i # Check inodes (filesystem
can be full even with
```

```

free space!))

# Find the culprit:
du -sh /* | sort -rh | head -10      # Top-level directories
du -sh /var/log/* | sort -rh | head -10 # Log directories
find /var/log -type f -size +100M     # Large files > 100MB

# Common space hogs:
# - Log files: /var/log/, /var/log/journal/
# - Docker: /var/lib/docker/, docker system prune -a
# - Package cache: apt-get clean, yum clean all
# - Core dumps: /var/crash/, /var/lib/systemd/coredump/
# - Old kernels: /boot/, apt-get autoremove --purge

# === Filesystem corruption ===
# If disk errors appear:
dmesg | grep -i "I/O error\|buffer I/O"

# Check filesystem (unmounted):
sudo fsck.ext4 -f /dev/sdb1

# XFS repair:
sudo xfs_repair /dev/sdb1

# === Slow I/O ===
iostat -x 1 5                      # Check %util, await, aqu-sz
iotop -oP                          # Which process is doing I/O?

# Check for I/O throttling:
cat /sys/fs/cgroup/system.slice/*.service/io.stat 2>/dev/null

# === Process stuck in D (uninterruptible sleep) ===
ps aux | grep " D"
# D state = waiting for I/O in kernel (can't be killed)
cat /proc/<PID>/stack                # Show kernel stack (what's
it waiting for?)
cat /proc/<PID>/wchan                # Kernel function it's
blocked in

# === NFS issues ===
mount | grep nfs                    # Check NFS mounts
# If NFS server is down, processes hang in D state

```

```
# Fix: remount with 'soft' option (but risk of data corruption)
# Better: fix the NFS server or use 'hard,intr' with reasonable
timeo
```

Diagnosing Network Issues

Network issues can be: **'connection refused'** (service not listening), **'connection timed out'** (firewall or routing), **'connection reset'** (service crashed or overloaded), or **'intermittent failures'** (packet loss, DNS issues). At FAANG, network troubleshooting is a layered skill — you start with basic connectivity tests and escalate to packet captures.

```
# === Layer 1: Is the port open? ===
ss -tlnp | grep :80          # Is something listening on
port 80?
nc -zv localhost 80          # Local connectivity test

# Layer 2: Is the network reachable?
ping -c 3 10.0.0.1           # Basic ICMP test
mtr 10.0.0.1                 # Path analysis (traceroute +
ping)

# Layer 3: Is the firewall blocking?
sudo iptables -L -n | grep :80 # Check iptables rules
# Cloud: check security groups / network ACLs

# Layer 4: Socket state
ss -tanp | grep :80          # Connection state
# TIME_WAIT high: normal for proxies, but can exhaust ports
# CLOSE_WAIT high: application bug (not closing sockets)

# Layer 5: Application-level
curl -v http://localhost:80   # HTTP headers + timing
curl -w "@curl-format.txt" -o /dev/null -s http://localhost:80
# Custom timing format:
cat > curl-format.txt << 'EOF'
```

```

time_namelookup: %{time_namelookup}s
time_connect:    %{time_connect}s
time_appconnect: %{time_appconnect}s
time_pretransfer: %{time_pretransfer}s
time_redirect:   %{time_redirect}s
time_starttransfer: %{time_starttransfer}s
-----
time_total:      %{time_total}s
EOF

# === DNS issues ===
dig google.com +short           # Quick DNS check
dig @8.8.8.8 myapp.internal      # Bypass local DNS resolver
getent hosts myapp.internal      # System resolver (respects
/etc/hosts)

# === Packet capture for deep debugging ===
sudo tcpdump -ni any port 80 -c 100 # Capture 100 packets on port
80
sudo tcpdump -ni any 'tcp[tcpflags] & (tcp-syn) != 0' # SYN
packets only

# === TCP retransmit check ===
netstat -s | grep -i "retrans\|segments" # TCP retransmit count
# Compare to total segments: retransmits > 1% indicates packet loss

# === Connection tracking ===
cat /proc/net/nf_conntrack | wc -l      # Connection tracking table
size
cat /proc/sys/net/netfilter/nf_conntrack_max # Max connections
tracked
# If connection table is full, new connections are dropped!

```

FAANG Interview Q&A

Q: 'Walk me through debugging 'It doesn't work' from a production system.'

A: 'First, I need specifics — 'doesn't work' is not actionable. What exactly is failing? What's the error? Which users are affected? When did it start? Was there a recent deploy? With that information: (1) Check the service's health endpoint. (2) Check the four golden signals: latency, traffic, errors, saturation. (3) Check the application logs for error messages. (4) Check the load balancer metrics — is traffic being evenly distributed? (5) Check upstream dependencies — is the database responding? Is the cache hot? (6) Check the host metrics — CPU, memory, disk, network. (7) If everything looks normal, I'd look at recent changes — a deploy, a config change, or a dependency upgrade. At FAANG, the first question in any incident is always: 'What changed?' because 90% of incidents are caused by changes.'

Q: 'What's the difference between a crash and a hang?'

A: 'A crash is when the process terminates unexpectedly (segfault, OOM kill, uncaught exception). You can restart it and move on — but you need to find the root cause to prevent recurrence. Look at core dumps, application logs, and crash logs. A hang is when the process is still running but not making progress (deadlock, infinite loop, waiting on a resource that never comes). The process is alive but not serving. Debug with thread dumps (jstack, pstack, gdb), strace to see what syscall it's stuck on, and /proc//stack to see the kernel function it's blocked in. Hangs are harder to debug than crashes because the process is still alive — you need to inspect its state without killing it.'

Q: 'A service was working yesterday and is broken today. What's

your troubleshooting approach?'

A: 'This is the classic 'it was working' scenario. Approach: (1) Change detection — what changed since it was working? Deploy? Config change? DNS change? Dependency update? Certificate expiry? (2) Isolation — can I reproduce the issue in a non-production environment? Can I rollback the most recent change? (3) Compare — compare the working config (git history) to the current config. Compare the working version to the broken version. (4) Temporal correlation — did the issue start at the same time as another event (cron job, backup, traffic spike, upstream degradation)? At FAANG, we practice 'immutable deployments' — we never modify running servers, so 'it was working yesterday' means the issue is either a new deployment, a change in traffic patterns, a dependency failure, or a time-based condition (cert expiry, rate limit, database full). The rollback is almost always the fastest fix — then investigate why the new version broke things.'

References & Further Reading

- **The USE Method** — Brendan Gregg's systematic performance analysis methodology
- **Linux Performance: 60-Second Troubleshooting** — The essential 60-second triage checklist
- **Google SRE Book: Effective Troubleshooting** — Systematic approach to incident response

FAANG Architect's Note: Troubleshooting is a skill that improves with practice, but the best engineers follow a systematic method. They don't guess — they gather data, form hypotheses, and test them. The USE method (Utilization, Saturation, Errors) gives you a framework that works for any resource. The key mental shift: don't ask 'what's broken?', ask 'what changed?' Production systems fail when something changes — a deploy, a config, a traffic pattern, a dependency. Find the change, and you've found the root cause.

Production Tip: Before troubleshooting any issue, run the 60-second checklist. It takes one minute and gives you a complete picture of system health. I've lost count of how many times someone said 'the database is slow' and the 60-second check revealed it was actually a CPU issue on the application server, or a network bandwidth issue, or the database was just fine but the application had a connection pool leak. The checklist prevents wasted time going down the wrong path.

CHAPTER 27

System Rescue & Recovery

Containers & SRE · 7 topics

Every senior engineer faces an unbootable server, a corrupt filesystem, or a forgotten root password at some point. System rescue skills — the ability to recover a system using recovery media, chroot, and manual intervention — separate engineers who can recover from disaster from those who have to reinstall. At FAANG, automated recovery handles most cases, but when the automation fails, manual rescue skills are essential.

FAANG Linux recovery GRUB chroot rescue
filesystem repair

Boot Process: From Power-On to Login Prompt

Understanding the boot sequence lets you identify where a boot failure occurs. **BIOS/UEFI → Bootloader (GRUB) → Kernel → initramfs → root filesystem → systemd (PID 1) → login**. Each stage has specific failure modes. At FAANG, the boot sequence is optimized for speed (fast boot targets, parallel service startup) but the fundamentals are the same.

```
# === Boot stage 1: Firmware (BIOS/UEFI) ===  
# BIOS: legacy, MBR partition, 16-bit mode  
# UEFI: modern, GPT partition, 64-bit, Secure Boot  
# Check current mode:
```

```
[ -d /sys/firmware/efi ] && echo "UEFI" || echo "BIOS"
ls /sys/firmware/efi/efivars/          # UEFI variables (if UEFI)

# === Boot stage 2: Bootloader (GRUB) ===
# GRUB config: /boot/grub/grub.cfg (DO NOT EDIT – generated by
# update-grub)
# User config: /etc/default/grub (edit this, then run update-grub)

cat /etc/default/grub
# GRUB_DEFAULT=0                      # Default menu entry
# GRUB_TIMEOUT=5                      # Seconds before auto-boot
# GRUB_CMDLINE_LINUX=""               # Kernel boot parameters
# GRUB_CMDLINE_LINUX_DEFAULT="quiet splash" # Additional params

# Common boot parameters:
# single          – Boot to single-user mode (recovery)
# emergency       – Boot to emergency shell
# init=/bin/bash  – Override init (skip systemd)
# nomodeset       – Disable kernel mode-setting (display issues)
# selinux=0       – Disable SELinux (troubleshooting only)
# systemd.unit=rescue.target – Boot to rescue target

# Update GRUB after changes:
sudo update-grub          # Debian/Ubuntu
sudo grub2-mkconfig -o /boot/grub2/grub.cfg # RHEL/Amazon Linux

# === Boot stage 3: Kernel + initramfs ===
# initramfs (initial RAM filesystem) contains:
# - Essential kernel modules (storage, filesystem, network)
# - /init script that mounts root filesystem
# - Busybox or recovery tools

# List initramfs contents:
lsinitramfs /boot/initrd.img-$(uname -r) # Ubuntu
lsinitrd /boot/initramfs-$(uname -r).img  # RHEL

# Rebuild initramfs:
sudo update-initramfs -u          # Ubuntu
sudo dracut -f                    # RHEL/Amazon Linux

# === Boot stage 4: Root filesystem + systemd ===
# /init mounts rootfs, then pivots to real root
```

```
# systemd reads default.target → starts services
# Check default target:
systemctl get-default
```

GRUB: Bootloader Rescue & Repair

GRUB failure = system won't boot. **Common causes:** **misconfigured grub.cfg, missing kernel, damaged MBR/GPT, or wrong root filesystem UUID.** At FAANG, boot failures are handled by automated recovery (PXE boot, IPMI console, AWS Systems Manager) — but manual GRUB rescue is the fallback.

```
# === GRUB rescue shell ===
# If GRUB can't find its config, you get:
# grub rescue>

# Commands in grub rescue:
grub rescue> ls                                # List available drives
# (hd0) (hd0,msdos1) (hd0,msdos2)
grub rescue> ls (hd0,msdos1)/                  # List files on partition
grub rescue> set prefix=(hd0,msdos1)/boot/grub
grub rescue> set root=(hd0,msdos1)
grub rescue> insmod normal
grub rescue> normal                            # Load normal GRUB menu

# === GRUB interactive editing ===
# At GRUB menu, press 'e' to edit boot entry
# Common fixes:
# 1. Change root=UUID=... to root=/dev/sda1 (if UUID changed)
# 2. Add 'single' or 'emergency' to boot into recovery
# 3. Add 'init=/bin/bash' to skip init entirely
# 4. Fix broken root= parameter

# === Reinstall GRUB from live CD ===
# Boot from Ubuntu live USB, mount root filesystem:
sudo mkdir /mnt/root
sudo mount /dev/sda1 /mnt/root
```

```

sudo mount --bind /dev /mnt/root/dev
sudo mount --bind /proc /mnt/root/proc
sudo mount --bind /sys /mnt/root/sys

# Chroot and reinstall GRUB:
sudo chroot /mnt/root
grub-install /dev/sda
update-grub
exit
sudo umount -R /mnt/root
# Reboot

# === Fix broken kernel/boot files ===
# If /boot is empty (accidental rm -rf /boot):
# Boot from live CD, chroot, install new kernel:
apt-get install --reinstall linux-image-$(uname -r)
update-grub

# === GRUB password protection ===
grub-mkpasswd-pbkdf2          # Generate password hash
# Add to /etc/grub.d/40_custom:
# set superusers="admin"
# password_pbkdf2 admin grub.pbkdf2.sha512.10000...
# Then update-grub

```

Chroot Rescue & Single-User Mode

When a system won't boot normally, you can **boot from recovery media, mount the root filesystem, and chroot into it** to repair packages, fix configuration files, reset passwords, or rebuild the bootloader. **Single-user mode** boots with minimum services and a root shell. At FAANG, these techniques are documented in runbooks and tested during disaster recovery drills.

```

# === Single-user mode ===
# At GRUB prompt, edit boot entry, add 'single' to kernel line
# Press Ctrl+X to boot

```

```

# You get a root shell with minimal services
# Remount root as read-write:
mount -o remount,rw /

# Fix the issue, then reboot

# === Emergency mode ===
# Even more minimal than single-user
# At GRUB: add 'emergency' or 'systemd.unit=emergency.target'
# Only PID 1 runs, root filesystem is read-only

# === Chroot rescue (from live USB) ===
# Step 1: Boot from Ubuntu/Debian live USB
# Step 2: Identify and mount root:
lsblk                                # Find root partition
sudo mkdir /mnt/root
sudo mount /dev/sda2 /mnt/root      # Mount root filesystem

# Step 3: Mount essential filesystems:
sudo mount --bind /dev /mnt/root/dev
sudo mount --bind /proc /mnt/root/proc
sudo mount --bind /sys /mnt/root/sys
sudo mount --bind /run /mnt/root/run # Some systems need this

# Step 4: Mount boot partition if separate:
sudo mount /dev/sda1 /mnt/root/boot

# Step 5: Chroot:
sudo chroot /mnt/root /bin/bash
source /etc/profile
export PS1="(chroot) \[\e[31m\]\u@\h:\w\$[\e[0m\] "

# === Inside chroot: common repairs ===
# Reset root password:
passwd root

# Fix broken packages:
apt-get install -f

# Reinstall bootloader:
grub-install /dev/sda
update-grub

```

```
# Rebuild initramfs:
update-initramfs -u -k all

# Fix filesystem:
e2fsck -f /dev/sda2

# === Exit chroot ===
exit
sudo umount -R /mnt/root
sudo reboot
```

Filesystem Recovery & Data Rescue

Filesystem corruption can result from power loss, hardware failure, or improper shutdown. **ext4 has a journal for fast recovery; XFS has metadata writeback logging.** At FAANG, filesystem corruption is rare (EBS volumes have 99.999% durability, proper shutdown procedures are enforced), but when it happens, the response is methodical: assess damage → backup → repair → restore.

```
# === Check filesystem health ===
# Check without modifying (safe):
sudo fsck -n /dev/sdb1          # ext4: check only, no
repair
sudo xfs_repair -n /dev/sdb1    # XFS: check only

# === ext4 journal recovery ===
# ext4 has a journal that records metadata changes
# On unclean shutdown: journal is replayed on mount (automatic)
# If journal is corrupt:
sudo fsck.ext4 -f /dev/sdb1     # Force check
sudo fsck.ext4 -fy /dev/sdb1    # Force + yes (non-
interactive)
```

```

# === XFS repair ===
# XFS must be unmounted for repair
sudo xfs_repair /dev/sdb1
sudo xfs_repair -v /dev/sdb1          # Verbose
sudo xfs_repair -L /dev/sdb1          # Clear log (DANGER: may
lose recent metadata)

# === Bad blocks ===
# Check for hardware-level bad sectors:
sudo badblocks -sv /dev/sdb1
# If bad blocks found: replace the drive!
# For EBS: snapshot, create new volume, detach old

# === SMART data (hardware health) ===
sudo smartctl -a /dev/sda              # Full SMART report
sudo smartctl -H /dev/sda              # Overall health
(PASSED/FAILED)
# Key SMART attributes:
# 5: Reallocated_Sector_Ct (> 0 = bad!)
# 187: Reported_Uncorrectable_Errors
# 197: Current_Pending_Sector
# 198: Offline_Uncorrectable

# === ddrescue (recover data from failing disk) ===
sudo apt install gddrescue
sudo ddrescue -d /dev/sdb /dev/sdc rescue.log
# Retry failed sectors:
sudo ddrescue -d -r3 /dev/sdb /dev/sdc rescue.log

# === Backup superblock (ext4) ===
sudo mkfs.ext4 -n /dev/sdb1            # Show backup superblock
locations
sudo fsck.ext4 -b 32768 /dev/sdb1      # Use alternate superblock

# === XFS backup superblock ===
sudo xfs_db -x -c "sb 0" -c "p" /dev/sdb1 # Check primary
superblock
sudo xfs_repair -b 1 /dev/sdb1          # Use backup superblock (1,
2, 4, 8, ...)

# === If data is critical ===
# 1. STOP: don't mount the filesystem, don't run fsck

```

```
# 2. Create a block-level backup:
sudo dd if=/dev/sdb1 of=/backup/sdb1.img bs=1M status=progress
# 3. Work on the backup, not the original
# 4. Use testdisk or photorec for file recovery
# 5. At FAANG: snapshot EBS volume, then repair snapshot
```

FAANG Interview Q&A

Q: 'A server won't boot. You can't SSH in. What do you do?'

A: 'First, access the console out-of-band — IPMI/iLO/iDRAC for bare metal, AWS Console → EC2 Serial Console for cloud, or recovery mode for VMs. Check what stage it reaches: (1) Power-on: does it POST? Check power supply, IPMI status. (2) Bootloader: can you see GRUB? Edit GRUB entry and add 'single' for recovery. (3) Kernel: is it panicking? Add 'earlyprintk' to boot params. (4) Filesystem: is fsck failing? Boot from recovery media (live CD, rescue ISO) and check. (5) Systemd: is a service hanging? Add 'systemd.log_level=debug systemd.log_target=console' to boot params. At FAANG, the first response is usually: attach the EBS volume to a healthy instance and repair from there — faster than debugging the failing boot process.'

Q: 'A team member accidentally deleted /etc/passwd on a production server. How do you recover?'

A: 'Don't reboot! The system is still running — as long as existing processes are alive, they keep their credentials. First, restore the file from backup: `cp /var/backups/passwd /etc/passwd` (if backups exist). If no backup, reconstruct from /etc/passwd- (a backup

maintained by pwconv): `cp /etc/passwd- /etc/passwd` . If that doesn't exist, recreate users from `getent passwd` output captured on other servers, or reconstruct from `/etc/shadow` and `/etc/group`. At FAANG, we have immutable infrastructure — `/etc/passwd` is managed by configuration management and restored automatically. If a file is deleted, the config management agent (Chef, Ansible) recreates it on the next run.'

Q: 'What would you include in a disaster recovery plan for a Linux-based service?'

A: 'A DR plan should cover: (1) Recovery objectives: RTO (Recovery Time Objective — how fast) and RPO (Recovery Point Objective — how much data loss). (2) Backup strategy: filesystem-level (duplicity/rsync), block-level (EBS snapshots), database-level (WAL archiving). (3) Recovery procedures: step-by-step runbooks for rebuilding a server, restoring from backup, promoting a replica. (4) Boot recovery: live USB images, PXE boot configuration, GRUB rescue procedures. (5) Testing: regular DR drills where the team actually goes through the recovery process. At FAANG, DR is not a document — it's an automated process. We practice 'chaos engineering' where servers are intentionally terminated to verify auto-recovery. The manual DR procedures are for the 0.1% of cases where automation fails.'

References & Further Reading

- **GRUB Rescue Shell** — Official GRUB rescue documentation

- **Ubuntu Live CD Recovery** — Community guide for chroot rescue procedures
- **Arch Linux Filesystem Repair** — Comprehensive fsck and recovery reference

FAANG Architect's Note: System rescue is one of those skills that seems obsolete in the cloud era — until you need it. At FAANG, we have automated recovery for 99% of scenarios: instance health checks, auto-scaling, self-healing Kubernetes, immutable infrastructure. But that 1% — the boot loop after a kernel update, the EBS volume that won't attach, the GRUB config that got corrupted by a race condition — that's when manual rescue skills matter. The engineers I trust most in an incident are the ones who have actually chrooted into a broken system and fixed it. They don't panic; they follow the recovery procedure step by step.

Production Tip: Before you need it, create a rescue USB or ISO for your distribution and test it. Know how to: (1) Boot from the rescue media. (2) Identify and mount the root filesystem (with LVM, encryption, or RAID). (3) Chroot and repair. Practice this once in a staging environment. When the real incident happens, you'll be glad you did. Also: always keep at least one kernel version that you know boots correctly. Don't delete all old kernels — keep the last two known-good versions.

CHAPTER 28

Security Hardening

Containers & SRE · 8 topics

Linux security is defense in depth: from kernel hardening (SELinux, AppArmor, seccomp) to system configuration (sysctl, PAM, auditd) to operational practices (patch management, least privilege, incident response). At FAANG, every server is hardened against a baseline, and deviations are automatically detected and remediated.

FAANG Linux security SELinux AppArmor seccomp
auditd hardening

Kernel Hardening: sysctl Security Settings

The Linux kernel has many security-related parameters accessible via `sysctl`. **These settings mitigate common attack vectors: IP spoofing, SYN floods, source routing, and kernel pointer leaks.** At FAANG, these are enforced by configuration management (Ansible, Chef) and verified by compliance scanners.

```
# === /etc/sysctl.d/99-security.conf ===  
  
# === Network hardening ===  
# Disable IP forwarding (unless the host is a router)  
net.ipv4.ip_forward = 0  
net.ipv6.conf.all.forwarding = 0  
  
# Disable source packet routing  
net.ipv4.conf.all.accept_source_route = 0
```

```

net.ipv6.conf.all.accept_source_route = 0

# Enable IP spoofing protection (reverse-path filtering)
net.ipv4.conf.all.rp_filter = 1

# Ignore ICMP redirects (MITM vector)
net.ipv4.conf.all.accept_redirects = 0
net.ipv6.conf.all.accept_redirects = 0
net.ipv4.conf.all.secure_redirects = 0
net.ipv4.conf.all.send_redirects = 0

# SYN flood protection
net.ipv4.tcp_syncookies = 1
net.ipv4.tcp_syn_retries = 5
net.ipv4.tcp_synack_retries = 2

# Don't log martians (bogus packets)
net.ipv4.conf.all.log_martians = 1

# === Kernel hardening ===
# Restrict kernel pointer access (prevents KASLR bypass)
kernel.kptr_restrict = 2

# Restrict dmesg to root only
kernel.dmesg_restrict = 1

# Restrict perf events to root only
kernel.perf_event_paranoid = 3

# Disable kexec (prevent kernel replacement without reboot)
kernel.kexec_load_disabled = 0

# ASLR (Address Space Layout Randomization)
kernel.randomize_va_space = 2

# Restrict ptrace (prevents process injection)
kernel.yama.ptrace_scope = 1

# === Apply ===
sudo sysctl -p /etc/sysctl.d/99-security.conf
sudo sysctl --system # Apply all sysctl files

```

```
# === Check current settings ===
sudo sysctl -a | grep -E
"(kernel.(kptr|dmesg|perf|yama)|net.ipv4.conf.all.
(rp_filter|accept_source))"
```

Seccomp: Secure Computing Mode

Seccomp restricts which syscalls a process can make. **seccomp-bpf (since Linux 3.5) allows programmable filters using BPF bytecode.** At FAANG, seccomp is mandatory for all containers — it's part of the container runtime default. A process that makes a syscall outside the allowed set is either killed (SECCOMP_RET_KILL) or denied with an error (SECCOMP_RET_ERRNO).

```
# === Check seccomp status ===
cat /proc/<PID>/status | grep Seccomp
# 0 = disabled
# 1 = strict (only read, write, exit, sigreturn)
# 2 = filter (custom seccomp-bpf policy)

# === Docker seccomp profiles ===
# Default profile: /usr/share/containers/seccomp.json (or Docker's
built-in)
# Run container with custom seccomp:
docker run --security-opt seccomp=custom.json nginx

# === Disable seccomp (not recommended!) ===
docker run --security-opt seccomp=unconfined nginx

# === Common seccomp policies ===
# For web apps: allow ~250-300 syscalls (most common)
# For databases: allow ~200-250 syscalls
# For minimal containers: allow ~50 syscalls
# Example denied syscalls: kexec_load, uselib, create_module,
delete_module
```

```
#
# Tools:
# - inspektor-gadget: inspect seccomp profiles of running
#   containers
# - strace: generate a list of syscalls used by your app, then
#   build profile

# === Generate seccomp profile ===
# Record syscalls:
strace -c -f -o syscalls.txt ./myapp
# Then convert to seccomp JSON profile using:
#
https://github.com/docker/labs/blob/master/security/seccomp/seccomp-profiles/allow-all.json
n

# === Check if seccomp blocked a syscall ===
# Look for: 'exit code 159' or SIGSYS (seccomp violation)
dmesg | grep -i "seccomp\|SIGSYS"
journalctl -k | grep -i seccomp

# === seccomp with systemd ===
[Service]
SystemCallFilter=@system-service # Allow standard service
syscalls
SystemCallArchitectures=native # Only allow native architecture
syscalls
SystemCallErrorNumber=EPERM      # Return 'Operation not
permitted' instead of killing
```

SELinux & AppArmor

SELinux (Security-Enhanced Linux) and **AppArmor** are Linux Security Modules (LSM) that enforce mandatory access control (MAC). **SELinux labels every file, process, and port with a security context and defines policies for allowed interactions.** AppArmor uses path-based profiles instead of labels. At FAANG,

SELinux is enabled on RHEL/Amazon Linux; AppArmor is the default on Ubuntu/Debian. Both achieve the same goal: confine processes even if they run as root.

```
# === SELinux (RHEL/Amazon Linux) ===
# Check status:
getenforce                # Enforcing | Permissive |
Disabled                  #
sestatus                  # Detailed SELinux status

# Modes:
# Enforcing - Security policy is enforced (production)
# Permissive - Policy violations are logged but not blocked
#               (debugging)
# Disabled - SELinux is off (not recommended)

# Switch modes (runtime):
sudo setenforce 0          # Permissive (for
troubleshooting)
sudo setenforce 1          # Enforcing
# Permanent: /etc/selinux/config - SELINUX=enforcing

# === SELinux context ===
ls -Z /var/www/html/index.html # Show SELinux context
# system_u:object_r:httpd_sys_content_t:s0
# user:role:type:level

ps auxZ | grep nginx        # Show process SELinux context

# === Common SELinux commands ===
semanage fcontext -l | grep /var/log # List file contexts
semanage port -l | grep http         # List port contexts

# Fix file context:
sudo restorecon -Rv /var/www/html    # Restore default contexts

# Change file context:
sudo semanage fcontext -a -t httpd_sys_content_t "/web(/.*)?"
sudo restorecon -Rv /web
```

```

# Enable/disable booleans:
getsebool -a | grep http          # List SELinux booleans
sudo setsebool -P httpd_can_network_connect on # Allow Apache to
make outbound
connections

# === SELinux audit log ===
sudo ausearch -m avc -ts recent    # Recent denials
sudo sealert -a /var/log/audit/audit.log # Generate readable
denials

# === AppArmor (Ubuntu/Debian) ===
sudo aa-status                      # List loaded profiles
sudo apparmor_status

# Profile modes:
# enforce – Policy enforced (production)
# complain – Policy violations logged but allowed (debugging)

# Switch profile mode:
sudo aa-complain /usr/sbin/nginx    # Set to complain mode
sudo aa-enforce /usr/sbin/nginx     # Set to enforce mode

# === AppArmor profile example ===
# /etc/apparmor.d/usr.sbin.nginx
# profile nginx /usr/sbin/nginx {
#   include <abstractions/base>
#   include <abstractions/nginx>
#   /etc/nginx/** r,
#   /var/log/nginx/* w,
#   /var/www/html/** r,
# }

# === Disable (not recommended for production) ===
# To disable temporarily:
sudo ln -sf /etc/apparmor.d/usr.sbin.nginx /etc/apparmor.d/disable/
sudo apparmor_parser -R /etc/apparmor.d/usr.sbin.nginx

```

Auditd: Linux Audit System

The Linux Audit System (`auditd`) records security-relevant events: file access, syscalls, user logins, and configuration changes. **At FAANG, auditd is mandatory on all servers — it provides the audit trail for security incidents and compliance.** The audit logs are shipped to centralized SIEM systems (Splunk, Elasticsearch) for analysis and long-term retention.

```
# === auditd setup ===
sudo apt install auditd audispd-plugins
sudo systemctl enable --now auditd

# === Rules file: /etc/audit/rules.d/audit.rules ===
# Monitor password file changes
-w /etc/passwd -p wa -k identity
-w /etc/shadow -p wa -k identity
-w /etc/sudoers -p wa -k sudoers
-w /etc/ssh/sshd_config -p wa -k sshd

# Monitor SUID/SGID file changes
-a always,exit -F arch=b64 -S execve -F euid=0 -k root_command

# Monitor user/group changes
-w /usr/sbin/useradd -p x -k useradd
-w /usr/sbin/usermod -p x -k usermod
-w /usr/sbin/groupadd -p x -k groupadd

# Monitor network configuration changes
-w /sbin/iptables -p x -k iptables
-w /sbin/ip -p x -k ip

# Monitor important directories
-w /etc/ -p wa -k etc_changes
-w /root/.bashrc -p wa -k root_config

# Monitor syscalls (performance impact!)
-a always,exit -F arch=b64 -S openat,open -F key=file_open

# === Query audit logs ===
# Recent events:
```

```

sudo ausearch -ts recent -k identity # Events tagged 'identity'
sudo ausearch -m USER_LOGIN         # Login events
sudo ausearch -m EXECVE              # Commands executed (sudo)

# Live monitoring:
sudo tail -f /var/log/audit/audit.log | aureport --interpret

# Summary report:
sudo aureport --summary              # Summary by event type
sudo aureport --user                 # By user
sudo aureport -ts this-week          # This week's events

# === Check auditd status ===
sudo auditctl -s                     # Status
sudo auditctl -l                     # List loaded rules

# === Log file ===
/var/log/audit/audit.log             # Raw audit events
# Format:
# type=SYSCALL msg=audit(1705300000.123:456): arch=c000003e
# syscall=2 success=yes exit=3
# a0=...
# Interpret: ausearch --interpret or audit2why

```

FAANG Interview Q&A

Q: 'A compliance auditor asks: how do you ensure that only authorized users can access production servers?'

A: 'At FAANG, we use a multi-layered approach: (1) No direct SSH — all access goes through a bastion host or session manager (AWS SSM, Teleport). (2) Authentication is via short-lived certificates (SSH CA) — no long-lived SSH keys on developer workstations. (3) Authorization is checked by the bastion — it integrates with the corporate identity system (Okta, LDAP) and enforces MFA. (4) All sessions are recorded — every keystroke is

logged and retained for compliance. (5) Access is time-bound — sessions expire after 4-8 hours, and access reviews are automated. (6) Changes are made through CI/CD pipelines, not SSH — SSH access is reserved for emergencies and debugging, not routine changes.'

Q: 'What's the difference between DAC and MAC?'

A: 'DAC (Discretionary Access Control) is the standard Unix permission model — the owner of a file decides who can read/write/execute it. Root can bypass all DAC checks. MAC (Mandatory Access Control) is enforced by the kernel via LSM (SELinux/AppArmor) — it overrides DAC decisions. Even if a file is `chmod 777`, SELinux can deny access if the process doesn't have the right security context. MAC is defense in depth: if an attacker compromises a service (e.g., nginx with a remote code execution), MAC prevents them from accessing files labeled for other services, even if they're running as root.'

Q: 'A server was compromised. What do you do?'

A: 'Follow the incident response protocol: (1) Isolate the server — remove it from the network, block traffic at the firewall, but DON'T shut it down (memory forensics). (2) Capture forensic data: memory dump (LiME, fmem), disk image, audit logs, process list, network connections. (3) Determine the attack vector: check `sshd` logs, web server logs, package manager logs for unauthorized installs. (4) Check persistence mechanisms: cron jobs, `systemd` services, SSH `authorized_keys`, kernel modules. (5) Contain: rotate

all credentials the server had access to. (6) Eradicate: rebuild the server from a known-good image, apply patches, review access controls. (7) Postmortem: what allowed the compromise? How do we prevent recurrence? At FAANG, incident response is a well-rehearsed process with dedicated security engineers, runbooks, and automated containment workflows.'

References & Further Reading

- **Linux Security Modules** — Kernel.org documentation on LSM framework
- **RHEL 9 Security Hardening** — Red Hat's comprehensive security guide
- **CIS Benchmarks for Linux** — Industry-standard security configuration guidelines

FAANG Architect's Note: Security is everyone's job at FAANG, not just the security team's. The principle is 'security as code' — security configurations are version-controlled, reviewed, and automatically enforced. The engineers who stand out are those who think about security as a design constraint, not an afterthought. They add seccomp profiles to their container definitions, they set kernel hardening params in their deployment configs, and they ensure their services follow least privilege — running as non-root, with only the capabilities they need, in a confined SELinux domain.

Production Tip: Before enabling SELinux enforcement on an existing server, run in permissive mode for at least a week. Collect the denials with `ausearch -m avc -ts boot` and use `audit2allow -a` to generate the required policy rules. The most common SELinux-related production outage is an engineer enabling enforcement without first identifying all the necessary permissions — the application crashes because it can't access files that DAC would allow but SELinux denies. At FAANG, SELinux is 'enforcing by default' in AMIs, so applications are developed and tested with SELinux enabled from day one.

CHAPTER 29

Linux at FAANG Scale

Containers & SRE · 7 topics

Running Linux on 10,000+ servers is fundamentally different from running Linux on one server. At FAANG, every Linux concern — kernel versions, package management, patching, monitoring, debugging — is automated, standardized, and scaled. This guide covers the architectural patterns, operational practices, and infrastructure systems that make Linux manageable at FAANG scale.

FAANG

Linux

fleet management

Golden AMI

capacity planning

SRE

scale

The Golden AMI Pipeline

At FAANG, servers are never installed manually. They're provisioned from a **Golden AMI** (Amazon Machine Image) — a pre-baked image with the approved OS version, kernel, packages, security agents, and configuration. **Immutable infrastructure means: you never SSH into a running server to fix it — you build a new AMI and redeploy.** The Golden AMI pipeline is one of the most critical pieces of infrastructure at any FAANG company.

```
# === Golden AMI pipeline stages ===  
# 1. Base Image – Official Ubuntu/RHEL/Amazon Linux from vendor  
# 2. Hardening – Apply CIS benchmarks, kernel params, security  
config
```

```

# 3. Agents – Install monitoring agent, logging agent, security
agent
# 4. Packages – Pre-install common packages (Python, curl, jq,
etc.)
# 5. Validation – Boot the AMI, run smoke tests, verify compliance
# 6. Signing – Sign the AMI for supply chain security
# 7. Promotion – Promote through environments: dev → staging → prod

# === Example: Packer template (simplified) ===
# packer build ubuntu-faang.pkr.hcl
# source "amazon-ebs" "ubuntu" {
#   ami_name      = "faang-ubuntu-22.04-{{timestamp}}"
#   instance_type = "c5.xlarge"
#   source_ami    = "ami-0c7217cdde317cfec" # Canonical Ubuntu
22.04
#   ssh_username = "ubuntu"
# }
# build {
#   provisioner "shell" {
#     scripts = [
#       "scripts/harden.sh",
#       "scripts/install-agents.sh",
#       "scripts/cleanup.sh"
#     ]
#   }
# }

# === What's in a Golden AMI ===
# OS: Amazon Linux 2023, Ubuntu 22.04 LTS, RHEL 9
# Kernel: Distribution kernel with FAANG-specific config
# Security: SELinux enforcing, auditd, OSSEC/Wazuh agent
# Monitoring: node_exporter, custom metric agent,
Datadog/CloudWatch
# Logging: rsyslog forwarder, journald configured for central
logging
# Networking: tuned sysctl params, ENA driver, Cilium prerequisites
# Packages: Python3, Go, curl, jq, htop, iotop, perf, tcpdump

# === AMI lifecycle ===
# - Weekly: Security patch AMI (CVE fixes)
# - Monthly: Feature AMI (new agents, config changes)
# - Quarterly: Major version AMI (Ubuntu 22.04 → 24.04 migration)

```

```
# Each AMI has a version tag: faang-ubuntu-22.04-v2024-01-15

# === Immutable deployments ===
# Old model: SSH into server, apt update, apt upgrade
# FAANG model: Build new AMI → Deploy autoscaling group → Terminate
old instances
# Benefits: reproducible, testable, revertible (just deploy old
AMI)
# Tools: Packer (build), Terraform/CloudFormation (deploy),
Spinnaker/ArgoCD (pipeline)
```

Kernel Management at Scale

At FAANG, kernel selection is strategic. **Most FAANGs run custom or semi-custom kernels** — either long-term stable with backported patches or an in-house fork. The kernel must support: eBPF (for Cilium, Falco), cgroups v2 (for container resource management), NVMe (for instance storage), and ENA (Elastic Network Adapter). Kernel upgrades are one of the riskiest operations and follow strict rollout procedures.

```
# === Kernel version strategy ===
# Amazon Linux 2023: kernel 6.1 LTS (AWS-maintained)
# Ubuntu 22.04 LTS: kernel 5.15 HWE (Hardware Enablement)
# RHEL 9: kernel 5.14 (Red Hat-maintained)
# Google: custom kernel based on 5.x/6.x with Google patches
# Meta: custom kernel (kernel.org + patches for efficiency)

# === Kernel configuration for FAANG workloads ===
# cat /boot/config-$(uname -r) | grep -E "
(CGROUP|BPF|NAMESPACE|NVME)"
# CONFIG_CGROUPS=y          - cgroups v2
# CONFIG_BPF=y              - eBPF support
# CONFIG_BPF_JIT=y          - BPF JIT compiler
# CONFIG_NAMESPACES=y       - Namespace support
# CONFIG_NVME_CORE=y        - NVMe SSD support
```

```
# CONFIG_XDP_SOCKETS=y          - XDP socket support
# CONFIG_IO_URING=y             - io_uring async I/O
# CONFIG_NUMA_BALANCING=y       - NUMA-aware scheduling

# === Patching strategy ===
# Critical CVEs: patched within 24-48 hours (emergency AMI build)
# Regular CVEs: patched weekly (scheduled AMI build)
# Kernel upgrades: quarterly (scheduled, tested, canaried)

# === Kernel rollout procedure ===
# 1. Build AMI with new kernel
# 2. Test in dev environment (automated)
# 3. Canary fleet: 1% of hosts, run for 24 hours
# 4. Watch metrics: kernel panics, OOM events, application latency
# 5. Expand to 10%, watch 24 hours
# 6. Expand to 50%, watch 12 hours
# 7. Full rollout
# Rollback: deploy previous AMI (instant)

# === Black screen of death: live patching ===
# Some FAANGs use live patching (Kpatch, Ksplice) for critical CVEs
# Patch the kernel without rebooting:
sudo kpatch load patch.ko
# kpatch list      # Show loaded patches
# Live patching can only fix function-level changes, not data
structure changes
```

Observability Infrastructure

At FAANG scale, you can't SSH into individual servers to check 'top'. **Every metric is collected, aggregated, and alerted on automatically.** The observability stack at FAANG typically includes: a metrics system (Prometheus/Thanos), a logging system (Elasticsearch/Splunk), a tracing system (Jaeger), and a

profiling system (continuous profiling with eBPF). Every new Linux feature or configuration change is validated by watching these metrics.

```
# === What's monitored on every server ===
# CPU: utilization per core, context switches, load average, steal
time
# Memory: total, used, available, page cache, swap, OOM events
# Disk: per-device IOPS, latency (await), queue depth, utilization,
errors
# Network: bandwidth per interface, errors, drops, retransmits
# Kernel: slab usage, file handles, inode count, conntrack table
# Process: top 5 by CPU, top 5 by memory, zombie count

# === Sampling vs comprehensive ===
# At 10K+ servers, collecting EVERYTHING is expensive
# Strategy:
#   - Per-second: CPU, memory, disk, network (high-cardinality,
low-retention)
#   - Per-minute: process list, connection stats (medium-
cardinality)
#   - Per-hour: filesystem usage, inode count (low-cardinality,
long-retention)
#   - On-event: OOM, kernel panic, process crash (immediate alert)

# === Standard dashboards ===
# Every server type has a standard dashboard:
# "EC2 c5.4xlarge General Purpose" dashboard:
#   - Row 1: CPU (per-core), Memory, Load, Context Switches
#   - Row 2: Disk (per-volume IOPS, latency), Network (throughput,
errors)
#   - Row 3: Top processes (CPU, memory), Zombie count
#   - Row 4: Kernel (slab, file handles, conntrack)
#   - Row 5: Application-specific (varies by service)

# === Automated remediation ===
# Some issues are auto-remediated without human involvement:
# - Instance unhealthy → auto-scaling replaces it
# - Disk usage > 90% → log rotation + alert if still high
# - OOM kill → restart container, increase memory limit, alert
```

```
# - Kernel panic → instance replaced, crash dump uploaded for analysis

# === Pod/container-level monitoring ===
# For Kubernetes clusters: per-pod metrics (CPU, memory, network)
# Container-level: cgroup metrics, not host metrics
# Tools: cAdvisor, kube-state-metrics, metrics-server
```

Capacity Planning & Linux Resource Management

At FAANG scale, capacity planning is a continuous process. **You don't guess how many servers you need — you model it based on traffic forecasts, historical trends, and per-server benchmarks.** Linux resource management feeds directly into capacity models: how much overhead does the OS consume? What's the performance profile of each instance type? How does kernel version affect throughput?

```
# === Per-server resource budget ===
# For a c5.4xlarge (16 vCPU, 32 GB RAM):
# OS overhead: ~1 GB RAM, ~0.5 vCPU
# Monitoring agent: ~200 MB RAM, ~0.1 vCPU
# Security agent: ~300 MB RAM, ~0.1 vCPU
# Logging agent: ~200 MB RAM, ~0.05 vCPU
# Available for application: ~30 GB RAM, ~15 vCPU
# These numbers are measured, not guessed – from production profiling

# === Right-sizing methodology ===
# 1. Profile the application (perf, flamegraphs, memory profiling)
# 2. Run load tests at increasing instance sizes
# 3. Find the 'knee' in the cost/performance curve
# 4. Add 20% headroom for traffic spikes
# 5. Validate in production with auto-scaling
```

```
# === Instance family selection ===
# Compute-optimized (C series): CPU-bound apps, high perf/$, less
memory
# Memory-optimized (R series): in-memory caches, databases
# General-purpose (M series): balanced, default choice
# GPU (P/G series): ML training, inference
# Storage-optimized (I series): high local disk I/O

# === Linux-specific capacity factors ===
# - Kernel version: newer kernels have better scheduler, eBPF,
io_uring
# - Filesystem: XFS vs ext4 affects max throughput
# - NUMA: memory locality affects latency (use numactl)
# - Huge pages: reduce TLB misses, improve memory-intensive
workloads
# - Network: ENA vs Intel vs eBPF affects max connections

# === Forecasting ===
# Model: traffic → requests per second → CPU time per request →
total CPU → servers
# Daily: trend + seasonality → capacity needed
# Weekly: batch jobs, backups, data pipeline → peak usage
# Yearly: growth rate + new features → procurement plan

# === Headroom management ===
# Target: 60-70% utilization at peak (leaves room for traffic
spikes)
# Auto-scaling trigger: 70% CPU for 5 minutes (scale up)
# Auto-scaling cooldown: 10 minutes (let metrics stabilize)
# Over-provisioning budget: 10-20% for safety
```

FAANG Interview Q&A

Q: 'How would you design a Linux server fleet management system for 50,000 servers?'

A: 'Key components: (1) Golden AMI pipeline — Packer to build,

automated testing, versioned AMIs. (2) Configuration management — Ansible/Chef for any post-boot config, with idempotency and drift detection. (3) Fleet orchestration — AWS SSM or equivalent for running commands across the fleet without SSH. (4) Patch management — automated weekly patching via AMI rebuild, not in-place upgrades. (5) Monitoring — Prometheus + Thanos for metrics, centralized logging, and alerting with proper SLOs. (6) Instance lifecycle — auto-scaling groups manage instances, not humans. (7) Access control — bastion hosts, short-lived certificates, no direct SSH, all sessions logged. (8) Compliance — automated CIS scanning, auditd forwarding, compliance dashboards. The key principle: treat servers as cattle, not pets. If a server is broken, don't fix it — replace it.'

Q: 'Your team is migrating from Ubuntu 20.04 to Ubuntu 24.04 across 10,000 servers. What's your plan?'

A: 'Phase 1 — Assessment: catalog differences (kernel version, glibc, Python 3.8→3.12, OpenSSL), identify compatibility issues in our applications, test each service against the new OS. Phase 2 — AMI pipeline: build Golden AMI with Ubuntu 24.04, update all agents and configs. Phase 3 — Canary: deploy to 1% of non-production hosts, run integration tests for 1 week. Phase 4 — Staged rollout: 10% → 25% → 50% → 100% in non-prod. Phase 5 — Production canary: 1% of prod for 2 weeks with monitoring dashboards. Phase 6 — Full prod rollout: 10% per day, with automated rollback at any sign of regression. Phase 7 — Cleanup: decommission remaining 20.04 AMIs, update runbooks and documentation. This takes 3-4 months from start to finish.'

Q: 'How do you handle a kernel vulnerability (e.g., Dirty Pipe, Log4Shell-level but kernel) across the fleet?'

A: 'First, assess blast radius: which instances are affected (kernel version range, exposed attack surface). Second, contain: if possible, mitigate with sysctl/seccomp/iptables while the patch is being prepared. Third, fast-track patch: begin emergency AMI build with patched kernel, bypass normal staging (but still test on a small canary). Fourth, deploy: emergency automated rollout — canary for 1 hour, then full fleet rollout over 8-12 hours (vs typical 1-week rollout). Fifth, post-mortem: how did this CVE get through our vulnerability scanning? Why wasn't it caught earlier? At FAANG, there is a 'Security Incident Response' team that coordinates kernel CVEs — the response team includes security engineers, Linux kernel engineers, and the fleet operations team.'

References & Further Reading

- **Packer Documentation** — Tool for building Golden AMIs and machine images
- **Amazon Linux 2023** — AWS cloud-optimized Linux distribution
- **Google SRE Book: Eliminating Toil** — The philosophy behind automating fleet operations

FAANG Architect's Note: The biggest mindset shift when moving from single-server Linux to fleet-scale Linux is: *you never touch individual servers*. At FAANG, I went months without SSH'ing into a server — because everything was automated. When I did SSH in, it was during a critical incident where automation had failed, and the pressure was intense. The engineers who succeed at this scale are not the ones who know every obscure Linux command — they're the ones who design systems that don't need those commands. They build pipelines that produce consistent images, monitoring that catches issues before they page someone, and automation that fixes the common problems without human intervention. The Linux skills matter most when that automation fails — and it will, eventually.

Production Tip: The most important metric in fleet management is **time to recover** — how long does it take to go from 'pager alert' to 'system healthy'? At FAANG, the target is under 5 minutes for automated recovery and under 30 minutes for manual recovery. Every investment in AMI pipelines, monitoring, and runbooks should be measured against this metric. If your 'rebuild a server from scratch' runbook takes 4 hours, it needs to be automated. If your 'diagnose a slow node' process is 'SSH in and check top', it needs a dashboard. Measure your recovery time, then systematically reduce it.

CHAPTER 30

FAANG Linux Interview Q&A

Containers & SRE · 6 topics

Linux knowledge is tested throughout the FAANG interview process — from initial phone screens to system design to on-site 'bar raiser' rounds. This guide covers the types of Linux questions you'll encounter across different roles (SRE, Infrastructure, Backend, Systems), with answers that demonstrate senior-level depth.

FAANG

Linux

interview

preparation

system design

debugging

Role-Specific Linux Expectations

Not all roles require the same depth of Linux knowledge.

SRE/Infrastructure roles expect deep system-level knowledge.

Backend/ML roles expect practical proficiency.

Frontend/Product roles need basic familiarity. At FAANG, the interview rubric clearly expects different levels — demonstrating appropriate depth for the role is key.

```
# === Role expectations ===  
  
# SRE / Infrastructure Engineer:  
# - Kernel internals: syscalls, cgroups, namespaces, scheduler,  
  memory mgmt  
# - Performance: perf, flamegraphs, USE method, profiling  
# - Networking: full stack (TCP, DNS, HTTP, iptables/nftables, BGP
```

```

basics)
# - Storage: filesystems, LVM, NFS, RAID, distributed storage
# - Security: SELinux, seccomp, AppArmor, auditd, hardening
# - Containers: cgroups v2, OCI runtime, Docker internals,
  Kubernetes
# - Debugging: strace, gdb, crash dumps, core analysis
# Expected: Can debug any production issue and optimize system
  performance

# Backend / Software Engineer:
# - Practical: file operations, processes, signals, networking
  basics
# - Tools: SSH, git, grep, awk, sed, jq, curl
# - Scripting: bash scripting, error handling, job control
# - Containers: Dockerfiles, container basics
# Expected: Can develop and deploy services independently

# ML / Data Engineer:
# - Practical: package management, environment setup, Docker
# - Performance: GPU monitoring (nvidia-smi), memory profiling
# - Storage: filesystem basics, data pipeline I/O patterns
# Expected: Can set up and run ML workloads efficiently

# === How interviewers assess Linux depth ===
# "Tell me about a time you debugged a production issue"
# Junior answer: 'I restarted the server'
# Mid answer: 'I checked the logs and found an OOM error'
# Senior answer: 'I checked cgroup memory.events, found OOM kills,
#   analyzed the heap dump, identified a memory leak in the
  connection pool,
#   and added monitoring on cgroup memory pressure to prevent
  recurrence'

# === Sample question progression ===
# Basic: 'How do you check disk space?' -> df -h
# Intermediate: 'How do you find large files?' -> du -sh /* | sort
  -rh
# Advanced: 'A server has 50% disk free but can't write files.
  Why?' -> df -i (inodes)
# Expert: 'How does OOM killer select which process to kill?' ->
  oom_score algorithm

```

Common Interview Questions by Topic

These questions appear across FAANG Linux interviews. **The key is not just knowing the right command, but explaining why it works and how you'd use it in a real scenario.** For each question, interviewers look for: understanding of the underlying concept, practical experience, and systematic thinking.

```
# === PROCESSES ===
# Q: What are the Linux process states?
# A: R (running/runnable), S (sleeping), D (uninterruptible sleep),
#    Z (zombie), T (stopped), t (tracing stop), X (dead)
#    D state is important: process waiting on I/O, can't be killed,
#    usually indicates NFS/disk issue

# Q: What is a zombie process? How do you clean them?
# A: Process that has exited but parent hasn't called wait().
#    Can't be killed (already dead). Kill the parent to let init
#    reap them.
#    Check with: ps aux | grep 'Z'

# === MEMORY ===
# Q: Explain the output of 'free -h'. What's the difference between
#    'free', 'available', and 'buff/cache'?
# A: 'free' = completely unused RAM. 'buff/cache' = disk cache
#    (reclaimable).
#    'available' = estimated memory for new apps (= free +
#    reclaimable cache).
#    Linux uses free RAM for cache – this is normal, not 'wasted'.

# Q: What triggers OOM killer?
# A: When the kernel can't allocate memory and swap is full or
#    disabled.
#    The kernel selects a victim based on oom_score (RSS + swap +
#    page tables).
#    You can protect processes with oom_score_adj = -1000.

# === STORAGE ===
```

```

# Q: A disk is at 100% I/O utilization. How do you fix it?
# A: First, identify the source: iostat -x for device stats, iotop
    -oP for process.
#   Then: is it legitimate load (scale up) or a problem (missing
    index,
#   log flood, backup running during peak)?

# Q: What's the difference between ext4 and XFS?
# A: ext4: good general-purpose, reliable, widely compatible.
#   XFS: better for large files, parallel I/O, online defrag.
#   XFS is default on RHEL/Amazon Linux; ext4 on Ubuntu.

# === NETWORKING ===
# Q: How do you check if a port is listening?
# A: ss -tlnp | grep :PORT (modern, fast, detailed)
#   netstat -tlnp (legacy, may need net-tools)

# Q: A user says 'the network is slow'. How do you diagnose?
# A: Start with mtr to trace path + latency.
#   Check interface errors: ip -s link.
#   Check TCP retransmits: netstat -s | grep retrans.
#   Check bandwidth: sar -n DEV 1.
#   'Slow network' is usually an application issue, not the
    network.

# === CONTAINERS ===
# Q: How does Docker isolate processes?
# A: Namespaces (mount, PID, net, IPC, UTS, user) + cgroups (CPU,
    mem, I/O, PIDs) +
    seccomp + LSM

# Q: A container is OOM-killed but 'free -h' inside shows free
    memory.
#   Why?
# A: /proc/meminfo shows host memory, not cgroup limit.
#   Check /sys/fs/cgroup/memory.current and memory.events.

# === DEBUGGING ===
# Q: A process is using 100% CPU. How do you find the hot path?
# A: perf top -p <PID> for live sampling,
#   perf record -g for call graphs,
#   generate a flamegraph.

```

```
# Q: What do you check in the first 60 seconds of an incident?
# A: uptime, dmesg | tail, vmstat 1, mpstat -P ALL 1, pidstat 1,
#    iostat -xz 1, free -m, sar -n DEV 1, sar -n TCP,ETCP 1, top
```

System Design with Linux Constraints

System design interviews often include Linux-specific constraints.

At FAANG, you're expected to design systems that work within Linux's capabilities and limitations. Understanding file descriptor limits, epoll vs select, memory-mapped files, and Linux-specific optimization is what separates a 'works on my laptop' design from a 'runs at FAANG scale' design.

```
# === File descriptor limits ===
# Default ulimit -n = 1024 (far too low for a web server)
# Production: ulimit -n 65536 or more
# 'Too many open files' = ulimit exhaustion
# Also check: system-wide limit (cat /proc/sys/fs/file-max)

# === Epoll vs Select/Poll ===
# epoll is O(1) for event notification – scales to millions of
connections
# select/poll are O(n) – good for < 1000 connections
# This is why nginx/Node.js use epoll (via libuv/libevent)
# Check: cat /proc/sys/fs/epoll/max_user_watches

# === Memory-mapped files (mmap) ===
# mmap allows direct file access via memory operations
# Faster than read/write for random access
# Used by: databases (PostgreSQL shared buffers),
#         Kafka (page cache + mmap for log segments)

# === Linux-specific performance patterns ===
# - SO_REUSEPORT: multiple processes bind to same port (nginx,
haproxy)
# - SO_BUSY_POLL: busy-poll for lower latency (Redis)
```

```
# - TCP_DEFER_ACCEPT: delay accept() until data arrives (web
servers)
# - O_DIRECT: bypass page cache for databases (PostgreSQL, MySQL)
# - splice/sendfile: zero-copy data transfer

# === Design question with Linux constraints ===
# "Design a logging system that handles 100K writes/sec on a single
server"
# Linux considerations:
# - File write buffering: use buffered I/O (write() is fast, fsync
is slow)
# - Group commits: batch fsync every N ms (like PostgreSQL WAL)
# - Use append-only files (no random writes)
# - Consider filesystem: XFS for large sequential writes
# - Use a separate disk for logs (don't compete with data)
# - splice() for zero-copy from producer to socket
# - ulimit -n for file descriptors (log rotation creates new files)
```

Common Pitfalls & Anti-Patterns

Knowing what NOT to do is as important as knowing what to do.

At FAANG, certain patterns are banned in production — they've caused enough outages that they're now in the 'never do this' category.

```
# === NEVER DO THESE IN PRODUCTION ===

# 1. Don't disable swap entirely
# swap provides a safety buffer against OOM
# Set vm.swappiness=10 (swap only when necessary) instead of 0
# But also: on modern systems with cgroups, swap may be disabled
safely

# 2. Don't drop caches in production
# echo 3 > /proc/sys/vm/drop_caches destroys page cache
# This causes massive I/O as everything must be re-read from disk
# Only do this in controlled benchmarks
```

```
# 3. Don't use kill -9 as the first resort
# Always SIGTERM first (graceful shutdown)
# Use SIGKILL only when SIGTERM is ignored

# 4. Don't disable SELinux/AppArmor
# 'I disabled it to make it work' = losing defense in depth
# Instead: put it in permissive mode, collect denials, fix them,
enable

# 5. Don't chmod 777
# If permissions don't work, fix them properly
# 777 means anyone can modify any file – including attackers

# 6. Don't run containers/applications as root
# If a root process is compromised, the attacker has full system
access
# Always create a service account or use USER in Dockerfile

# 7. Don't rely on /proc/meminfo for container memory
# Use cgroup memory metrics (memory.current, memory.events)
# /proc/meminfo shows host memory, not container limits

# 8. Don't skip log rotation
# 'I'll clean it up later' = pager at 3 AM when disk is full
# Configure logrotate BEFORE deploying

# 9. Don't disable seccomp 'for performance'
# The performance cost of seccomp is negligible
# The security cost of disabling it is significant

# 10. Don't make changes without a rollback plan
# Before any production change: 'How do I undo this?'
# If you can't answer, don't make the change
```

References & Further Reading

- **Systems Performance: Enterprise and the Cloud** —
Brendan Gregg, the definitive Linux performance reference

- **UNIX and Linux System Administration Handbook** — Comprehensive admin reference
- **FAANG Linux Systems Interview Questions** — Community-curated question bank

FAANG Architect's Note: Linux interview questions at FAANG test more than memorization — they test your mental model of how the system works. When I interview candidates, I don't care if they remember the exact flag for `iostat`. I care about: (1) Can they systematically debug a problem? (2) Do they understand what the kernel is doing? (3) Have they actually fixed real production issues? The best answer to 'How do you debug high CPU?' is not 'I use `perf`' — it's a story about a time you used `perf` to find and fix a real issue. Build your experience, practice explaining your debugging process out loud, and always understand the 'why' behind the command.

Interview Tip: During the interview, think out loud. Interviewers can't read your mind. Say: 'I'd check uptime first to see load averages. If load is high, I'd run `mpstat` to check per-CPU balance. If one CPU is pegged, I'd use `perf top` to find the hot function...' This shows your systematic approach. Even if you're wrong about the specific tool, demonstrating the methodology is better than sitting silently and then blurting out a final answer.

Index

This book covers the following topics and commands for quick reference.

A

- AppArmor
- apt
- auditd
- authentication
- awk

B

- bash
- basics
- Bazel
- bcc
- benchmarking
- BPF
- BPF & eBPF Deep Dive
- bpfttrace
- build

C

- capacity planning
- cgroups
- cgroups & Namespaces
- chroot
- CLI
- CLI Fundamentals
- CNI
- compiler

- container lifecycle

- containers

- CRI

- CSI

- CUDA

D

- debugging

- disk management

- dnf

- DNS

- Docker

- Docker & Container Lifecycle

E

- eBPF

F

- FAANG

- FAANG Linux Interview Q&A

- File Permissions & ACLs

- filesystem

- Filesystem Hierarchy Standard

- filesystem repair

- flamegraphs

- fleet management

- ftrace

G

git
Git Version Control
Golden AMI
GPU
GPU Computing on Linux
grep
groups
GRUB

H

hardening
history
History of Linux & Unix
huge pages

I

I/O
I/O Stack & Filesystem
Performance
incident response
init
interview
iostat
iptables

J

journald

K

kernel
Kernel Architecture & Syscalls
kernel modules
kubelet
Kubernetes

Kubernetes for Linux Engineers

L

linker
Linux
Linux at FAANG Scale
Linux Build & Toolchain
Linux Networking
logging
LVM

M

machine learning
memory
Memory Management & OOM
Handling
MIG
monitoring
Monitoring & Observability

N

namespaces
networking
NVIDIA

O

observability
OCI
OOM
optimization

P

Package Management
package management

page cache
PAM
perf
performance
Performance Tuning & Profiling
permissions
preparation
Process Management & Signals
processes
production debugging
Production Troubleshooting
profiling
Prometheus

R

recovery
rescue

S

scale
scheduler
seccomp
security
Security Hardening
sed
SELinux
service management
Shell Scripting Basics
shell-scripting
signals
SRE

SSH
SSH: Secure Shell
storage
storage performance
Storage, Filesystems & Disk Management
sudo
syscalls
system design
System Rescue & Recovery
systemd
systemd: The Modern Init System

T

tcpdump
Text Processing: grep, sed, awk
text-processing
toolchain
troubleshooting
tunneling

U

users
Users, Groups & Authentication

V

version-control

X

XDP

Y

yum